

C basics

<https://www.embarcadero.com/free-tools/dev-cpp/free-download/thank-you>

How to install C

There are many compilers available for c and c++. You need to download any one. Here, we are going to use **Turbo C++**. It will work for both C and C++. To install the Turbo C software, you need to follow following steps.

1. Download Turbo C++
2. Create turboc directory inside c drive and extract the tc3.zip inside c:\turboc
3. Double click on install.exe file
4. Click on the tc application file located inside c:\TC\BIN to write the c program

1) Download Turbo C++ software

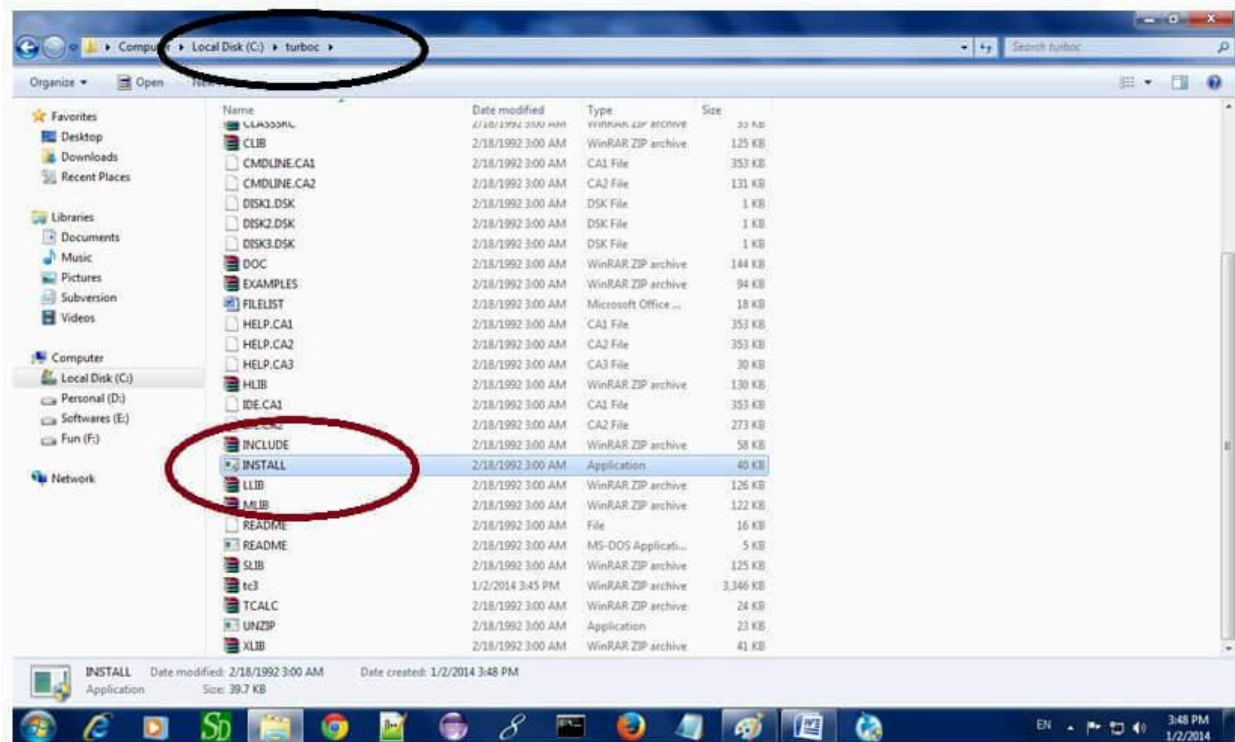
You can download turbo c++ from many sites. [download Turbo c++](#)

2) Create turboc directory in c drive and extract the tc3.zip

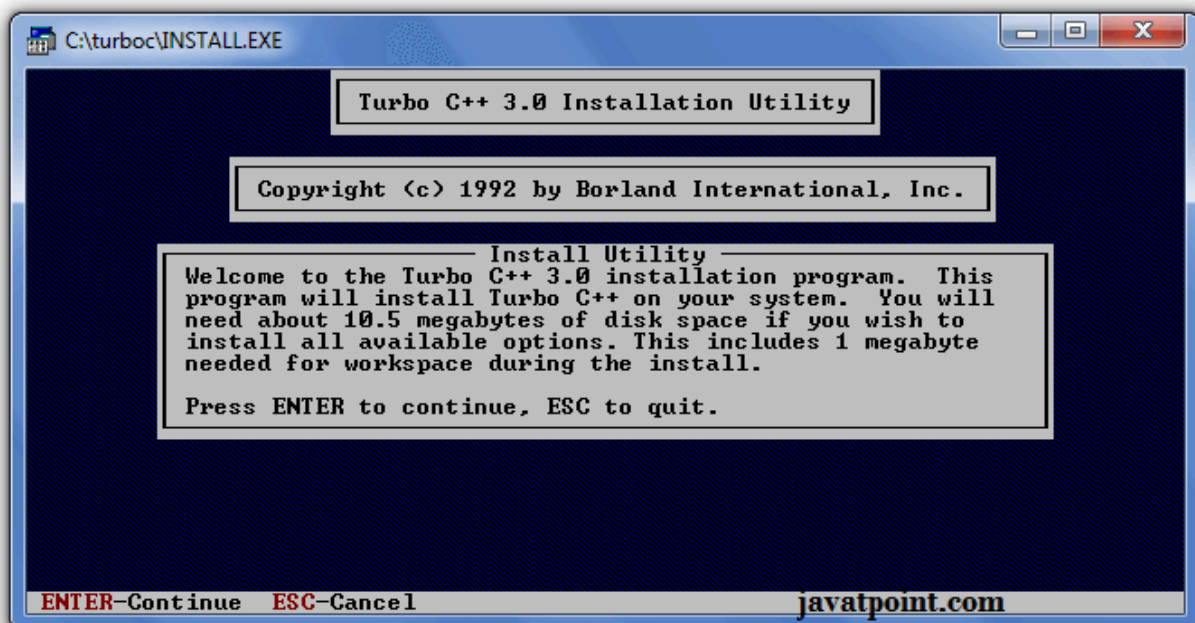
Now, you need to create a new directory turboc inside the c: drive. Now extract the tc3.zip file in c:\truboc directory.

3) Double click on the install.exe file and follow steps

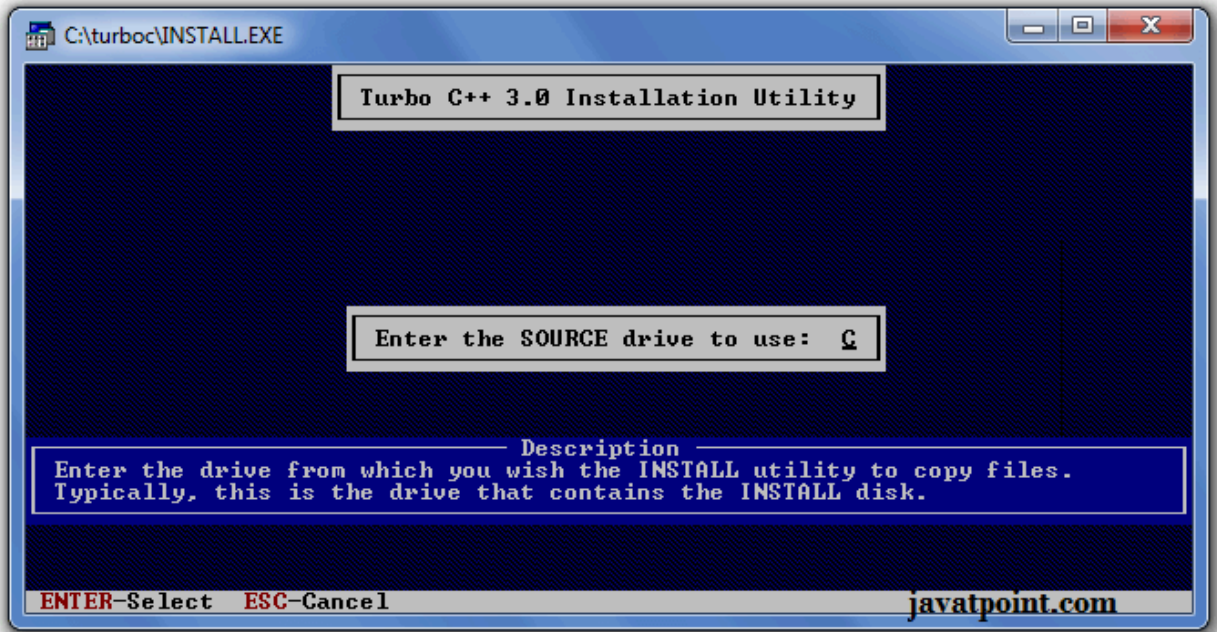
Now, click on the install icon located inside the c:\turboc



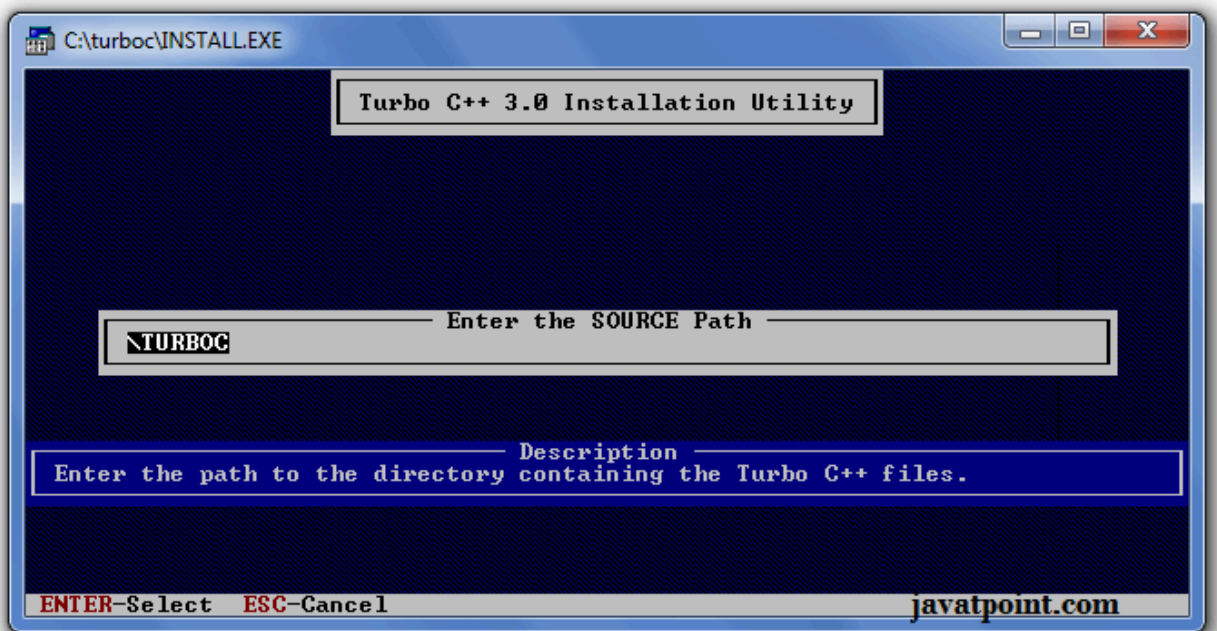
It will ask you to install c or not, press enter to install.



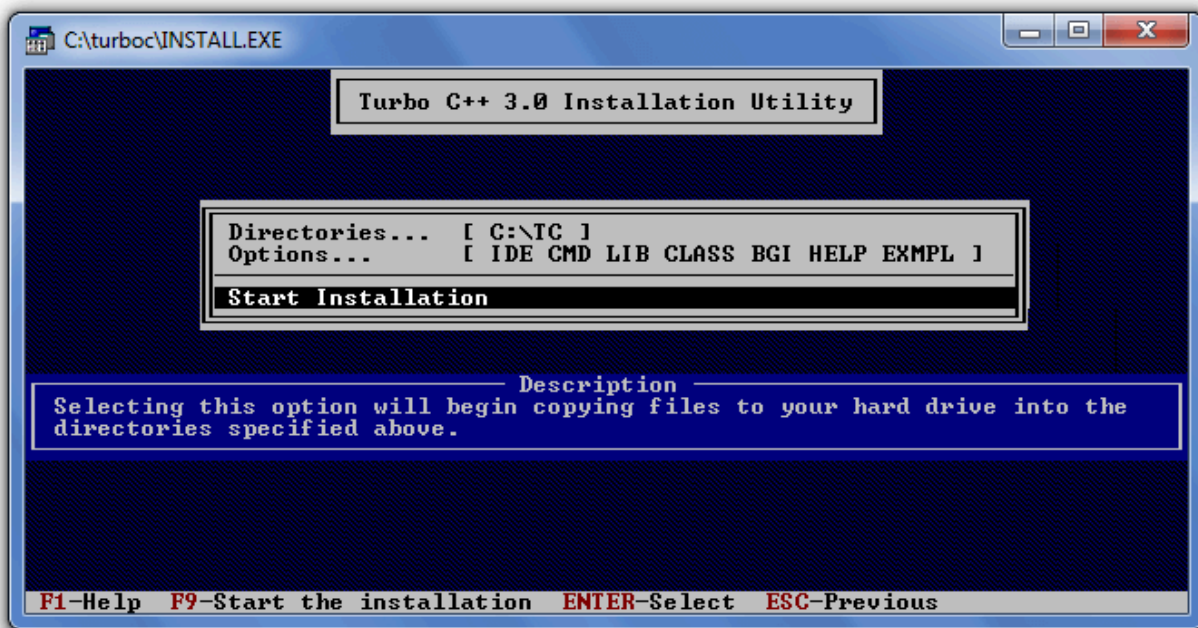
Change your drive to c, press c.



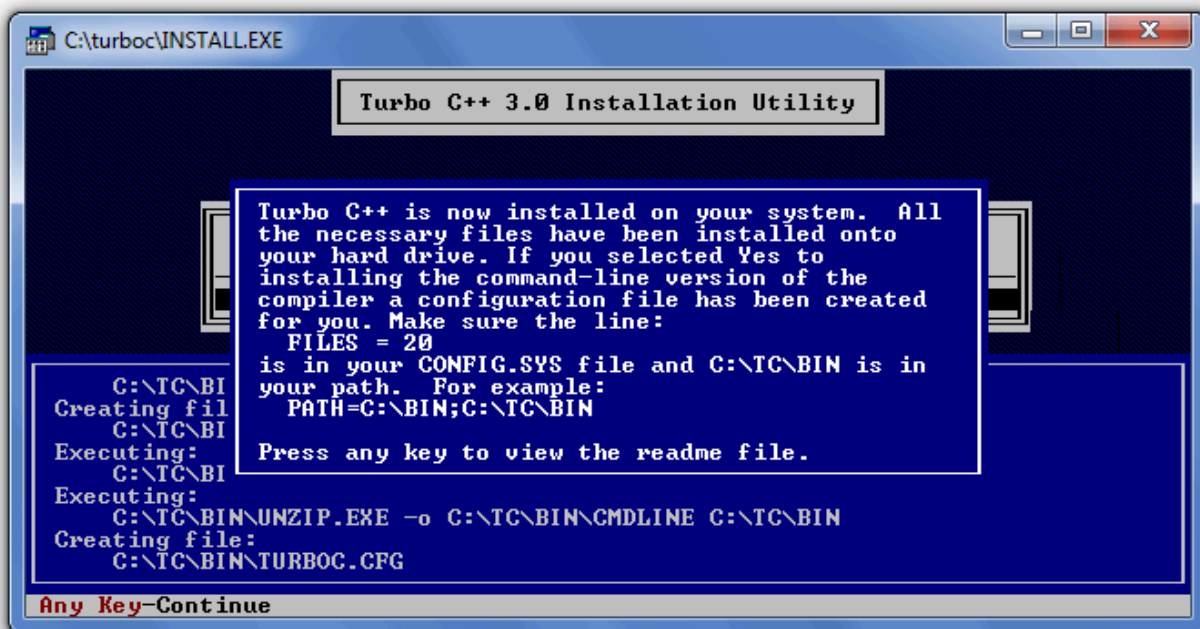
Press enter, it will look inside the c:\turboc directory for the required files.



Select Start installation by the down arrow key then press enter.

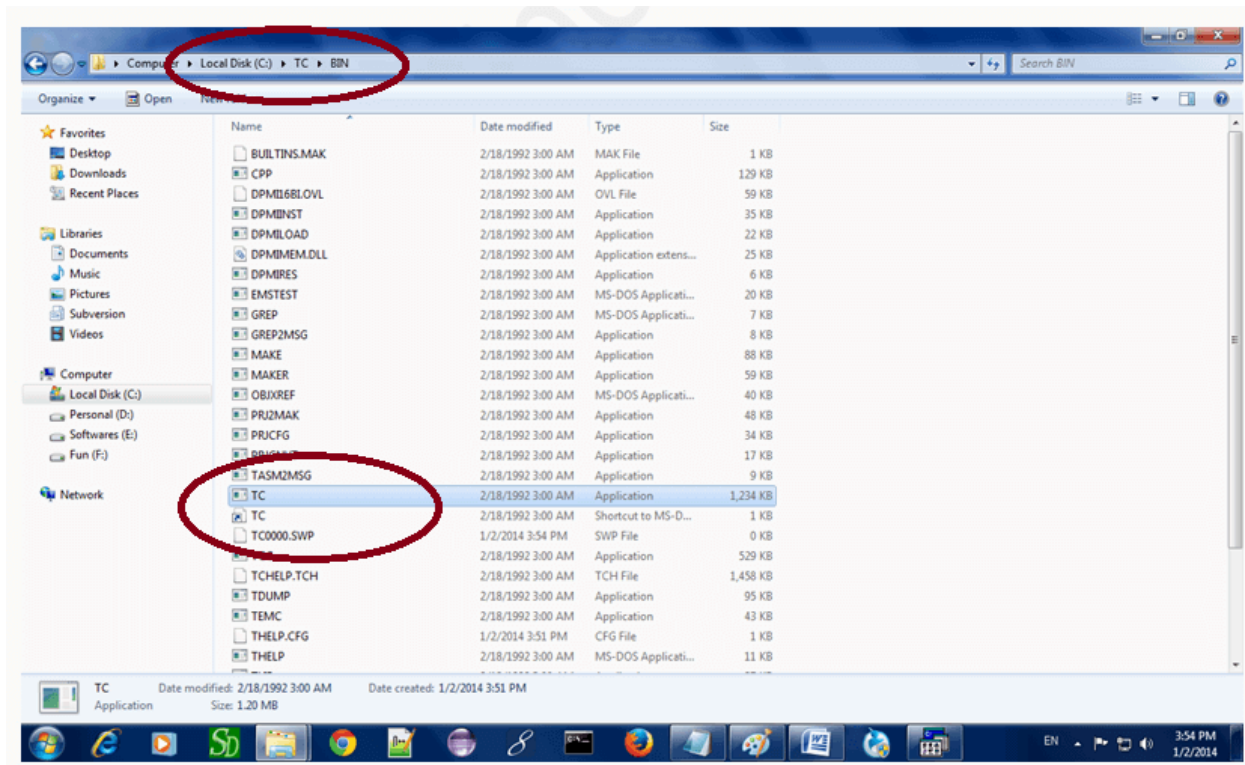


Now C is installed, press enter to read documentation or close the software.



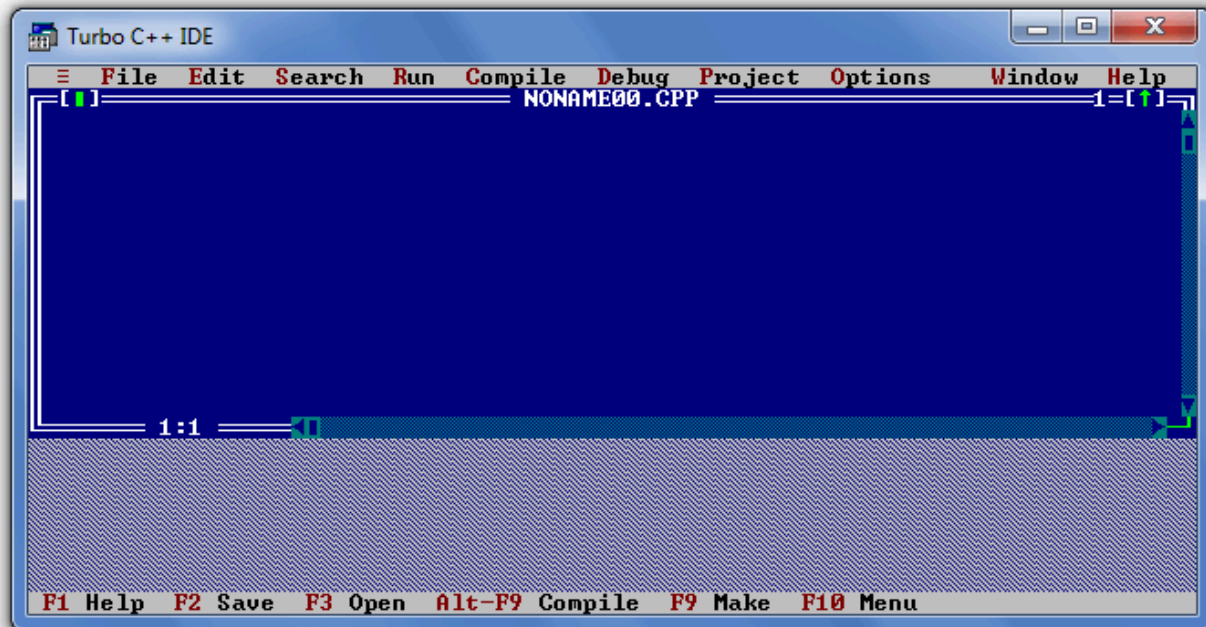
4) Click on the tc application located inside c:\TC\BIN

Now double click on the tc icon located in c:\TC\BIN directory to write the c program.



In windows 7 or window 8, it will show a dialog block to ignore and close the application because fullscreen mode is not supported. Click on Ignore button.

Now it will showing following console.



First C Program

Before starting the abcd of C language, you need to learn how to write, compile and run the first c program.

To write the first c program, open the C console and write the following code:

```
1. #include <stdio.h>
2. int main(){
3. printf("Hello C Language");
4. return 0;
5. }
```

#include <stdio.h> includes the **standard input output** library functions. The **printf()** function is defined in **stdio.h**.

int main() The **main()** function is the **entry point of every program** in c language.

printf() The **printf()** function is **used to print data** on the console.

return 0 The return 0 statement, returns execution status to the OS. The 0 value is used for successful execution and 1 for unsuccessful execution.

How to compile and run the c program

There are 2 ways to compile and run the c program, by menu and by shortcut.

By menu

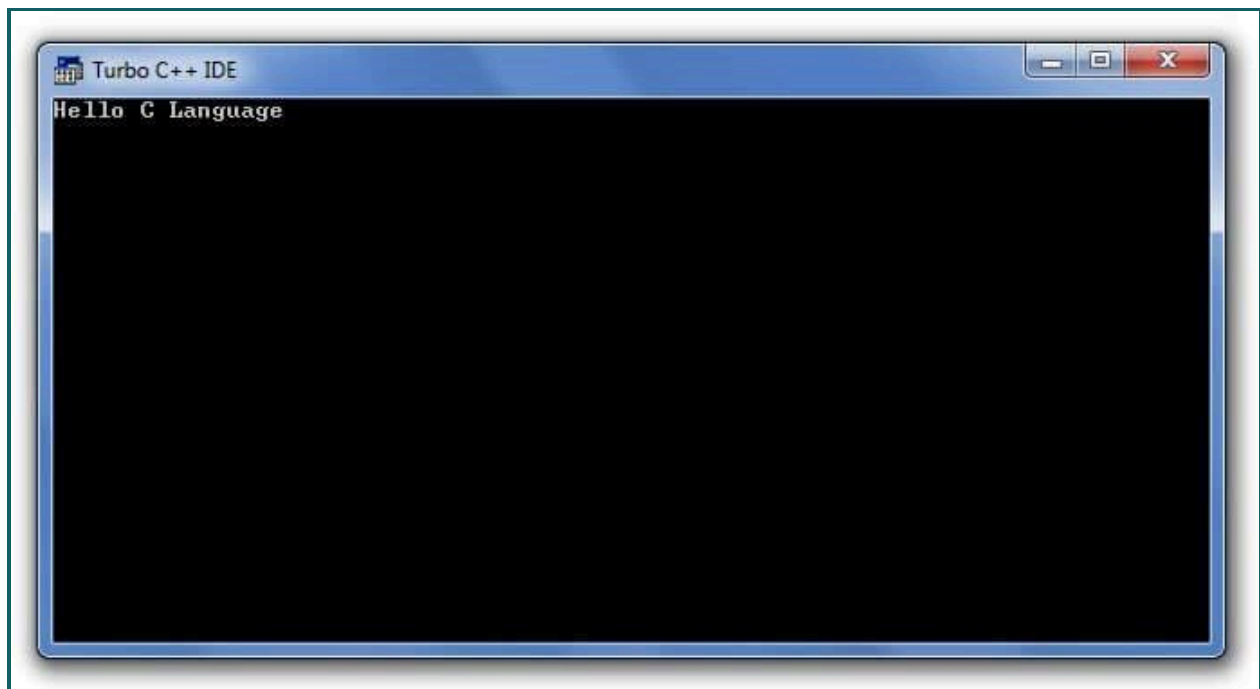
Now **click on the compile menu then compile sub menu** to compile the c program.

Then **click on the run menu then run sub menu** to run the c program.

By shortcut

Or, **press ctrl+f9** keys compile and run the program directly.

You will see the following output on user screen.

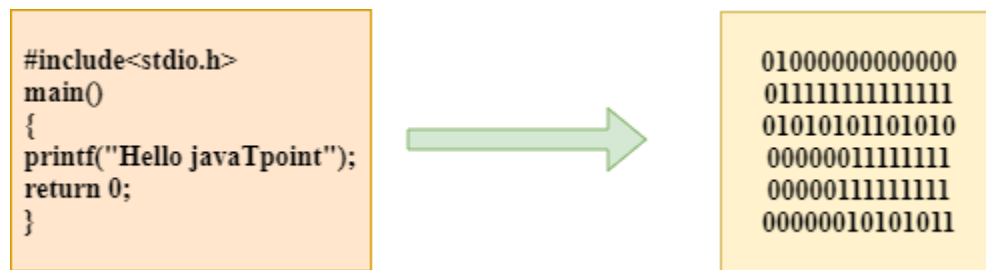


You can view the user screen any time by pressing the **alt+f5** keys.

Compilation process in c

What is a compilation?

The compilation is a process of converting the source code into object code. It is done with the help of the compiler. The compiler checks the source code for the syntactical or structural errors, and if the source code is error-free, then it generates the object code.

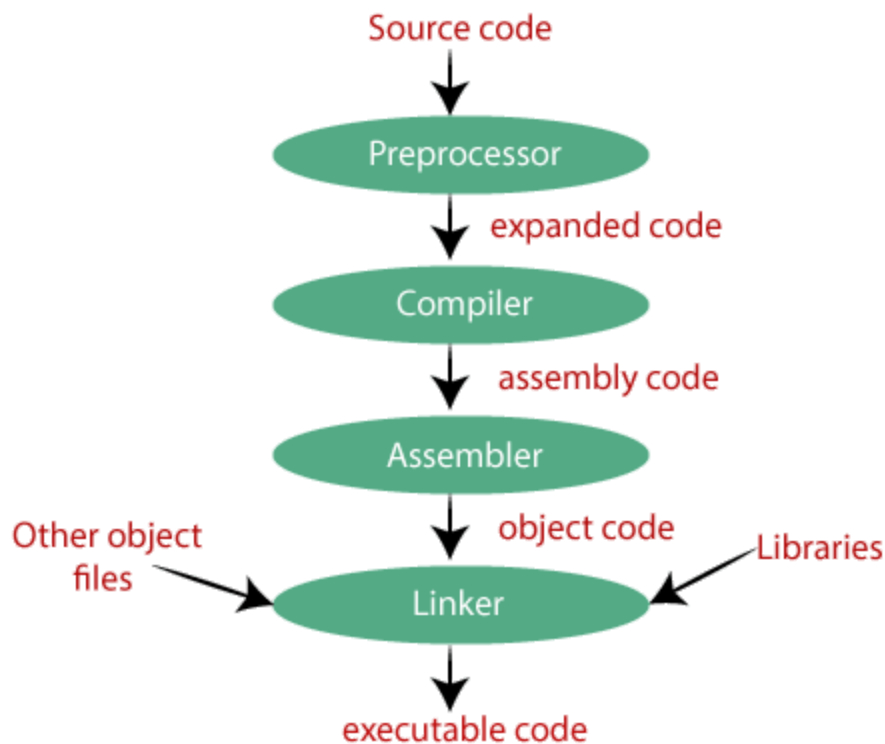


The c compilation process converts the source code taken as input into the object code or machine code. The compilation process can be divided into four steps, i.e., Pre-processing, Compiling, Assembling, and Linking.

The preprocessor takes the source code as an input, and it removes all the comments from the source code. The preprocessor takes the preprocessor directive and interprets it. For example, if `<stdio.h>`, the directive is available in the program, then the preprocessor interprets the directive and replace this directive with the content of the 'stdio.h' file.

The following are the phases through which our program passes before being transformed into an executable form:

- **Preprocessor**
- **Compiler**
- **Assembler**
- **Linker**



Preprocessor

The source code is the code which is written in a text editor and the source code file is given an extension ".c". This source code is first passed to the preprocessor, and then the preprocessor expands this code. After expanding the code, the expanded code is passed to the compiler.

Compiler

The code which is expanded by the preprocessor is passed to the compiler. The compiler converts this code into assembly code. Or we can say that the C compiler converts the pre-processed code into assembly code.

Assembler

The assembly code is converted into object code by using an assembler. The name of the object file generated by the assembler is the same as the source file. The extension of the object file in DOS is '.obj,' and in UNIX, the extension is '.o'. If the name of the source file is '**hello.c**', then the name of the object file would be 'hello.obj'.

Linker

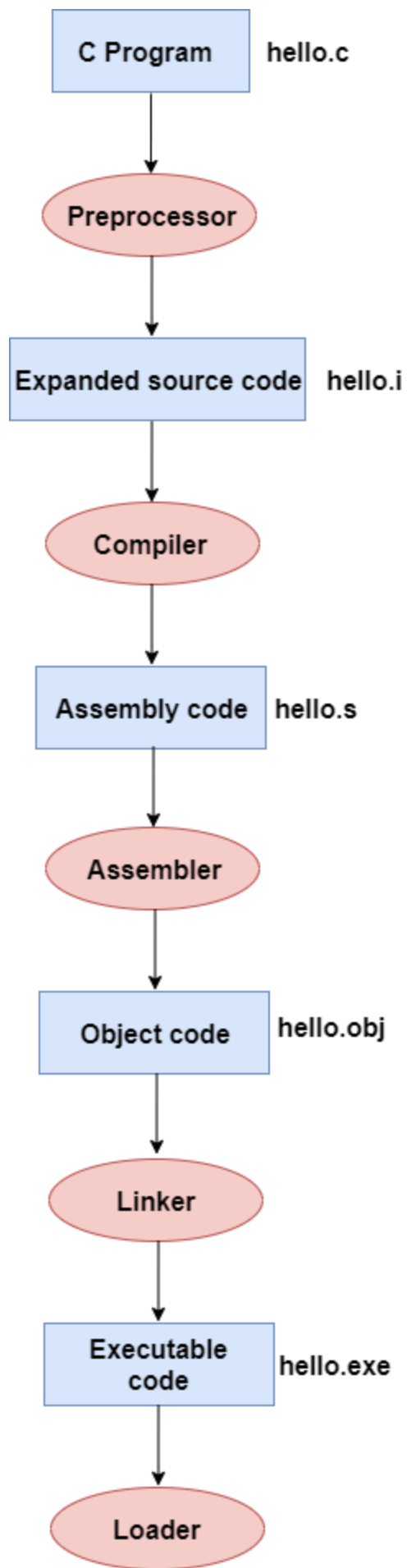
Mainly, all the programs written in C use library functions. These library functions are pre-compiled, and the object code of these library files is stored with '.lib' (or '.a') extension. The main working of the linker is to combine the object code of library files with the object code of our program. Sometimes the situation arises when our program refers to the functions defined in other files; then linker plays a very important role in this. It links the object code of these files to our program. Therefore, we conclude that the job of the linker is to link the object code of our program with the object code of the library files and other files. The output of the linker is the executable file. The name of the executable file is the same as the source file but differs only in their extensions. In DOS, the extension of the executable file is '.exe', and in UNIX, the executable file can be named as 'a.out'. For example, if we are using printf() function in a program, then the linker adds its associated code in an output file.

Let's understand through an example.

hello.c

1. `#include <stdio.h>`
2. `int main()`
3. `{`
4. `printf("Hello javaTpoint");`
5. `return 0;`
6. `}`

Now, we will create a flow diagram of the above program:



In the above flow diagram, the following steps are taken to execute a program:

- Firstly, the input file, i.e., **hello.c**, is passed to the preprocessor, and the preprocessor converts the source code into expanded source code. The extension of the expanded source code would be **hello.i**.
- The expanded source code is passed to the compiler, and the compiler converts this expanded source code into assembly code. The extension of the assembly code would be **hello.s**.
- This assembly code is then sent to the assembler, which converts the assembly code into object code.
- After the creation of an object code, the linker creates the executable file. The loader will then load the executable file for the execution.

printf() and scanf() in C

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

printf() function

The **printf() function** is used for output. It prints the given statement to the console.

The syntax of printf() function is given below:

1. printf("format string",argument_list);

The **format string** can be %d (integer), %c (character), %s (string), %f (float) etc.

scanf() function

The **scanf() function** is used for input. It reads the input data from the console.

1. `scanf("format string",argument_list);`

Program to print cube of given number

Let's see a simple example of c language that gets input from the user and prints the cube of the given number.

1. `#include<stdio.h>`
2. `int main(){`
3. `int number;`
4. `printf("enter a number:");`
5. `scanf("%d",&number);`
6. `printf("cube of number is:%d ",number*number*number);`
7. `return 0;`
8. `}`

Output

enter a number:5

cube of number is:125

The `scanf("%d",&number)` statement reads integer number from the console and stores the given value in number variable.

The `printf("cube of number is:%d ",number*number*number)` statement prints the cube of number on the console.

Program to print sum of 2 numbers

Let's see a simple example of input and output in C language that prints addition of 2 numbers.

1. `#include<stdio.h>`

```
2. int main(){
3. int x=0,y=0,result=0;
4.
5. printf("enter first number:");
6. scanf("%d",&x);
7. printf("enter second number:");
8. scanf("%d",&y);
9.
10.result=x+y;
11.printf("sum of 2 numbers:%d ",result);
12.
13.return 0;
14.}
```

Output

enter first number:9

enter second number:9

sum of 2 numbers:18

Variables in C

A **variable** is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

Let's see the syntax to declare a variable:

```
1. type variable_list;
```

The example of declaring the variable is given below:

1. `int a;`
2. `float b;`
3. `char c;`

Here, a, b, c are variables. The int, float, char are the data types.

We can also provide values while declaring the variables as given below:

1. `int a=10,b=20; //declaring 2 variable of integer type`
2. `float f=20.8;`
3. `char c='A';`

Rules for defining variables

- A variable can have alphabets, digits, and underscore.
- A variable name can start with the alphabet, and underscore only. It can't start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Valid variable names:

1. `int a;`
2. `int _ab;`
3. `int a30;`

Invalid variable names:

1. `int 2;`
2. `int a b;`
3. `int long;`

Types of Variables in C

There are many types of variables in c:

1. local variable
2. global variable
3. static variable
4. automatic variable
5. external variable

Local Variable

A variable that is declared inside the function or block is called a local variable.

It must be declared at the start of the block.

1. `void function1(){`
2. `int x=10; //local variable`
3. `}`

You must have to initialize the local variable before it is used.

Global Variable

A variable that is declared outside the function or block is called a global variable. Any function can change the value of the global variable. It is available to all the functions.

It must be declared at the start of the block.

1. `int value=20; //global variable`
2. `void function1(){`
3. `int x=10; //local variable`
4. `}`

Static Variable

A variable that is declared with the static keyword is called static variable.

It retains its value between multiple function calls.

1. `void function1(){`
2. `int x=10; //local variable`
3. `static int y=10; //static variable`
4. `x=x+1;`
5. `y=y+1;`
6. `printf("%d,%d",x,y);`
7. `}`

If you call this function many times, the **local variable will print the same value** for each function call, e.g. 11,11,11 and so on. But the **static variable will print the incremented value** in each function call, e.g. 11, 12, 13 and so on.

Automatic Variable

All variables in C that are declared inside the block, are automatic variables by default. We can explicitly declare an automatic variable using **auto keyword**.

1. `void main(){`
2. `int x=10; //local variable (also automatic)`
3. `auto int y=20; //automatic variable`
4. `}`

External Variable

We can share a variable in multiple C source files by using an external variable. To declare an external variable, you need to use **extern keyword**.

myfile.h

1. `extern int x=10; //external variable (also global)`

program1.c

1. `#include "myfile.h"`
2. `#include <stdio.h>`
3. `void printValue(){`
4. `printf("Global variable: %d", global_variable);`
5. `}`

C Programming Operators

An operator is a symbol that operates on a value or a variable. For example: + is an operator to perform addition.

C has a wide range of operators to perform various operations.

C Arithmetic Operators

An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

Operator	Meaning of Operator
+	addition or unary plus

-	subtraction or unary minus
*	multiplication
/	division
%	remainder after division (modulo division)

Example 1: Arithmetic Operators

```
// Working of arithmetic operators
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 9, b = 4, c;
```

```
    c = a+b;
```

```
    printf("a+b = %d \n", c);
```

```
    c = a-b;
```

```
    printf("a-b = %d \n", c);
```

```
    c = a*b;
```

```
    printf("a*b = %d \n", c);
```

```
    c = a/b;
```

```
    printf("a/b = %d \n", c);
```

```
    c = a%b;
```

```
printf("Remainder when a divided by b = %d \n",c);
```

```
return 0;
```

```
}
```

Output

```
a+b = 13
```

```
a-b = 5
```

```
a*b = 36
```

```
a/b = 2
```

```
Remainder when a divided by b=1
```

The operators `+`, `-` and `*` computes addition, subtraction, and multiplication respectively as you might have expected.

In normal calculation, $9/4 = 2.25$. However, the output is `2` in the program.

It is because both the variables `a` and `b` are integers. Hence, the output is also an integer. The compiler neglects the term after the decimal point and shows answer `2` instead of `2.25`.

The modulo operator `%` computes the remainder. When `a=9` is divided by `b=4`, the remainder is `1`. The `%` operator can only be used with integers.

Suppose `a = 5.0`, `b = 2.0`, `c = 5` and `d = 2`. Then in C programming,

```
// Either one of the operands is a floating-point number
```

```
a/b = 2.5
```

```
a/d = 2.5
```

```
c/b = 2.5
```

```
// Both operands are integers
```

```
c/d = 2
```

C Increment and Decrement Operators

C programming has two operators increment ++ and decrement -- to change the value of an operand (constant or variable) by 1.

Increment ++ increases the value by 1 whereas decrement -- decreases the value by 1. These two operators are unary operators, meaning they only operate on a single operand.

Example 2: Increment and Decrement Operators

```
// Working of increment and decrement operators
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, b = 100;
```

```
    float c = 10.5, d = 100.5;
```

```
    printf("++a = %d \n", ++a);
```

```
    printf("--b = %d \n", --b);
```

```
    printf("++c = %f \n", ++c);
```

```
    printf("--d = %f \n", --d);
```

```
return 0;
```

```
}
```

Run Code

Output

```
++a = 11
```

```
--b = 99
```

```
++c = 11.500000
```

```
--d = 99.500000
```

Here, the operators ++ and -- are used as prefixes. These two operators can also be used as postfixes like `a++` and `a--`. Visit [this page](#) to learn more about how [increment and decrement operators work when used as postfix](#).

C Assignment Operators

An assignment operator is used for assigning a value to a variable. The most common assignment operator is =

Operator	Example	Same as
=	a = b	a = b
+=	a += b	a = a+b

--	a -= b	a = a-b
*=	a *= b	a = a*b
/=	a /= b	a = a/b
%=	a %= b	a = a%b

Example 3: Assignment Operators

```
// Working of assignment operators
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 5, c;
```

```
    c = a;        // c is 5
```

```
    printf("c = %d\n", c);
```

```
    c += a;       // c is 10
```

```
    printf("c = %d\n", c);
```

```
    c -= a;       // c is 5
```

```
    printf("c = %d\n", c);
```

```
    c *= a;       // c is 25
```

```
    printf("c = %d\n", c);
```

```
    c /= a;       // c is 5
```

```
    printf("c = %d\n", c);
```

```
    c %= a;       // c = 0
```

```
    printf("c = %d\n", c);
```

```
return 0;
```

```
}
```

Run Code

Output

```
c = 5
```

```
c = 10
```

```
c = 5
```

```
c = 25
```

```
c = 5
```

```
c = 0
```

C Relational Operators

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Relational operators are used in [decision making](#) and [loops](#).

Operator	Meaning of Operator	Example
==	Equal to	5 == 3 is evaluated to 0
>	Greater than	5 > 3 is evaluated to 1

<	Less than	5 < 3 is evaluated to 0
!=	Not equal to	5 != 3 is evaluated to 1
>=	Greater than or equal to	5 >= 3 is evaluated to 1
<=	Less than or equal to	5 <= 3 is evaluated to 0

Example 4: Relational Operators

```
// Working of relational operators
#include <stdio.h>

int main()
{
    int a = 5, b = 5, c = 10;

    printf("%d == %d is %d \n", a, b, a == b);
    printf("%d == %d is %d \n", a, c, a == c);
    printf("%d > %d is %d \n", a, b, a > b);
    printf("%d > %d is %d \n", a, c, a > c);
    printf("%d < %d is %d \n", a, b, a < b);
    printf("%d < %d is %d \n", a, c, a < c);
    printf("%d != %d is %d \n", a, b, a != b);
    printf("%d != %d is %d \n", a, c, a != c);
    printf("%d >= %d is %d \n", a, b, a >= b);
    printf("%d >= %d is %d \n", a, c, a >= c);
    printf("%d <= %d is %d \n", a, b, a <= b);
```

```
printf("%d <= %d is %d \n", a, c, a <= c);
```

```
return 0;
```

```
}
```

Run Code

Output

```
5 == 5 is 1
```

```
5 == 10 is 0
```

```
5 > 5 is 0
```

```
5 > 10 is 0
```

```
5 < 5 is 0
```

```
5 < 10 is 1
```

```
5 != 5 is 0
```

```
5 != 10 is 1
```

```
5 >= 5 is 1
```

```
5 >= 10 is 0
```

```
5 <= 5 is 1
```

```
5 <= 10 is 1
```

C Logical Operators

An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in [decision making in C programming](#).

Operator	Meaning	Example
&&	Logical AND. True only if all operands are true	If c = 5 and d = 2 then, expression ((c==5) && (d>5)) equals to 0.
	Logical OR. True only if either one operand is true	If c = 5 and d = 2 then, expression ((c==5) (d>5)) equals to 1.
!	Logical NOT. True only if the operand is 0	If c = 5 then, expression !(c==5) equals to 0.

Example 5: Logical Operators

```
// Working of logical operators

#include <stdio.h>

int main()
{
    int a = 5, b = 5, c = 10, result;

    result = (a == b) && (c > b);

    printf("(a == b) && (c > b) is %d \n", result);

    result = (a == b) && (c < b);

    printf("(a == b) && (c < b) is %d \n", result);
}
```

```
result = (a == b) || (c < b);
```

```
printf("(a == b) || (c < b) is %d \n", result);
```

```
result = (a != b) || (c < b);
```

```
printf("(a != b) || (c < b) is %d \n", result);
```

```
result = !(a != b);
```

```
printf("!(a != b) is %d \n", result);
```

```
result = !(a == b);
```

```
printf("!(a == b) is %d \n", result);
```

```
return 0;
```

```
}
```

Run Code

Output

```
(a == b) && (c > b) is 1
```

```
(a == b) && (c < b) is 0
```

```
(a == b) || (c < b) is 1
```

```
(a != b) || (c < b) is 0
```

```
!(a != b) is 1
```

```
!(a == b) is 0
```

Explanation of logical operator program

- `(a == b) && (c > 5)` evaluates to 1 because both operands `(a == b)` and `(c > 5)` is 1 (true).
- `(a == b) && (c < b)` evaluates to 0 because operand `(c < b)` is 0 (false).
- `(a == b) || (c < b)` evaluates to 1 because `(a == b)` is 1 (true).
- `(a != b) || (c < b)` evaluates to 0 because both operand `(a != b)` and `(c < b)` are 0 (false).
- `!(a != b)` evaluates to 1 because operand `(a != b)` is 0 (false). Hence, `!(a != b)` is 1 (true).
- `!(a == b)` evaluates to 0 because `(a == b)` is 1 (true). Hence, `!(a == b)` is 0 (false).

C Bitwise Operators

During computation, mathematical operations like: addition, subtraction, multiplication, division, etc are converted to bit-level which makes processing faster and saves power.

Bitwise operators are used in C programming to perform bit-level operations.

Operators	Meaning of operators
&	Bitwise AND

	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement
<<	Shift left
>>	Shift right

Visit [bitwise operator in C](#) to learn more.

Other Operators

Comma Operator

Comma operators are used to link related expressions together. For example:

```
int a, c = 5, d;
```

The sizeof operator

The `sizeof` is a unary operator that returns the size of data (constants, variables, array, structure, etc).

Example 6: sizeof Operator

```
#include <stdio.h>

int main()
{
    int a;
    float b;
    double c;
    char d;

    printf("Size of int=%lu bytes\n", sizeof(a));
    printf("Size of float=%lu bytes\n", sizeof(b));
    printf("Size of double=%lu bytes\n", sizeof(c));
    printf("Size of char=%lu byte\n", sizeof(d));

    return 0;
}
```

Run Code

Output

Size of int = 4 bytes

Size of float = 4 bytes

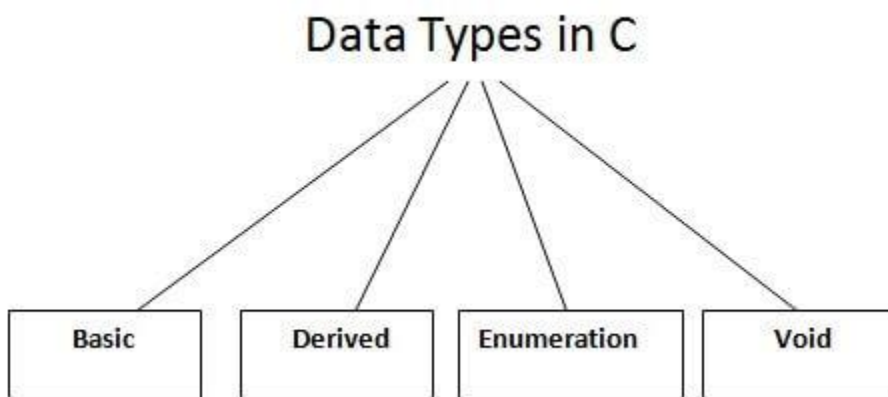
Size of double = 8 bytes

Size of char = 1 byte

Other operators such as ternary operator `?:`, reference operator `&`, dereference operator `*` and member selection operator `->` will be discussed in later tutorials.

Data Types in C

A data type specifies the type of data that a variable can store such as integer, floating, character, etc.



There are the following data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	enum
Void Data Type	void

Basic Data Types

The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals.

The memory size of the basic data types may change according to 32 or 64-bit operating system.

Let's see the basic data types. Its size is given **according to 32-bit architecture**.

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 255
short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 65,535
int	2 byte	-32,768 to 32,767
signed int	2 byte	-32,768 to 32,767
unsigned int	2 byte	0 to 65,535
short int	2 byte	-32,768 to 32,767
signed short int	2 byte	-32,768 to 32,767
unsigned short int	2 byte	0 to 65,535
long int	4 byte	-2,147,483,648 to 2,147,483,647

signed long int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned long int	4 byte	0 to 4,294,967,295
float	4 byte	
double	8 byte	
long double	10 byte	

Int:

Integers are entire numbers without any fractional or decimal parts, and the **int data type** is used to represent them.

It is frequently applied to variables that include **values**, such as **counts**, **indices**, or other numerical numbers. The **int data type** may represent both **positive** and **negative numbers** because it is signed by default.

An **int** takes up **4 bytes** of memory on most devices, allowing it to store values between around -2 billion and +2 billion.

Char:

Individual characters are represented by the **char data type**. Typically used to hold **ASCII** or **UTF-8 encoding scheme characters**, such as **letters**, **numbers**, **symbols**, or **commas**. There are **256 characters** that can be represented by a single char, which takes up one byte of memory. Characters such as **'A'**, **'b'**, **'5'**, or **'\$'** are enclosed in single quotes.

Float:

To represent integers, use the **floating data type**. Floating numbers can be used to represent fractional units or numbers with decimal places.

The **float type** is usually used for variables that require very good precision but may not be very precise. It can store values with an accuracy of about **6 decimal places** and a range of about **3.4×10^{38}** in **4 bytes** of memory.

Double:

Use two data types to represent **two floating integers**. When additional precision is needed, such as in scientific calculations or financial applications, it provides greater accuracy compared to float.

Double type, which uses **8 bytes** of memory and has an accuracy of about **15 decimal places, yields larger values**. C treats floating point numbers as doubles by default if no explicit type is supplied.

1. **int** age = 25;
2. **char** grade = 'A';
3. **float** temperature = 98.6;
4. **double** pi = 3.14159265359;

In the example above, we declare four variables: an **int variable** for the person's age, a **char variable** for the student's grade, a **float variable** for the temperature reading, and two variables for the **number pi**.

Derived Data Type

Beyond the fundamental data types, C also supports **derived data types**, including **arrays, pointers, structures, and unions**. These data types give programmers the ability to handle heterogeneous data, directly modify memory, and build complicated data structures.

Array:

An **array, a derived data type**, lets you store a sequence of **fixed-size elements** of the same type. It provides a mechanism for joining multiple targets of the same data under the same name.

The index is used to access the elements of the array, with a **0 index** for the first entry. The size of the array is fixed at declaration time and cannot be changed during program execution. The array components are placed in adjacent memory regions.

Here is an example of declaring and utilizing an array:

```
1. #include <stdio.h>
2.
3. int main() {
4.     int numbers[5]; // Declares an integer array with a size of 5 elements
5.
6.     // Assign values to the array elements
7.     numbers[0] = 10;
8.     numbers[1] = 20;
9.     numbers[2] = 30;
10.    numbers[3] = 40;
11.    numbers[4] = 50;
12.
13.    // Display the values stored in the array
14.    printf("Values in the array: ");
15.    for (int i = 0; i < 5; i++) {
16.        printf("%d ", numbers[i]);
17.    }
18.    printf("\n");
19.
20.    return 0;
21.}
```

Output:

Values in the array: 10 20 30 40 50

Pointer:

A **pointer** is a derived data type that keeps track of another data type's memory address. When a **pointer** is declared, the **data type** it refers to is **stated first**, and then the **variable name** is preceded by **an asterisk (*)**.

You can have incorrect access and change the value of variable using pointers by specifying the memory address of the variable. **Pointers** are commonly used in **tasks** such as **function pointers**, **data structures**, and **dynamic memory allocation**.

Here is an example of declaring and employing a pointer:

```
1. #include <stdio.h>
2.
3. int main() {
4.     int num = 42;    // An integer variable
5.     int *ptr;        // Declares a pointer to an integer
6.
7.     ptr = &num;      // Assigns the address of 'num' to the pointer
8.
9.     // Accessing the value of 'num' using the pointer
10.    printf("Value of num: %d\n", *ptr);
11.
12.    return 0;
13.}
```

Output:

Value of num: 42

Structure:

A structure is a derived data type that enables the creation of composite data types by allowing the grouping of many data types under a single name. It gives you the ability to create your own unique data structures by fusing together variables of various sorts.

1. A structure's members or fields are used to refer to each variable within it.
2. Any data type, including different structures, can be a member of a structure.
3. A structure's members can be accessed by using the dot (.) operator.

A declaration and use of a structure is demonstrated here:

```
1. #include <stdio.h>
2. #include <string.h>
3. // Define a structure representing a person
4. struct Person {
5.     char name[50];
6.     int age;
7.     float height;
8. };
9.
10. int main() {
11.     // Declare a variable of type struct Person
12.     struct Person person1;
13.
14.     // Assign values to the structure members
15.     strcpy(person1.name, "John Doe");
16.     person1.age = 30;
17.     person1.height = 1.8;
18.
19.     // Accessing the structure members
20.     printf("Name: %s\n", person1.name);
21.     printf("Age: %d\n", person1.age);
22.     printf("Height: %.2f\n", person1.height);
23.
```

```
24. return 0;
25. }
```

Output:

```
Name: John Doe
Age: 30
Height: 1.80
```

Union:

A derived data type called a **union** enables you to store various data types in the same memory address. In contrast to structures, where each member has a separate memory space, members of a union all share a single memory space. A value can only be held by one member of a union at any given moment. When you need to represent many data types interchangeably, unions come in handy. Like structures, you can access the members of a union by using the **dot (.)** operator.

Here is an example of a union being declared and used:

```
1. #include <stdio.h>
2. // Define a union representing a numeric value
3. union NumericValue {
4.     int intValue;
5.     float floatValue;
6.     char stringValue[20];
7. };
8. int main() {
9.     // Declare a variable of type union NumericValue
10.    union NumericValue value;
11.    // Assign a value to the union
12.    value.intValue = 42;
13.    // Accessing the union members
```

```
14. printf("Integer Value: %d\n", value.intValue);
15. // Assigning a different value to the union
16. value.floatValue = 3.14;
17. // Accessing the union members
18. printf("Float Value: %.2f\n", value.floatValue);
19.
20. return 0;
21.}
```

Output:

```
Integer Value: 42
Float Value: 3.14
```

Enumeration Data Type

A set of named constants or **enumerators** that represent a collection of connected values can be defined in C using the **enumeration data type (enum)**. **Enumerations** give you the means to give names that make sense to a group of integral values, which makes your code easier to read and maintain.

Here is an example of how to define and use an enumeration in C:

```
1. #include <stdio.h>
2.
3. // Define an enumeration for days of the week
4. enum DaysOfWeek {
5.     Monday,
6.     Tuesday,
7.     Wednesday,
8.     Thursday,
9.     Friday,
```

```
10. Saturday,
11. Sunday
12.};
13.
14.int main() {
15. // Declare a variable of type enum DaysOfWeek
16. enum DaysOfWeek today;
17.
18. // Assign a value from the enumeration
19. today = Wednesday;
20.
21. // Accessing the enumeration value
22. printf("Today is %d\n", today);
23.
24. return 0;
25.}
```

Output:

Today is 2

Void Data Type

The **void data type** in the C language is used to denote the lack of a particular type. **Function return types**, **function parameters**, and **pointers** are three situations where it is frequently utilized.

Function Return Type:

A **void return type** function does not produce a value. A **void function** executes a task or action and ends rather than returning a value.

Example:

```
1. void printHello() { printf("Hello, world!\n"); }
```

Function Parameters:

The **parameter void** can be used to indicate that a function accepts no arguments.

Example:

```
1. void processInput(void) { /* Function logic */ }
```

Pointers:

Any address can be stored in a pointer of type **void***, making it a universal pointer. It offers a method for working with pointers to ambiguous or atypical types.

Example:

```
1. void* dataPtr;
```

The **void data type** is helpful for defining functions that don't accept any arguments when working with generic pointers or when you wish to signal that a function doesn't return a value. It is significant to note that while **void*** can be used to build generic pointers, void itself cannot be declared as a variable type.

Here is a sample of code that shows how to utilize void in various situations:

```
1. #include <stdio.h>
2. // Function with void return type
3. void printHello() {
4.     printf("Hello, world!\n");
5. }
6. // Function with void parameter
7. void processInput(void) {
8.     printf("Processing input...\n");
9. }
```

```
10.  
11. int main() {  
12.     // Calling a void function  
13.     printHello();  
14.  
15.     // Calling a function with void parameter  
16.     processInput();  
17.  
18.     // Using a void pointer  
19.     int number = 10;  
20.     void* dataPtr = &number;  
21.     printf("Value of number: %d\n", *(int*)dataPtr);  
22.  
23.     return 0;  
24. }
```

Output:

```
Hello, world!  
Processing input...  
Value of number: 10
```

Conclusion:

As a result, **data types** are essential in the C programming language because they define the kinds of information that variables can hold. They provide the data's size and format, enabling the compiler to allot memory and carry out the necessary actions. Data types supported by C include **void**, **enumeration**, **derived**, and **basic types**. In addition to floating-point types like **float** and **double**, basic data types in C also include integer-based kinds like **int**, **char**, and **short**. These forms can be **signed** or **unsigned**, and they fluctuate in size and range. To create dependable and efficient code, it is crucial to comprehend the memory size and scope of these types.

A few examples of **derived data types** are **unions**, **pointers**, **structures**, and **arrays**. Multiple elements of the same kind can be stored together in contiguous memory due to arrays. **Pointers** keep track of memory addresses, allowing for fast data structure operations and dynamic memory allocation. While **unions** allow numerous variables to share the same memory space, structures group relevant variables together.

Code becomes more legible and maintainable when named constants are defined using enumeration data types. **Enumerations** give named constants integer values to enable the meaningful representation of related data. The void data type indicates the lack of a particular type. It is used as a return type for both **functions** and **function parameters** that don't take any arguments and don't return a value. The **void* pointer** also functions as a general pointer that can **store addresses** of various types.

C programming requires a solid understanding of **data types**. Programmers can ensure adequate memory allocation, avoid **data overflow** or **truncation**, and enhance the readability and maintainability of their code by selecting the right **data type**. C programmers may create **effective, dependable**, and well-structured code that satisfies the requirements of their applications by having a firm understanding of data types.

Keywords in C

A keyword is a **reserved word**. You cannot use it as a variable name, constant name, etc. There are only 32 reserved words (keywords) in the C language.

A list of 32 keywords in the c language is given below:

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static

struct	switch	typedef	union	unsigned	void	volatile	while
--------	--------	---------	-------	----------	------	----------	-------

We will learn about all the C language keywords later.

C if else Statement

The if-else statement in C is used to perform the operations based on some specific condition. The operations specified in if block are executed if and only if the given condition is true.

There are the following variants of if statement in C language.

- If statement
- If-else statement
- If else-if ladder
- Nested if

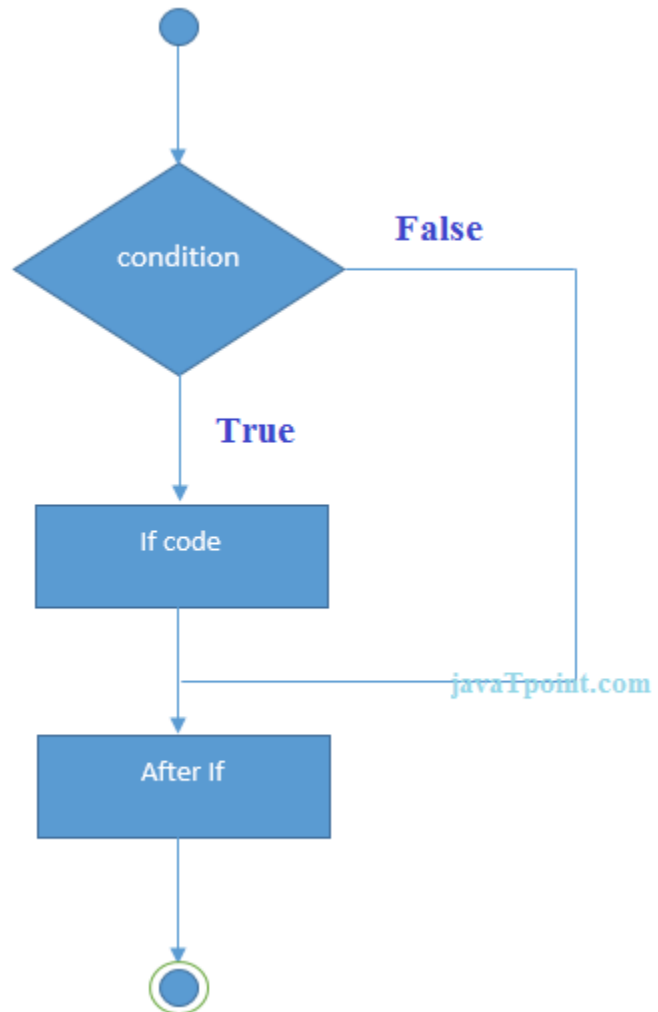
If Statement

The if statement is used to check some given condition and perform some operations depending upon the correctness of that condition. It is mostly used in the scenario where we need to perform the different operations for the different conditions. The syntax of the if statement is given below.

1. `if(expression){`
2. `//code to be executed`

3. }

Flowchart of if statement in C



Let's see a simple example of C language if statement.

1. `#include<stdio.h>`
2. `int main(){`
3. `int number=0;`
4. `printf("Enter a number:");`
5. `scanf("%d",&number);`
6. `if(number%2==0){`

```
7. printf("%d is even number",number);
8. }
9. return 0;
10.}
```

Output

```
Enter a number:4
4 is even number
enter a number:5
```

Program to find the largest number of the three.

```
1. #include <stdio.h>
2. int main()
3. {
4.     int a, b, c;
5.     printf("Enter three numbers?");
6.     scanf("%d %d %d",&a,&b,&c);
7.     if(a>b && a>c)
8.     {
9.         printf("%d is largest",a);
10.    }
11.    if(b>a && b > c)
12.    {
13.        printf("%d is largest",b);
14.    }
15.    if(c>a && c>b)
16.    {
17.        printf("%d is largest",c);
18.    }
```

```
19.  if(a == b && a == c)
20.  {
21.      printf("All are equal");
22.  }
23. }
```

Output

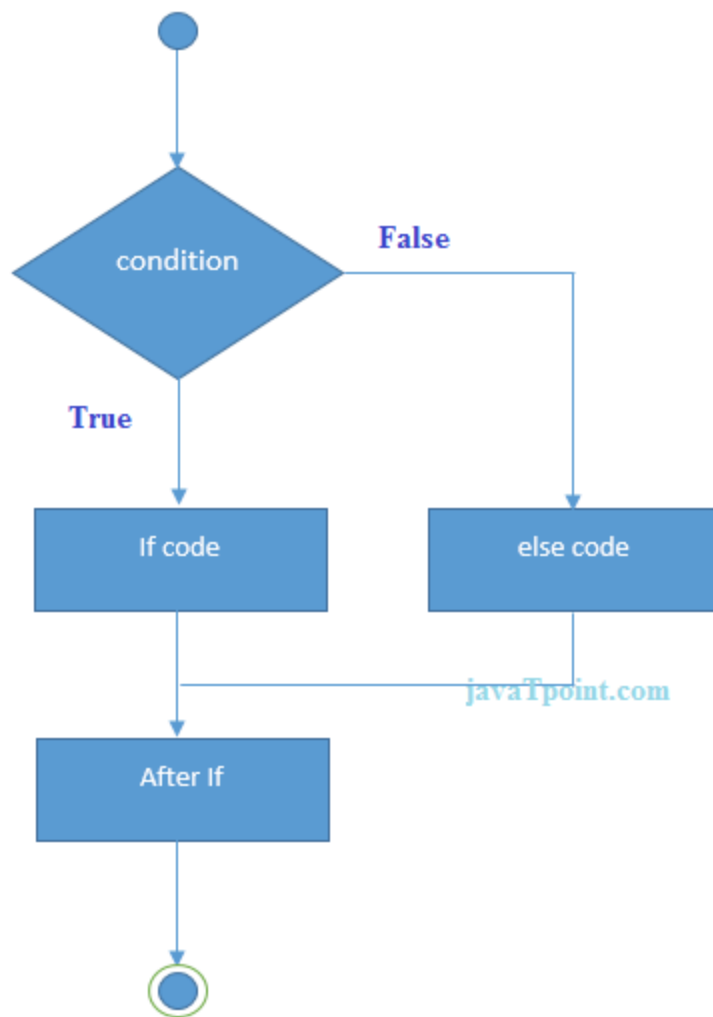
```
Enter three numbers?
12 23 34
34 is largest
```

If-else Statement

The if-else statement is used to perform two operations for a single condition. The if-else statement is an extension to the if statement using which, we can perform two different operations, i.e., one is for the correctness of that condition, and the other is for the incorrectness of the condition. Here, we must notice that if and else block cannot be executed simultaneously. Using if-else statement is always preferable since it always invokes an otherwise case with every if condition. The syntax of the if-else statement is given below.

```
1.  if(expression){
2.      //code to be executed if condition is true
3.  }else{
4.      //code to be executed if condition is false
5.  }
```

Flowchart of the if-else statement in C



Let's see the simple example to check whether a number is even or odd using if-else statement in C language.

```
1. #include<stdio.h>
2. int main(){
3.     int number=0;
4.     printf("enter a number:");
5.     scanf("%d",&number);
6.     if(number%2==0){
7.         printf("%d is even number",number);
8.     }
```

```
9. else{
10. printf("%d is odd number",number);
11.}
12. return 0;
13.}
```

Output

```
enter a number:4
4 is even number
enter a number:5
5 is odd number
```

Program to check whether a person is eligible to vote or not.

```
1. #include <stdio.h>
2. int main()
3. {
4.     int age;
5.     printf("Enter your age?");
6.     scanf("%d",&age);
7.     if(age>=18)
8.     {
9.         printf("You are eligible to vote...");
10.    }
11.    else
12.    {
13.        printf("Sorry ... you can't vote");
14.    }
15.}
```

Output

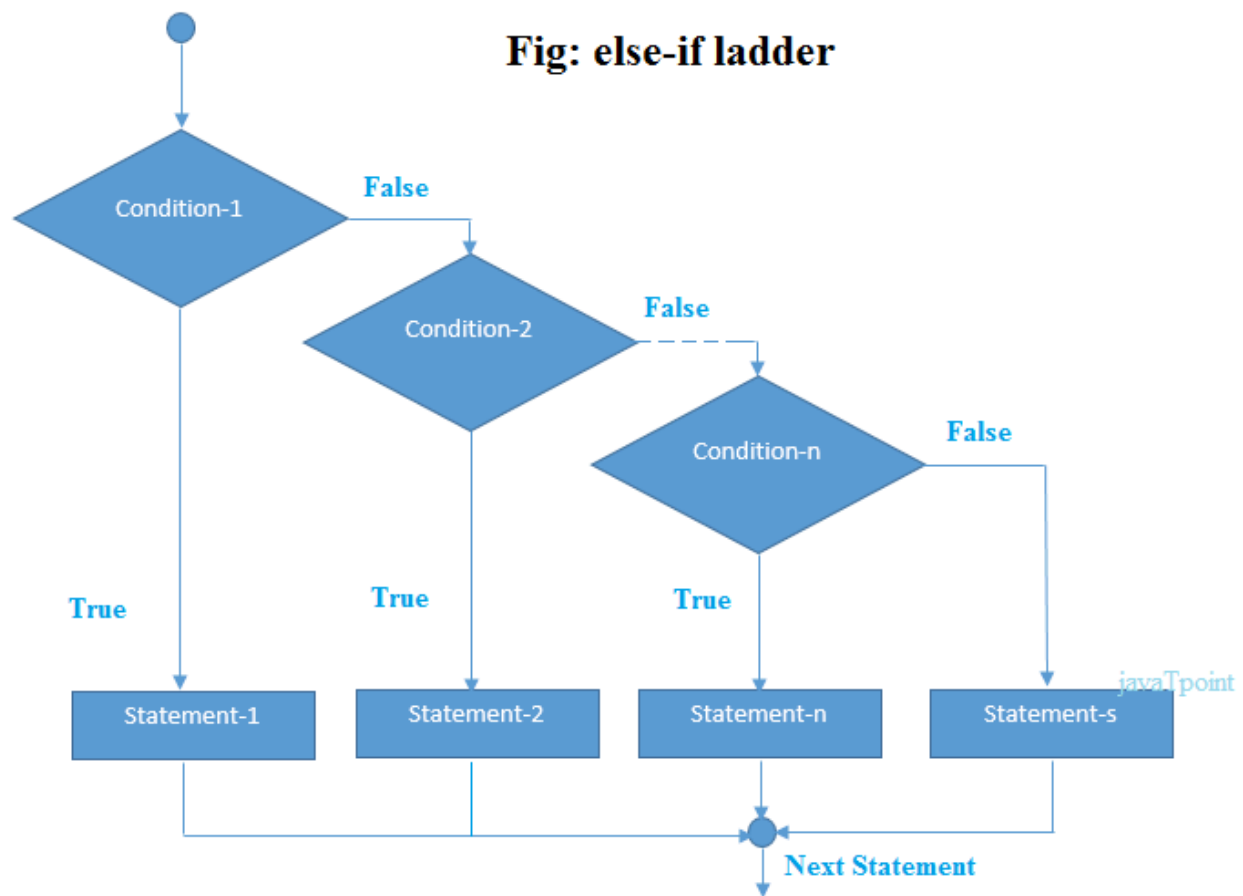
```
Enter your age?18
You are eligible to vote...
Enter your age?13
Sorry ... you can't vote
```

If else-if ladder Statement

The if-else-if ladder statement is an extension to the if-else statement. It is used in the scenario where there are multiple cases to be performed for different conditions. In if-else-if ladder statement, if a condition is true then the statements defined in the if block will be executed, otherwise if some other condition is true then the statements defined in the else-if block will be executed, at the last if none of the condition is true then the statements defined in the else block will be executed. There are multiple else-if blocks possible. It is similar to the switch case statement where the default is executed instead of else block if none of the cases is matched.

```
1. if(condition1){
2. //code to be executed if condition1 is true
3. }else if(condition2){
4. //code to be executed if condition2 is true
5. }
6. else if(condition3){
7. //code to be executed if condition3 is true
8. }
9. ...
10. else{
11. //code to be executed if all the conditions are false
12. }
```

Flowchart of else-if ladder statement in C



The example of an if-else-if statement in C language is given below.

```

1. #include<stdio.h>
2. int main(){
3. int number=0;
4. printf("enter a number:");
5. scanf("%d",&number);
6. if(number==10){
7. printf("number is equals to 10");
8. }
9. else if(number==50){
10. printf("number is equal to 50");
11.}
  
```

```
12. else if(number==100){
13. printf("number is equal to 100");
14.}
15. else{
16. printf("number is not equal to 10, 50 or 100");
17.}
18. return 0;
19.}
```

Output

```
enter a number:4
number is not equal to 10, 50 or 100
enter a number:50
number is equal to 50
```

Program to calculate the grade of the student according to the specified marks.

```
1. #include <stdio.h>
2. int main()
3. {
4.     int marks;
5.     printf("Enter your marks?");
6.     scanf("%d",&marks);
7.     if(marks > 85 && marks <= 100)
8.     {
9.         printf("Congrats ! you scored grade A ...");
10.    }
11.    else if (marks > 60 && marks <= 85)
12.    {
```

```
13.     printf("You scored grade B + ...");
14. }
15. else if (marks > 40 && marks <= 60)
16. {
17.     printf("You scored grade B ...");
18. }
19. else if (marks > 30 && marks <= 40)
20. {
21.     printf("You scored grade C ...");
22. }
23. else
24. {
25.     printf("Sorry you are fail ...");
26. }
27.}
```

Output

Enter your marks?10

Sorry you are fail ...

Enter your marks?40

You scored grade C ...

Enter your marks?90

Congrats ! you scored grade A ...

C Switch Statement

The switch statement in C is an alternate to if-else-if ladder statement which allows us to execute multiple operations for the different possible values of a single variable called switch variable. Here, We can define various statements in the multiple cases for the different values of a single variable.

The syntax of switch statement in **c language** is given below:

1. **switch**(expression){
2. **case** value1:
3. *//code to be executed;*
4. **break;** *//optional*
5. **case** value2:
6. *//code to be executed;*
7. **break;** *//optional*
8.
- 9.
10. **default:**
11. code to be executed **if** all cases are not matched;
12. }

Rules for switch statement in C language

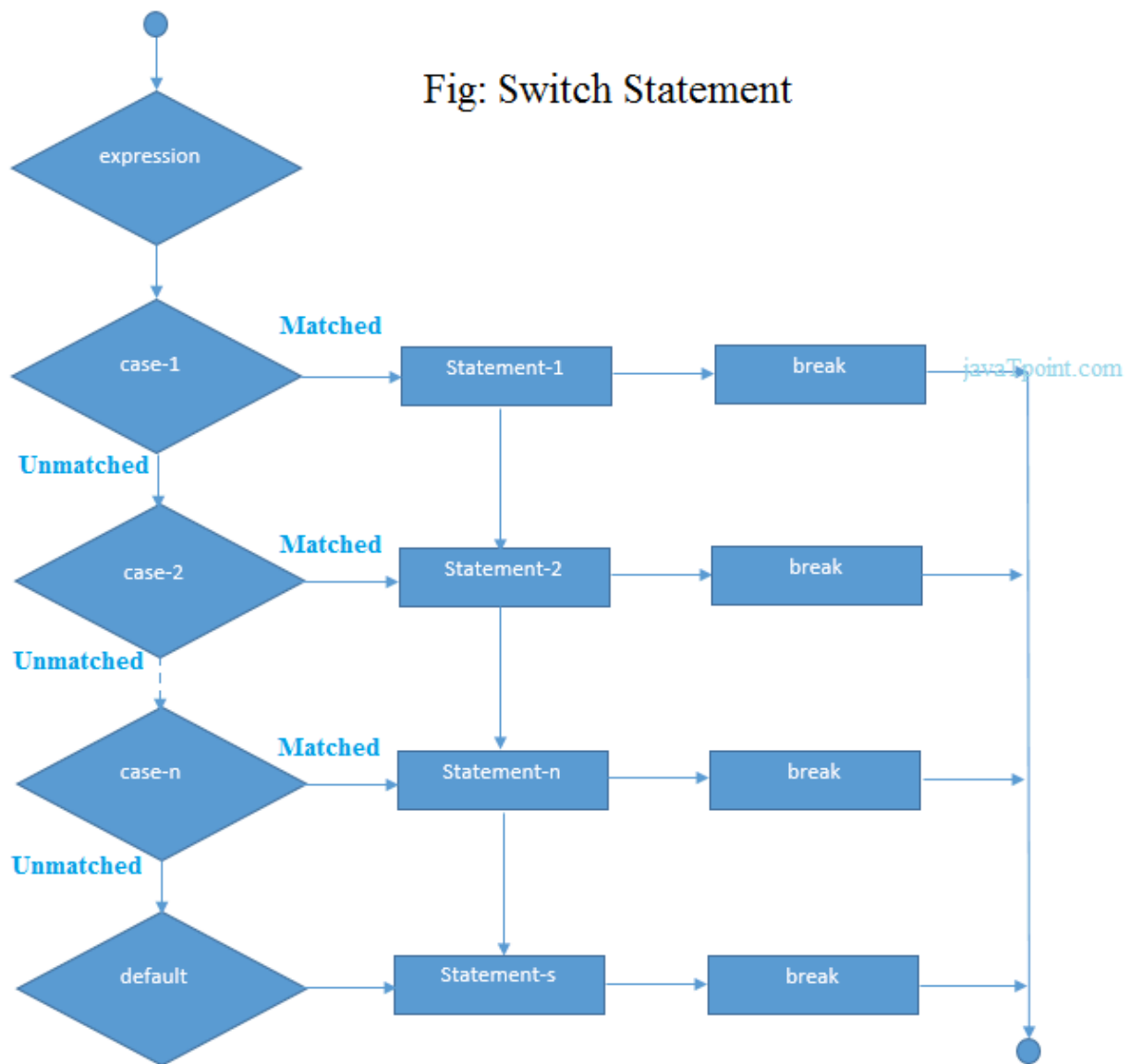
- 1) The *switch expression* must be of an integer or character type.
- 2) The *case value* must be an integer or character constant.
- 3) The *case value* can be used only inside the switch statement.
- 4) The *break statement* in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as *fall through* the state of C switch statement.

Let's try to understand it by the examples. We are assuming that there are following variables.

1. **int** x,y,z;
2. **char** a,b;
3. **float** f;

Valid Switch	Invalid Switch	Valid Case	Invalid Case
switch(x)	switch(f)	case 3;	case 2.5;
switch(x>y)	switch(x+2.5)	case 'a';	case x;
switch(a+b-2)		case 1+2;	case x+2;
switch(func(x,y))		case 'x'>'y';	case 1,2,3;

Flowchart of switch statement in C



Functioning of switch case statement

First, the integer expression specified in the switch statement is evaluated. This value is then matched one by one with the constant values given in the different cases. If a match is found, then all the statements specified in that case are executed along with the all the cases present after that case including the default statement. No two cases can have similar values. If the matched case contains a break statement, then all the cases present after that will be skipped, and the control comes out of the switch. Otherwise, all the cases following the matched case will be executed.

Let's see a simple example of c language switch statement.

```
1. #include<stdio.h>
2. int main(){
3.     int number=0;
4.     printf("enter a number:");
5.     scanf("%d",&number);
6.     switch(number){
7.         case 10:
8.             printf("number is equals to 10");
9.             break;
10.        case 50:
11.            printf("number is equal to 50");
12.            break;
13.        case 100:
14.            printf("number is equal to 100");
15.            break;
16.        default:
17.            printf("number is not equal to 10, 50 or 100");
18.    }
19.    return 0;
20.}
```

Output

enter a number:4
number is not equal to 10, 50 or 100

enter a number:50
number is equal to 50

Switch case example 2

```
1. #include <stdio.h>
2. int main()
3. {
4.     int x = 10, y = 5;
5.     switch(x>y && x+y>0)
6.     {
7.         case 1:
8.             printf("hi");
9.             break;
10.        case 0:
11.            printf("bye");
12.            break;
13.        default:
14.            printf(" Hello bye ");
15.    }
16.
17.}
```

Output

hi

C Switch statement is fall-through

In C language, the switch statement is fall through; it means if you don't use a break statement in the switch case, all the cases after the matching case will be executed.

Let's try to understand the fall through state of switch statement by the example given below.

```
1. #include<stdio.h>
```

```
2. int main(){
3. int number=0;
4.
5. printf("enter a number:");
6. scanf("%d",&number);
7.
8. switch(number){
9. case 10:
10. printf("number is equal to 10\n");
11. case 50:
12. printf("number is equal to 50\n");
13. case 100:
14. printf("number is equal to 100\n");
15. default:
16. printf("number is not equal to 10, 50 or 100");
17. }
18. return 0;
19. }
```

Output

```
enter a number:10
number is equal to 10
number is equal to 50
number is equal to 100
number is not equal to 10, 50 or 100
```

Output

```
enter a number:50
number is equal to 50
number is equal to 100
```

number is not equal to 10, 50 or 100

Nested switch case statement

We can use as many switch statement as we want inside a switch statement. Such type of statements is called nested switch case statements. Consider the following example.

```
1. #include <stdio.h>
2. int main () {
3.
4.     int i = 10;
5.     int j = 20;
6.
7.     switch(i) {
8.
9.         case 10:
10.            printf("the value of i evaluated in outer switch: %d\n",i);
11.        case 20:
12.            switch(j) {
13.                case 20:
14.                    printf("The value of j evaluated in nested switch: %d\n",j);
15.            }
16.        }
17.
18.    printf("Exact value of i is : %d\n", i);
19.    printf("Exact value of j is : %d\n", j);
20.
21.    return 0;
```

22.}

Output

the value of i evaluated in outer switch: 10
The value of j evaluated in nested switch: 20
Exact value of i is : 10
Exact value of j is : 20

if-else vs switch

What is an if-else statement?

An if-else statement in C programming is a conditional statement that executes a different set of statements based on the condition that is true or false. The 'if' block will be executed only when the specified condition is true, and if the specified condition is false, then the else block will be executed.

Syntax of if-else statement is given below:

1. **if**(expression)
2. {
3. // statements;
4. }
5. **else**
6. {
7. // statements;
8. }

What is a switch statement?

A **switch statement** is a conditional statement used in **C programming** to check the value of a variable and compare it with all the cases. If the value is matched with any case, then its corresponding statements will be executed. Each case has some name or

number known as the identifier. The value entered by the user will be compared with all the cases until the case is found. If the value entered by the user is not matched with any case, then the default statement will be executed.

Syntax of the switch statement is given below:

1. **switch**(expression)
2. {
3. **case** constant 1:
4. // statements;
5. **break**;
6. **case** constant 2:
7. // statements;
8. **break**;
9. **case** constant n:
10. // statements;
11. **break**;
12. **default**:
13. // statements;
14. }

Similarity b/w if-else and switch

Both the if-else and switch are the decision-making statements. Here, decision-making statements mean that the output of the expression will decide which statements are to be executed.

Differences b/w if-else and switch statement

The following are the differences between if-else and switch statement are:



- **Definition**

if-else

Based on the result of the expression in the 'if-else' statement, the block of statements will be executed. If the condition is true, then the 'if' block will be executed otherwise 'else' block will execute.

Switch statement

The switch statement contains multiple cases or choices. The user will decide the case, which is to execute.

- **Expression**

If-else

It can contain a single expression or multiple expressions for multiple choices. In this, an expression is evaluated based on the range of values or conditions. It checks both equality and logical expressions.

Switch

It contains only a single expression, and this expression is either a single integer object or a string object. It checks only equality expression.

- **Evaluation**

If-else

An if-else statement can evaluate almost all the types of data such as integer, floating-point, character, pointer, or Boolean.

Switch

A switch statement can evaluate either an integer or a character.

- **Sequence of Execution**

If-else

In the case of 'if-else' statement, either the 'if' block or the 'else' block will be executed based on the condition.

Switch

In the case of the 'switch' statement, one case after another will be executed until the **break** keyword is not found, or the default statement is executed.

- **Default Execution**

If-else

If the condition is not true within the 'if' statement, then by default, the else block statements will be executed.

Switch

If the expression specified within the **switch** statement is not matched with any of the cases, then the default statement, if defined, will be executed.

- **Values**

If-else

Values are based on the condition specified inside the 'if' statement. The value will decide either the 'if' or 'else' block is to be executed.

Switch

In this case, value is decided by the user. Based on the choice of the user, the case will be executed.

- **Use**

If-else

It evaluates a condition to be true or false.

Switch

A **switch** statement compares the value of the variable with multiple cases. If the value is matched with any of the cases, then the block of statements associated with this case will be executed.

- **Editing**

If-else

Editing in 'if-else' statement is not easy as if we remove the 'else' statement, then it will create the havoc.

Switch

Editing in **switch** statement is easier as compared to the 'if-else' statement. If we remove any of the cases from the switch, then it will not interrupt the execution of other cases. Therefore, we can say that the **switch** statement is easy to modify and maintain.

- **Speed**

If-else

If the choices are multiple, then the speed of the execution of 'if-else' statements is slow.

Switch

The case constants in the switch statement create a jump table at the compile time. This jump table chooses the path of the execution based on the value of the expression.

If we have a multiple choice, then the execution of the switch statement will be much faster than the equivalent logic of 'if-else' statement.

Let's summarize the above differences in a tabular form.

	If-else	switch
Definition	Depending on the condition in the 'if' statement, 'if' and 'else' blocks are executed.	The user will decide which statement is to be executed.
Expression	It contains either logical or equality expression.	It contains a single expression which can be either a character or integer variable.
Evaluation	It evaluates all types of data, such as integer, floating-point, character or Boolean.	It evaluates either an integer, or character.
Sequence of execution	First, the condition is checked. If the condition is true then 'if' block is executed otherwise 'else' block	It executes one case after another till the break keyword is not found, or the default statement is executed.
Default execution	If the condition is not true, then by default, else block will be executed.	If the value does not match with any case, then by default, default statement is executed.

Editing	Editing is not easy in the 'if-else' statement.	Cases in a switch statement are easy to maintain and modify. Therefore, we can say that the removal or editing of any case will not interrupt the execution of other cases.
Speed	If there are multiple choices implemented through 'if-else', then the speed of the execution will be slow.	If we have multiple choices then the switch statement is the best option as the speed of the execution will be much higher than 'if-else'.

C Loops

The looping can be defined as repeating the same process multiple times until a specific condition satisfies. There are three types of loops used in the C language. In this part of the tutorial, we are going to learn all the aspects of C loops.

Why use loops in C language?

The looping simplifies the complex problems into the easy ones. It enables us to alter the flow of the program so that instead of writing the same code again and again, we can repeat the same code for a finite number of times. For example, if we need to print the first 10 natural numbers then, instead of using the printf statement 10 times, we can print inside a loop which runs up to 10 iterations.

Advantage of loops in C

- 1) It provides code reusability.
- 2) Using loops, we do not need to write the same code again and again.

3) Using loops, we can traverse over the elements of data structures (array or linked lists).

Types of C Loops

There are three types of loops in C language that is given below:

1. do while
2. while
3. for

do-while loop in C

The do-while loop continues until a given condition satisfies. It is also called post tested loop. It is used when it is necessary to execute the loop at least once (mostly menu driven programs).

The syntax of do-while loop in c language is given below:

1. **do**{
2. //code to be executed
3. }**while**(condition);

Flowchart and Example of do-while loop

while loop in C

The while loop in c is to be used in the scenario where we don't know the number of iterations in advance. The block of statements is executed in the while loop until the condition specified in the while loop is satisfied. It is also called a pre-tested loop.

The syntax of while loop in c language is given below:

1. **while**(condition){
2. //code to be executed
3. }

Flowchart and Example of while loop

for loop in C

The for loop is used in the case where we need to execute some part of the code until the given condition is satisfied. The for loop is also called as a pre-tested loop. It is better to use for loop if the number of iteration is known in advance.

The syntax of for loop in c language is given below:

1. **for**(initialization;condition;incr/decr){
2. *//code to be executed*
3. }

for loop in C

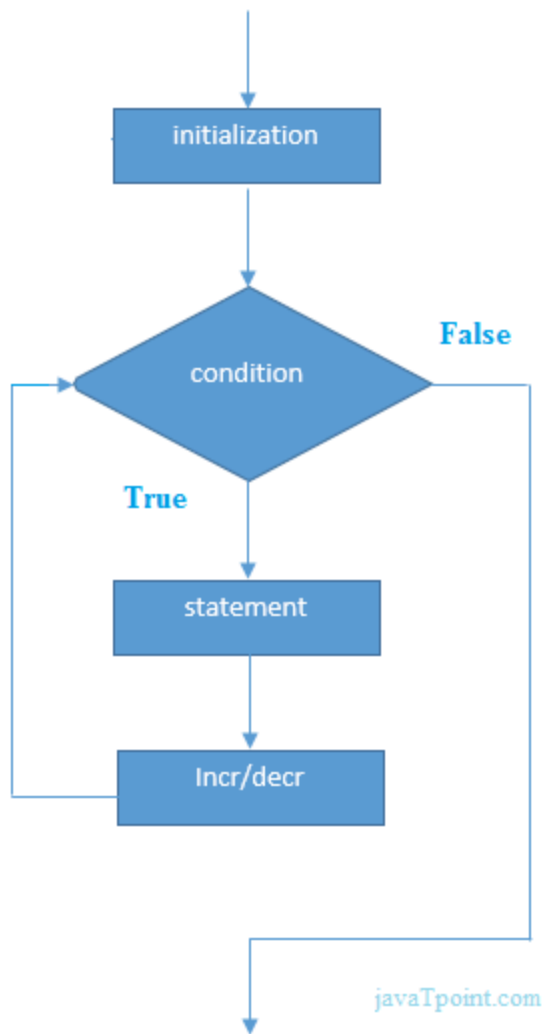
The **for loop in C language** is used to iterate the statements or a part of the program several times. It is frequently used to traverse the data structures like the array and linked list.

Syntax of for loop in C

The syntax of for loop in c language is given below:

1. **for**(Expression 1; Expression 2; Expression 3){
2. *//code to be executed*
3. }

Flowchart of for loop in C



C for loop Examples

Let's see the simple program of for loop that prints table of 1.

1. `#include<stdio.h>`
2. `int main(){`
3. `int i=0;`
4. `for(i=1;i<=10;i++){`
5. `printf("%d \n",i);`
6. `}`

7. `return 0;`
8. `}`

Output

1
2
3
4
5
6
7
8
9
10

C Program: Print table for the given number using C for loop

1. `#include<stdio.h>`
2. `int main(){`
3. `int i=1,number=0;`
4. `printf("Enter a number: ");`
5. `scanf("%d",&number);`
6. `for(i=1;i<=10;i++){`
7. `printf("%d \n",(number*i));`
8. `}`

```
9. return 0;  
10.}
```

Output

Enter a number: 2

2

4

6

8

10

12

14

16

18

20

Enter a number: 1000

1000

2000

3000

4000

5000

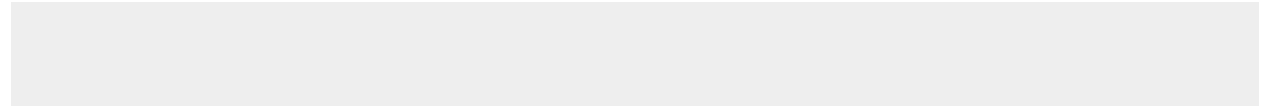
6000

7000

8000

9000

10000



do while loop in C

A **loop** is a programming control structure that allows you to execute a **block of code** indefinitely if a specific condition is met. Loops are used to execute repeating activities and boost programming performance. There are multiple loops in the C programming language, one of which is the "**do-while**" loop.

A "**do-while**" loop is a form of a **loop** in C that executes the code block first, followed by the condition. If the condition is **true**, the **loop** continues to run; else, it stops. However, whether the condition is originally **true**, it ensures that the code block is performed at least once.

do while loop syntax

The syntax of the C language do-while loop is given below:

1. **do**{
2. *//code to be executed*
3. **}while**(condition);

The components are divided into the following:

- The **do keyword** marks the beginning of the Loop.
- The **code block** within **curly braces {}** is the body of the loop, which contains the code you want to repeat.

- The **while keyword** is followed by a condition enclosed in parentheses (). After the code block has been run, this condition is verified. If the condition is **true**, the loop continues else, the **loop ends**.

Working of do while Loop in C

Let us look at an example of how a **do-while loop** works in C. In this example, we will write a simple program that questions the user for a **password** and keeps asking until the right password is input.

Example:

```
1. #include <stdio.h>
2. #include <string.h>
3. int main() {
4.     char password[] = "secret";
5.     char input[20];
6.     do {
7.         printf("Enter the password: ");
8.         scanf("%s", input);
9.     } while (strcmp(input, password) != 0);
10. printf("Access granted!\n");
11. return 0;
12. }
```

The program runs as follows:

1. The following header files are included: **<stdio.h>** for standard **input** and **output** routines and **<string.h>** for string **manipulation functions**.
2. The correct password is defined as a **character array (char password[])** with the value **"secret"**
3. After that, we define another character array input to store the user's input.

4. The **do keyword** indicates that the code block included within the **loop** will be performed at least once.
5. Using the **printf() function**, we display a prompt requesting the user to input their password inside the Loop.
6. Next, we read the **user's input** using the **scanf() function** and store it in the **input array**.
7. After reading the **input**, we use the **strcmp() function** to compare the input with the correct password. If the strings are **equal**, the **strcmp function** returns 0. So, we continue looping as long as the input and the password are not equal.
8. Once the **correct password** is entered, the loop terminates, and we print **"Access granted!"** using the **printf() function**.
9. After that, the program returns 0 to indicate successful execution.

Output:

Let us walk through a possible scenario:

```
Enter the password: 123
Enter the password: abc
Enter the password: secret
Access Granted!
```

Explanation:

In this example, the user initially enters the wrong passwords, **"123"** and **"abc"**. The loop prompts the user until the correct password **"secret"** is entered. Once the correct password is provided, the loop terminates, and the **"Access granted!"** message is displayed.

Example of do while loop in C:

Example 1:

Here is a simple example of a **"do-while" loop** in C that prints numbers from 1 to 5:

1. `#include <stdio.h>`
2. `int main() {`

```
3. int i = 1;
4. do {
5.     printf("%d\n", i);
6.     i++;
7. } while (i <= 5);
8. return 0;
9. }
```

Output:

```
1
2
3
4
5
```

Explanation:

In this example, the **code block** within the do loop will be executed at least once, printing numbers from **1 to 5**. After each iteration, the **i value** is incremented, and the condition **i <= 5** is checked. If the condition is still true, the loop continues; otherwise, it terminates.

Example 2:

Program to print table for the given number using do while Loop

```
1. #include<stdio.h>
2. intmain(){
3.     inti=1,number=0;
4.     printf("Enter a number: ");
5.     scanf("%d",&number);
6.     do{
7.         printf("%d \n",(number*i));
8.         i++;
9.     }while(i<=10);
10. return 0;
```

11. }

Output:

Enter a number: 5

5

10

15

20

25

30

35

40

45

50

Enter a number: 10

10

20

30

40

50

60

70

80

90

100

Example 3:

ADVERTISEMENT

Let's take a program that prints the multiplication table of a given number N using a **do...while Loop**:

1. `#include <stdio.h>`
2. `int main() {`
3. `int N;`
4. `printf("Enter a number to generate its multiplication table: ");`
5. `scanf("%d", &N);`
6. `inti = 1;`

```
7. do {
8. printf("%d x %d = %d\n", N, i, N * i);
9. i++;
10. } while (i<= 10);
11. return 0;
12. }
```

Output:

ADVERTISEMENT

ADVERTISEMENT

Let us say you enter the number 7 as input:

Please enter a number to generate its multiplication table: 7

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

The program calculates and prints the multiplication table for **7** from 1 to 10.

Infinite do while loop

An **infinite loop** is a loop that runs indefinitely as its condition is always **true** or it lacks a terminating condition. Here is an example of an **infinite do...while loop** in C:

Example:

```
1. #include <stdio.h>
2. int main() {
3.     inti = 1;
4.     do {
```

```
5. printf("Iteration %d\n", i);
6. i++;
7. } while (1); // Condition is always true
8.
9. return 0;
10. }
```

In this **example**, the **loop** will keep running **indefinitely** because **condition 1** is always **true**.

Output:

When you run the program, you will see that it continues printing "**Iteration x**", where x is the **iteration number** without stopping:

```
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
... (and so on)
```

To interrupt an infinite loop like this, you generally use a **break statement** within the **loop** or some external condition you can control, such as **hitting** a specific key combination. In most desktop settings, the keyboard shortcut **Ctrl+C** can escape the Loop.

Nested do while loop in C

In C, we take an example of a **nested do...while loop**. In this example, we will write a program that uses **nested do...while loops** to create a numerical pattern.

Example:

```
1. #include <stdio.h>
2. int main() {
3.     int rows, i = 1;
4.     printf("Enter the number of rows: ");
5.     scanf("%d", &rows);
6.     do {
```

```
7.  int j = 1;
8.  do {
9.    printf("%d ", j);
10. j++;
11. } while (j <= i);
12. printf("\n");
13. i++;
14. } while (i <= rows);
15. return 0;
16. }
```

In this program, we use **nested do...while loops** to generate a pattern of numbers. The **outer loop** controls the number of rows, and the **inner loop** generates the numbers for each row.

Output:

Let us say you input five as the number of rows:

Enter the number of rows: 5

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Explanation:

In this example, the program generates a pattern of numbers in a **triangular shape**. The **outer loop** iterates over the rows, and the **inner loop** iterates within each row, printing the numbers from 1 up to the current row number.

Difference between while and do while Loop

Here is a tabular comparison between the while loop and the do-while Loop in C:

Aspect	while loop	do-while loop
--------	------------	---------------

Syntax	<code>while (condition) { ... }</code>	<code>do { ... } while (condition);</code>
Loop Body Execution	Condition is checked before execution.	The body is executed before the condition.
First Execution	The condition must be true initially.	The body is executed at least once.
Loop Execution	May execute zero or more times.	Will execute at least once.
Example	<code>while (i < 5) { printf("%d\n", i); i++; }</code>	<code>do { printf("%d\n", i); i++; } while (i < 5);</code>
Common Use Cases	When the loop may not run at all.	When you want the loop to run at least once.

While Loop: The loop body is executed before the condition is checked. If the condition is initially **false**, the loop may not execute.

Do-while Loop: The **loop body** is executed at least once before the condition is **checked**. This guarantees that the loop completes at least one iteration.

When you want the **loop** to run based on a condition that may be **false** at first, use the **while loop**, and when you want the loop to run at least once regardless of the starting state, use the **do-while loop**.

while loop in C

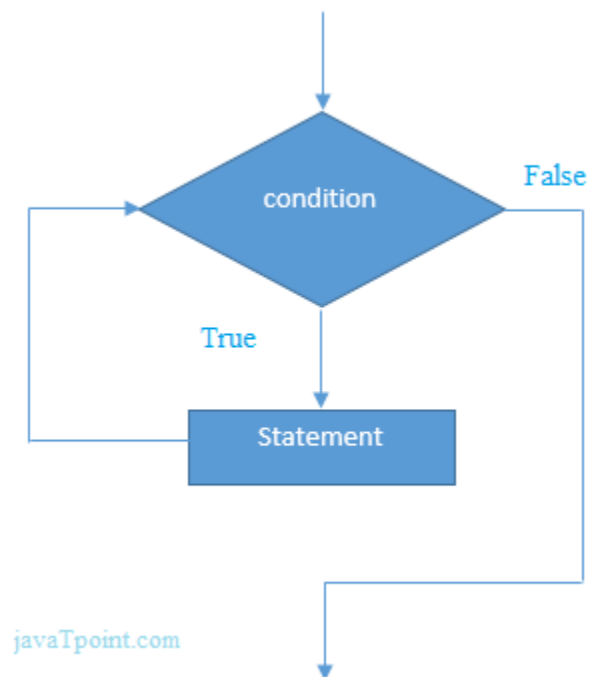
While loop is also known as a pre-tested loop. In general, a while loop allows a part of the code to be executed multiple times depending upon a given boolean condition. It can be viewed as a repeating if statement. The while loop is mostly used in the case where the number of iterations is not known in advance.

Syntax of while loop in C language

The syntax of while loop in c language is given below:

1. **while**(condition){
2. *//code to be executed*
3. }

Flowchart of while loop in C



Example of the while loop in C language

Let's see the simple program of while loop that prints table of 1.

1. **#include**<stdio.h>
2. **int** main(){
3. **int** i=1;
4. **while**(i<=10){
5. **printf**("%d \n",i);

```
6. i++;  
7. }  
8. return 0;  
9. }
```

Output

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Program to print table for the given number using while loop in C

```
1. #include<stdio.h>  
2. int main(){  
3. int i=1,number=0;  
4. printf("Enter a number: ");  
5. scanf("%d",&number);  
6. while(i<=10){  
7. printf("%d \n",(number*i));  
8. i++;  
9. }  
10. return 0;  
11.}
```

Output

Enter a number: 50

50
100
150
200
250
300
350
400
450
500

Enter a number: 100

100
200
300
400
500
600
700
800
900
1000

Properties of while loop

ADVERTISEMENT

ADVERTISEMENT

- A conditional expression is used to check the condition. The statements defined inside the while loop will repeatedly execute until the given condition fails.

- The condition will be true if it returns 0. The condition will be false if it returns any non-zero number.
- In while loop, the condition expression is compulsory.
- Running a while loop without a body is possible.
- We can have more than one conditional expression in while loop.
- If the loop body contains only one statement, then the braces are optional.

Example 1

```
1. #include<stdio.h>
2. void main ()
3. {
4.     int j = 1;
5.     while(j+=2,j<=10)
6.     {
7.         printf("%d ",j);
8.     }
9.     printf("%d",j);
10.}
```

Output

3 5 7 9 11

Example 2

```
1. #include<stdio.h>
2. void main ()
3. {
4.     while()
5.     {
```

```
6.     printf("hello Javatpoint");
7. }
8. }
```

Output

compile time error: while loop can't be empty

Example 3

```
1. #include<stdio.h>
2. void main ()
3. {
4.     int x = 10, y = 2;
5.     while(x+y-1)
6.     {
7.         printf("%d %d",x--,y--);
8.     }
9. }
```

Output

infinite loop

Infinitive while loop in C

If the expression passed in while loop results in any non-zero value then the loop will run the infinite number of times.

```
1. while(1){
2.     //statement
3. }
```

for loop in C

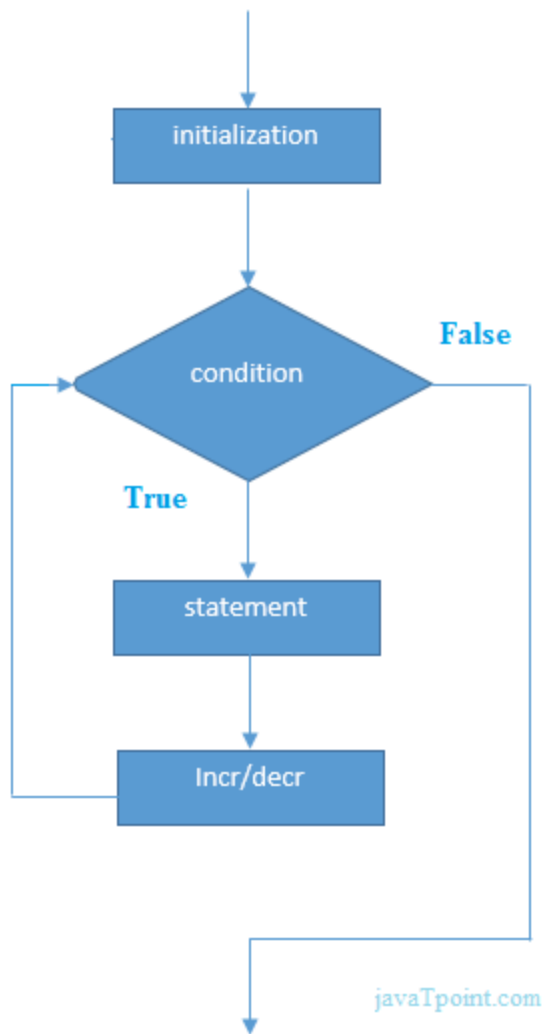
The **for loop in C language** is used to iterate the statements or a part of the program several times. It is frequently used to traverse the data structures like the array and linked list.

Syntax of for loop in C

The syntax of for loop in c language is given below:

1. **for**(Expression 1; Expression 2; Expression 3){
2. *//code to be executed*
3. }

Flowchart of for loop in C



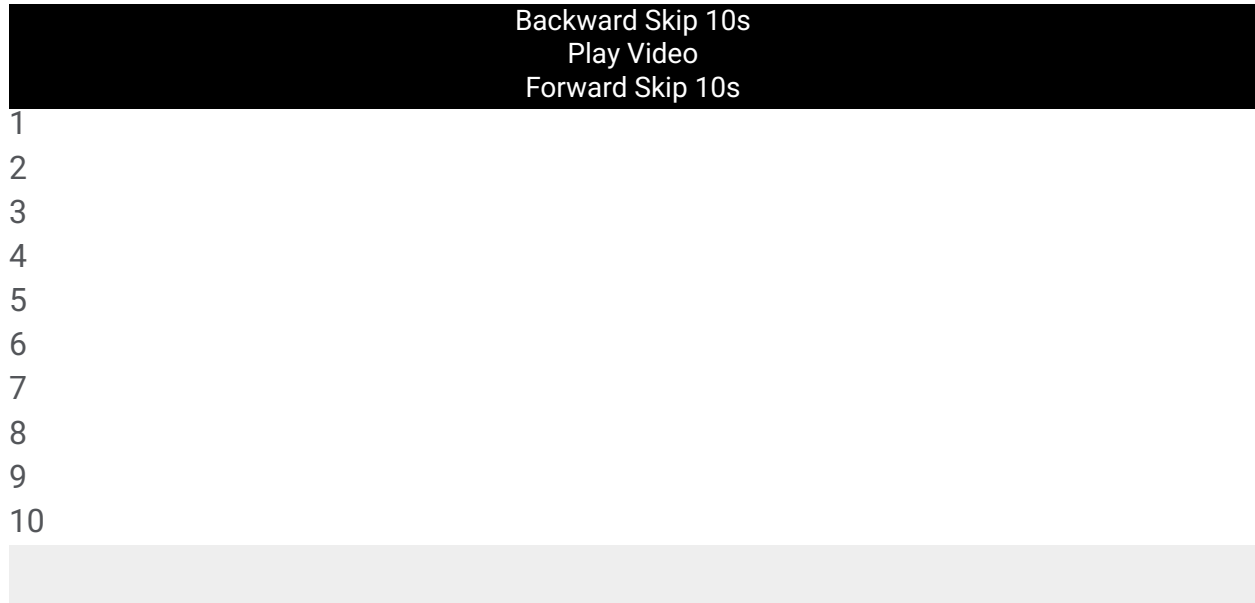
C for loop Examples

Let's see the simple program of for loop that prints table of 1.

1. `#include<stdio.h>`
2. `int main(){`
3. `int i=0;`
4. `for(i=1;i<=10;i++){`
5. `printf("%d \n",i);`
6. `}`

7. `return 0;`
8. `}`

Output



C Program: Print table for the given number using C for loop

1. `#include<stdio.h>`
2. `int main(){`
3. `int i=1,number=0;`
4. `printf("Enter a number: ");`
5. `scanf("%d",&number);`
6. `for(i=1;i<=10;i++){`
7. `printf("%d \n",(number*i));`
8. `}`
9. `return 0;`
10. `}`

Output

Enter a number: 2

2
4
6
8
10
12
14
16
18
20

Enter a number: 1000

1000
2000
3000
4000
5000
6000
7000
8000
9000
10000

Properties of Expression 1

ADVERTISEMENT

ADVERTISEMENT

- The expression represents the initialization of the loop variable.
- We can initialize more than one variable in Expression 1.
- Expression 1 is optional.
- In C, we can not declare the variables in Expression 1. However, It can be an exception in some compilers.

Example 1

```
1. #include <stdio.h>
2. int main()
3. {
4.     int a,b,c;
5.     for(a=0,b=12,c=23;a<2;a++)
6.     {
7.         printf("%d ",a+b+c);
8.     }
9. }
```

Output

35 36

Example 2

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i=1;
5.     for(;i<5;i++)
6.     {
7.         printf("%d ",i);
8.     }
9. }
```

Output

1 2 3 4

Properties of Expression 2

- Expression 2 is a conditional expression. It checks for a specific condition to be satisfied. If it is not, the loop is terminated.
- Expression 2 can have more than one condition. However, the loop will iterate until the last condition becomes false. Other conditions will be treated as statements.
- Expression 2 is optional.
- Expression 2 can perform the task of expression 1 and expression 3. That is, we can initialize the variable as well as update the loop variable in expression 2 itself.
- We can pass zero or non-zero value in expression 2. However, in C, any non-zero value is true, and zero is false by default.

Example 1

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i;
5.     for(i=0;i<=4;i++)
6.     {
7.         printf("%d ",i);
8.     }
9. }
```

output

0 1 2 3 4

Example 2

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i,j,k;
5.     for(i=0,j=0,k=0;i<4,k<8,j<10;i++)
6.     {
7.         printf("%d %d %d\n",i,j,k);
8.         j+=2;
9.         k+=3;
10.    }
11.}
```

Output

```
0 0 0
1 2 3
2 4 6
3 6 9
4 8 12
```

Example 3

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i;
5.     for(i=0;;i++)
6.     {
7.         printf("%d",i);
8.     }
```

9. }

Output

infinite loop

Properties of Expression 3

- Expression 3 is used to update the loop variable.
- We can update more than one variable at the same time.
- Expression 3 is optional.

Example 1

```
1. #include<stdio.h>
2. void main ()
3. {
4.     int i=0,j=2;
5.     for(i = 0;i<5;i++,j=j+2)
6.     {
7.         printf("%d %d\n",i,j);
8.     }
9. }
```

Output

```
0 2
1 4
2 6
3 8
4 10
```

Loop body

The braces {} are used to define the scope of the loop. However, if the loop contains only one statement, then we don't need to use braces. A loop without a body is possible. The braces work as a block separator, i.e., the value variable declared inside for loop is valid only for that block and not outside. Consider the following example.

```
1. #include<stdio.h>
2. void main ()
3. {
4.     int i;
5.     for(i=0;i<10;i++)
6.     {
7.         int i = 20;
8.         printf("%d ",i);
9.     }
10.}
```

Output

```
20 20 20 20 20 20 20 20 20 20
```

Infinite for loop in C

To make a for loop infinite, we need not give any expression in the syntax. Instead of that, we need to provide two semicolons to validate the syntax of the for loop. This will work as an infinite for loop.

```
1. #include<stdio.h>
2. void main ()
3. {
4.     for(;;)
5.     {
```

```
6.     printf("welcome to javatpoint");
7. }
8. }
```

If you run this program, you will see above statement infinite times.

Nested Loops in C

C supports nesting of loops in C. **Nesting of loops** is the feature in C that allows the looping of statements inside another loop. Let's observe an example of nesting loops in C.

Any number of loops can be defined inside another loop, i.e., there is no restriction for defining any number of loops. The nesting level can be defined at n times. You can define any type of loop inside another loop; for example, you can define '**while**' loop inside a '**for**' loop.

Syntax of Nested loop

```
1. Outer_loop
2. {
3.     Inner_loop
4.     {
5.         // inner loop statements.
6.     }
7.     // outer loop statements.
8. }
```

Outer_loop and **Inner_loop** are the valid loops that can be a 'for' loop, 'while' loop or 'do-while' loop.

Nested for loop

The nested for loop means any type of loop which is defined inside the 'for' loop.

```
1. for (initialization; condition; update)
2. {
3.     for(initialization; condition; update)
4.     {
5.         // inner loop statements.
6.     }
7.     // outer loop statements.
8. }
```

Example of nested for loop

```
1. #include <stdio.h>
2. int main()
3. {
4.     int n;// variable declaration
5.     printf("Enter the value of n :");
6.     // Displaying the n tables.
7.     for(int i=1;i<=n;i++) // outer loop
8.     {
9.         for(int j=1;j<=10;j++) // inner loop
10.        {
11.            printf("%d\t",(i*j)); // printing the value.
12.        }
13.        printf("\n");
14.    }
```

Explanation of the above code

- First, the 'i' variable is initialized to 1 and then program control passes to the $i \leq n$.
- The program control checks whether the condition ' $i \leq n$ ' is true or not.

- If the condition is true, then the program control passes to the inner loop.
- The inner loop will get executed until the condition is true.
- After the execution of the inner loop, the control moves back to the update of the outer loop, i.e., $i++$.
- After incrementing the value of the loop counter, the condition is checked again, i.e., $i \leq n$.
- If the condition is true, then the inner loop will be executed again.
- This process will continue until the condition of the outer loop is true.

Output:

```

input
Enter the value of n : 3
1      2      3      4      5      6      7      8      9      10
2      4      6      8      10     12     14     16     18     20
3      6      9      12     15     18     21     24     27     30

...Program finished with exit code 0
Press ENTER to exit console.

```

Nested while loop

The nested while loop means any type of loop which is defined inside the 'while' loop.

1. **while**(condition)
2. {
3. **while**(condition)
4. {
5. // inner loop statements.
6. }
7. // outer loop statements.
8. }

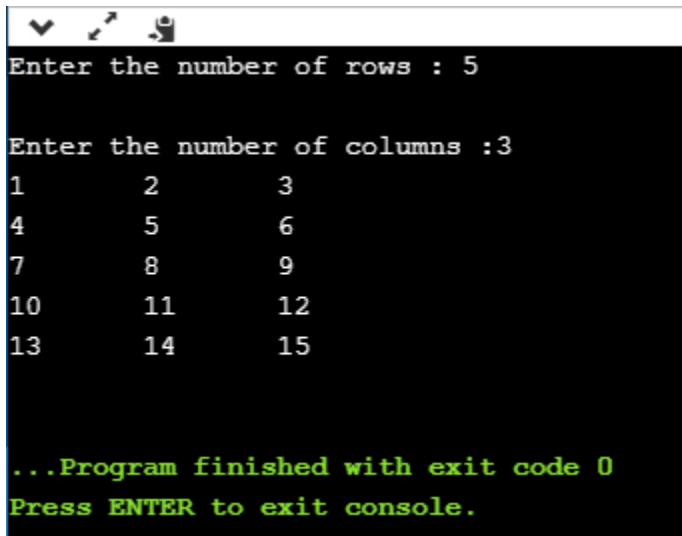
Example of nested while loop

```
1. #include <stdio.h>
2. int main()
3. {
4.     int rows; // variable declaration
5.     int columns; // variable declaration
6.     int k=1; // variable initialization
7.     printf("Enter the number of rows :"); // input the number of rows.
8.     scanf("%d",&rows);
9.     printf("\nEnter the number of columns :"); // input the number of columns.
10.    scanf("%d",&columns);
11.    int a[rows][columns]; //2d array declaration
12.    int i=1;
13.    while(i<=rows) // outer loop
14.    {
15.        int j=1;
16.        while(j<=columns) // inner loop
17.        {
18.            printf("%d\t",k); // printing the value of k.
19.            k++; // increment counter
20.            j++;
21.        }
22.        i++;
23.        printf("\n");
24.    }
25.}
```

Explanation of the above code.

- We have created the 2d array, i.e., `int a[rows][columns]`.
- The program initializes the 'i' variable by 1.
- Now, control moves to the while loop, and this loop checks whether the condition is true, then the program control moves to the inner loop.
- After the execution of the inner loop, the control moves to the update of the outer loop, i.e., `i++`.
- After incrementing the value of 'i', the condition (`i<=rows`) is checked.
- If the condition is true, the control then again moves to the inner loop.
- This process continues until the condition of the outer loop is true.

Output:



```

Enter the number of rows : 5

Enter the number of columns :3
1      2      3
4      5      6
7      8      9
10     11     12
13     14     15

...Program finished with exit code 0
Press ENTER to exit console.

```

Nested do..while loop

The nested do..while loop means any type of loop which is defined inside the 'do..while' loop.

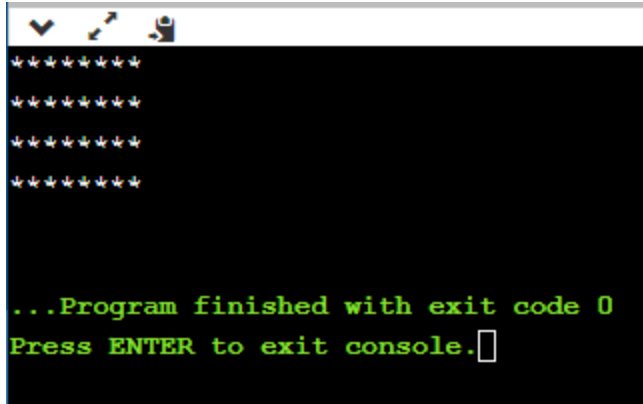
1. **do**
2. {
3. **do**
4. {

5. // inner loop statements.
6. }while(condition);
7. // outer loop statements.
8. }while(condition);

Example of nested do..while loop.

1. #include <stdio.h>
2. int main()
3. {
4. /*printing the pattern
5. *****
6. *****
7. *****
8. ***** */
9. int i=1;
10. do // outer loop
11. {
12. int j=1;
13. do // inner loop
14. {
15. printf("*");
16. j++;
17. }while(j<=8);
18. printf("\n");
19. i++;
20. }while(i<=4);
21. }

Output:



```
*****
*****
*****
*****

...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation of the above code.

- First, we initialize the outer loop counter variable, i.e., 'i' by 1.
 - As we know that the do..while loop executes once without checking the condition, so the inner loop is executed without checking the condition in the outer loop.
 - After the execution of the inner loop, the control moves to the update of the i++.
 - When the loop counter value is incremented, the condition is checked. If the condition in the outer loop is true, then the inner loop is executed.
 - This process will continue until the condition in the outer loop is true.
-

Infinite Loop in C

What is infinite loop?

An infinite loop is a looping construct that does not terminate the loop and executes the loop forever. It is also called an **indefinite** loop or an **endless** loop. It either produces a continuous output or no output.

When to use an infinite loop

An infinite loop is useful for those applications that accept the user input and generate the output continuously until the user exits from the application manually. In the following situations, this type of loop can be used:

ADVERTISEMENT

- All the operating systems run in an infinite loop as it does not exist after performing some task. It comes out of an infinite loop only when the user manually shuts down the system.
- All the servers run in an infinite loop as the server responds to all the client requests. It comes out of an indefinite loop only when the administrator shuts down the server manually.
- All the games also run in an infinite loop. The game will accept the user requests until the user exits from the game.

We can create an infinite loop through various loop structures. The following are the loop structures through which we will define the infinite loop:

- for loop
- while loop
- do-while loop
- go to statement
- C macros

For loop

Let's see the **infinite 'for'** loop. The following is the definition for the **infinite** for loop:

1. **for**(; ;)
2. {
3. // body of the for loop.
4. }

As we know that all the parts of the **'for' loop** are optional, and in the above for loop, we have not mentioned any condition; so, this loop will execute infinite times.

Let's understand through an example.

```
1. #include <stdio.h>
2. int main()
3. {
4.     for(;;)
5.     {
6.         printf("Hello javatpoint");
7.     }
8.     return 0;
9. }
```

In the above code, we run the 'for' loop infinite times, so **"Hello javatpoint"** will be displayed infinitely.

Output



```
input
atpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello
vatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello
avatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHell
javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHel
javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHe
o javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointH
lo javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpoint
llo javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpoi
ello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpoi
Hello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpo
tHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javatp
ntHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello javat
intHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello java
ointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello jav
pointHello javatpointHello javatpointHello javatpointHello javatpointHello javatpointHello ja
```

while loop

Now, we will see how to create an infinite loop using a while loop. The following is the definition for the infinite while loop:

```
1. while(1)
2. {
3.     // body of the loop..
4. }
```

In the above while loop, we put '1' inside the loop condition. As we know that any non-zero integer represents the true condition while '0' represents the false condition.

Let's look at a simple example.

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i=0;
5.     while(1)
6.     {
7.         i++;
8.         printf("i is :%d",i);
9.     }
10. return 0;
11.}
```

In the above code, we have defined a while loop, which runs infinite times as it does not contain any condition. The value of 'i' will be updated an infinite number of times.

Output

```
is :67951i is :67952i is :67953i is :67954i is :67955i is :67956i is :67957i is :67958i is :67959i is :67960i is :67961i is :67962i is :67963i is :67964i is :67965i is :67966i is :67967i is :67968i is :67969i is :67970i is :67971i is :67972i is :67973i is :67974i is :67975i is :67976i is :67977i is :67978i is :67979i is :67980i is :67981i is :67982i is :67983i is :67984i is :67985i is :67986i is :67987i is :67988i is :67989i is :67990i is :67991i is :67992i is :67993i is :67994i is :67995i is :67996i is :67997i is :67998i is :67999i is :68000i is :68001i is :68002i is :68003i is :68004i is :68005i is :68006i is :68007i is :68008i is :68009i is :68010i is :68011i is :68012i is :68013i is :68014i is :68015i is :68016i is :68017i is :68018i is :68019i is :68020i is :68021i is :68022i is :68023i is :68024i is :68025i is :68026i is :68027i is :68028i is :68029i is :68030i is :68031i is :68032i is :68033i is :68034i is :68035i is :68036i is :68037i is :68038i is :68039i is :68040i is :68041i is :68042i is :68043i is :68044i is :68045i is :68046i is :68047i is :68048i is :68049i is :68050i is :68051i is :68052i is :68053i is :68054i is :68055i is :68056i is :68057i is :68058i is :68059i is :68060i is :68061i is :68062i is :68063i is :68064i is :68065i is :68066i is :68067i is :68068i is :68069i is :68070i is :68071i is :68072i is :68073i is :68074i is :68075i is :68076i is :68077i is :68078i is :68079i is :68080i is :68081i is :68082i is :68083i is :68084i is :68085i is :68086i is :68087i is :68088i is :68089i is :68090i is :68091i is :68092i is :68093i is :68094i is :68095i is :68096i is :68097i is :68098i is :68099i is :68100i is :68101i is :68102i is :68103i is :68104i is :68105i is :68106i is :68107i is :68108i is :68109i is :68110i is :68111i is :68112i is :68113i is :68114i is :68115i is :68116i is :68117i is :68118i is :68119i is :68120i is :68121i is :68122i
```

do..while loop

The **do..while** loop can also be used to create the infinite loop. The following is the syntax to create the infinite **do..while** loop.

1. **do**
2. {
3. *// body of the loop..*
4. }**while**(1);

The above do..while loop represents the infinite condition as we provide the '1' value inside the loop condition. As we already know that non-zero integer represents the true condition, so this loop will run infinite times.

goto statement

We can also use the goto statement to define the infinite loop.

1. `infinite_loop;`
2. `// body statements.`
3. `goto infinite_loop;`

In the above code, the goto statement transfers the control to the infinite loop.

Macros

We can also create the infinite loop with the help of a macro constant. Let's understand through an example.

```

1. #include <stdio.h>
2. #define infinite for(;;)
3. int main()
4. {
5.
6.     infinite
7.     {
8.         printf("hello");
9.     }
10.
11.     return 0;
12.}

```

In the above code, we have defined a macro named as 'infinite', and its value is 'for(;;)'. Whenever the word 'infinite' comes in a program then it will be replaced with a 'for(;;)'.

ADVERTISEMENT

ADVERTISEMENT

Output



The screenshot shows a terminal window with a title bar containing a maximize button, a close button, and a label 'input'. The terminal content consists of the word 'hello' printed repeatedly on multiple lines, indicating an infinite loop. The text is white on a black background.

Till now, we have seen various ways to define an infinite loop. However, we need some approach to come out of the infinite loop. In order to come out of the infinite loop, we can use the break statement.

Let's understand through an example.

```
1. #include <stdio.h>
2. int main()
3. {
4.     char ch;
5.     while(1)
6.     {
7.         ch=getchar();
8.         if(ch=='n')
9.         {
10.            break;
11.        }
12.        printf("hello");
13.    }
14.    return 0;
15.}
```

In the above code, we have defined the while loop, which will execute an infinite number of times until we press the key 'n'. We have added the 'if' statement inside the while loop. The 'if' statement contains the break keyword, and the break keyword brings control out of the loop.

Unintentional infinite loops

Sometimes the situation arises where unintentional infinite loops occur due to the bug in the code. If we are the beginners, then it becomes very difficult to trace them. Below are some measures to trace an unintentional infinite loop:

- We should examine the semicolons carefully. Sometimes we put the semicolon at the wrong place, which leads to the infinite loop.
- ```
1. #include <stdio.h>
2. int main()
```

```
3. {
4. int i=1;
5. while(i<=10);
6. {
7. printf("%d", i);
8. i++;
9. }
10. return 0;
11.}
```

In the above code, we put the semicolon after the condition of the while loop which leads to the infinite loop. Due to this semicolon, the internal body of the while loop will not execute.

- We should check the logical conditions carefully. Sometimes by mistake, we place the assignment operator (=) instead of a relational operator (==).

```
1. #include <stdio.h>
2. int main()
3. {
4. char ch='n';
5. while(ch='y')
6. {
7. printf("hello");
8. }
9. return 0;
10.}
```

In the above code, we use the assignment operator (ch='y') which leads to the execution of loop infinite number of times.

- ```
1. #include <stdio.h>
2. int main()
3. {
4.     for(int i=1;i<=1;i++)
5.     {
6.         printf("hello");
7.     }
8.     return 0;
9. }
```

- We should be careful when we are using the **break** keyword in the nested loop because it will terminate the execution of the nearest loop, not the entire loop.

1. `#include <stdio.h>`
2. `int main()`
3. `{`
4. `while(1)`
5. `{`
6. `for(int i=1;i<=10;i++)`
7. `{`
8. `if(i%2==0)`
9. `{`
10. `break;`
11. `}`
12. `}`

```
13. }  
14. return 0;  
15.}
```

In the above code, the while loop will be executed an infinite number of times as we use the break keyword in an inner loop. This break keyword will bring the control out of the inner loop, not from the outer loop.

- We should be very careful when we are using the floating-point value inside the loop as we cannot underestimate the floating-point errors.

```
1. #include <stdio.h>  
2. int main()  
3. {  
4. float x = 3.0;  
5. while (x != 4.0) {  
6. printf("x = %f\n", x);  
7. x += 0.1;  
8. }  
9. return 0;  
10.}
```

In the above code, the loop will run infinite times as the computer represents a floating-point value as a real value. The computer will represent the value of 4.0 as 3.999999 or 4.000001, so the condition (x !=4.0) will never be false. The solution to this problem is to write the condition as (k<=4.0).

C break statement

The break is a keyword in C which is used to bring the program control out of the loop. The break statement is used inside loops or switch statement. The break statement breaks the loop one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops. The break statement in C can be used in the following two scenarios:

1. With switch case
2. With loop

Syntax:

1. //loop or switch case
2. **break**;

Flowchart of break in c

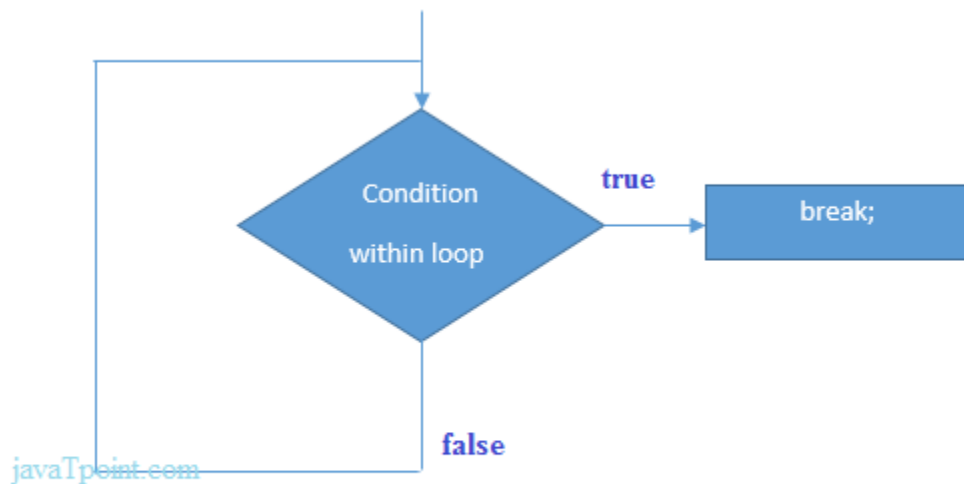


Figure: Flowchart of break statement

Example

1. `#include<stdio.h>`
2. `#include<stdlib.h>`
3. `void main ()`
4. `{`
5. `int i;`
6. `for(i = 0; i<10; i++)`
7. `{`

```
8.     printf("%d ",i);
9.     if(i == 5)
10.    break;
11. }
12. printf("came outside of loop i = %d",i);
13.
14. }
```

Output

```
0 1 2 3 4 5 came outside of loop i = 5
```

How does the break statement work?

The **"break" statement** works similarly to other programming languages in C programming. A control flow statement is used to **exit** a **loop** or **switch statement** early when a specific condition is met. The **"break" statement** is beneficial when you want to terminate a **loop** early or exit a **switch block** before it ends. The process of the **"break" statement** works in C is the same as in other programming languages:

- The **loop (while or for)** or the **switch** (or a series of **"if-else" statements**) begins execution.
- The program analyzes the condition within the **"if"** statement that includes the **"break"** statement throughout each iteration of the loop or each check in the **switch**.
- If the condition is **"true"**, the **"break"** statement is performed.
- When a **"break"** statement is found, the program immediately quits the loop or **switch block**, skipping any remaining iterations or checks.
- The program continues to execute the code following the **loop** or **switch**

Note: The *"break" statement* only affects the *innermost loop* or *switch block* which is contained within the statement. If there are nested loops or switch statements, the nearest one that includes the *"break" statement* will be broken out of.

Without the *"break"* statement, the loop or switch would continue its normal execution and process all the remaining iterations or checks, even if the desired condition has already been met. The *"break"* statement provides an efficient way to exit *loops* or *switch* blocks when you no longer need to perform further iterations or checks.

Backward Skip 10s
Play Video
Forward Skip 10s

Use of break statements in different cases with their examples:

Let us go through each use case of the *"break" statement* in C with detailed explanations and examples:

- **Simple Loops**
- **Nested Loops**
- **Infinite Loops**
- **Switch case**

1. Simple Loops:

When a specific condition is fulfilled, the *"break"* statement is widely used in simple loops like *"while"* and *"for"* to stop the loop early. It is helpful when you wish to terminate the loop early due to a condition.

Syntax:

It has the following syntax:

While loop

1. **while** (condition) {
2. **// Code block inside the loop**

3. **if** (some_condition) {
4. **break**; // Exit the loop if this condition is met
5. }
6. // Rest of the loop's code
7. }

For loop

1. **for** (initialization; condition; increment) {
2. // Code block inside the loop
3. **if** (some_condition) {
4. **break**; // Exit the loop if this condition is met
5. }
6. // Rest of the loop's code
7. }

Example:

Let's take an example to understand the use of the **break statement** in **simple loops** in C:

1. // Using break in a while loop
2. **#include** <stdio.h>
3. **int** main() {
4. inti = 1;
5. **while** (i<= 10) {
6. **if** (i == 5) {
7. **break**; // Exit the loop when i becomes 5
8. }
9. printf("%d ", i);
10. i++;
11. }
12. printf("\n");
13. **return** 0;

```
14. }
```

Output

```
1 2 3 4
```

Explanation:

In this example, the **"while"** loop outputs the **digits 1** through **4**. When ***i* equals 5**, the **"if"** condition if (***i* == 5**) is **true**, and the **"break" expression** is performed, forcing the loop to finish early.

2. Nested Loops:

The **break statement** can be applied in **nested loops** to break out of the **inner** and **outer** loops simultaneously when a specific condition is met. It allows you to stop processing further iterations in **multiple loops** at once.

Syntax:

It has the following syntax:

1. **for** (outer_loop_initialization; outer_loop_condition; outer_loop_increment) {
2. // Code block of the outer loop
3. **for** (inner_loop_initialization; inner_loop_condition; inner_loop_increment) {
4. // Code block of the inner loop
5. **if** (some_condition) {
6. **break**; // Exit both inner and outer loops if this condition is met
7. }
8. // Rest of the inner loop's code
9. }
10. // Rest of the outer loop's code
11. }

Example:

Let's take an example to understand the use of the **break statement** in **nested loops** in C:

```
1. // Using break in nested loops
2. #include <stdio.h>
3. int main() {
4.     for (inti = 1; i<= 3; i++) {
5.         for (int j = 1; j <= 3; j++) {
6.             if (i == 2 && j == 2) {
7.                 break; // Exit both loops when i=2 and j=2
8.             }
9.             printf("(%d, %d) ", i, j);
10.        }
11.    }
12.    printf("\n");
13.    return 0;
14. }
```

Output

(1, 1) (1, 2) (1, 3)

ADVERTISEMENT

ADVERTISEMENT

Explanation:

In this example, the **nested loops** output pairs of integers ranging from **1 to 3**. When **i** equals **2** and **j** equals **2**, the **"if" condition** if (**i == 2 && j == 2**) is **true**, and the **"break"** statement is run, causing both loops to end at the same time.

3. Infinite Loops:

An **infinite loop** runs continuously unless terminated by a **"break" statement** or another condition within the **loop**. In **infinite loops**, the **"break" statement** is typically used to give a means to leave the loop based on specified criteria.

Syntax:

It has the following syntax:

1. **while** (1) { // Infinite loop using a true condition
2. // Code block inside the loop
3. **if** (some_condition) {
4. **break**; // Exit the loop if this condition is met
5. }
6. // Rest of the loop's code
7. }

Example:

Let's take an example to understand the use of the **break statement** in **infinite loops** in C:

1. // Using break in an infinite loop
2. **#include** <stdio.h>
3. **int** main() {
4. **int** number;
5. **while** (1) {
6. printf("Enter a number (0 to exit): ");
7. scanf("%d", &number);
8. **if** (number == 0) {
9. **break**; // Exit the loop when the user enters 0
10. }
11. printf("You entered: %d\n", number);
12. }
13. **return** 0;
14. }

Output

Enter a number (0 to exit): 7

You entered: 7

Enter a number (0 to exit): 5

You entered: 5

Enter a number (0 to exit): 0

Explanation:

In this example, the program keeps asking the user to enter a number in an **infinite loop**. If the user enters **0**, the **"if" condition** if (**number == 0**) is true, and the **"break"** statement is executed, ending the loop and terminating the program.

4. Switch Case:

The **"break" statement** is used in a **"switch" statement** to exit the switch block after a particular case is executed. Without the **"break" statement**, the program would continue executing the code for all the subsequent cases, potentially leading to unexpected behavior.

Syntax:

It has the following syntax:

1. **switch** (expression) {
2. **case** value1:
3. // Code block for case value1
4. **break**; // Exit the switch block after executing this case
5. **case** value2:
6. // Code block for case value2
7. **break**; // Exit the switch block after executing this case
8. // More cases...
9. **default**:
10. // Code block for the default case
11. **break**; // Exit the switch block after executing this case
12. }

Example:

Let's take an example to understand the use of the **break statement** in the **switch case** in C:

1. // Using break in a switch statement
2. #include <stdio.h>

```
3. int main() {
4. int choice;
5. printf("Menu:\n");
6. printf("1. Option 1\n");
7. printf("2. Option 2\n");
8. printf("3. Quit\n");
9. printf("Enter your choice: ");
10. scanf("%d", &choice);
11.
12. switch (choice) {
13. case 1:
14. printf("You selected Option 1\n");
15. break;
16. case 2:
17. printf("You selected Option 2\n");
18. break;
19. case 3:
20. printf("Quitting the menu...\n");
21. break;
22. default:
23.
24. printf("Invalid choice. Try one more time.\n");
25. break;
26. }
27. return 0;
28. }
```

Output (Example 1):

Menu:
Option 1
Option 2

```
Quit
Enter your choice: 2
You selected Option 2
```

Output (Example 2):

```
Menu:
Option 1
Option 2
Quit
Enter your choice: 4
Invalid choice. Try one more time.
```

Explanation:

In this case, the program presents a **menu & prompts** the user to select an option. The matching **"case" block** is run based on the user's input. After the execution of the corresponding **"case" block**, the **"break" statement** exits the switch block, preventing the program from executing the code for other cases.

C continue statement

The **continue statement** in C language is used to bring the program control to the beginning of the loop. The continue statement skips some lines of code inside the loop and continues with the next iteration. It is mainly used for a condition so that we can skip some code for a particular condition.

Syntax:

1. `//loop statements`
2. `continue;`
3. `//some lines of the code which is to be skipped`

Continue statement example 1

1. `#include<stdio.h>`
2. `void main ()`

```
3. {
4.     int i = 0;
5.     while(i!=10)
6.     {
7.         printf("%d", i);
8.         continue;
9.         i++;
10.    }
11.}
```

Output

infinite loop

Continue statement example 2

```
1. #include<stdio.h>
2. int main(){
3.     int i=1;//initializing a local variable
4.     //starting a loop from 1 to 10
5.     for(i=1;i<=10;i++){
6.         if(i==5){//if value of i is equal to 5, it will continue the loop
7.             continue;
8.         }
9.         printf("%d \n",i);
10.    }//end of for loop
11.    return 0;
12.}
```

Output

```
1
2
3
4
6
7
8
9
10
```

As you can see, 5 is not printed on the console because loop is continued at `i==5`.

C continue statement with inner loop

In such case, C continue statement continues only inner loop, but not outer loop.

```
1. #include<stdio.h>
2. int main(){
3.     int i=1,j=1;//initializing a local variable
4.     for(i=1;i<=3;i++){
5.         for(j=1;j<=3;j++){
6.             if(i==2 && j==2){
7.                 continue;//will continue loop of j only
8.             }
9.             printf("%d %d\n",i,j);
10.        }
11.    }//end of for loop
12.    return 0;
13.}
```

Output

```
1 1
```

```
1 2
1 3
2 1
2 3
3 1
3 2
3 3
```

As you can see, 2 2 is not printed on the console because inner loop is continued at `i==2` and `j==2`.

C goto statement

The goto statement is known as jump statement in C. As the name suggests, goto is used to transfer the program control to a predefined label. The goto statement can be used to repeat some part of the code for a particular condition. It can also be used to break the multiple loops which can't be done by using a single break statement. However, using goto is avoided these days since it makes the program less readable and complicated.

Syntax:

1. label:
2. `//some part of the code;`
3. `goto` label;

goto example

Let's see a simple example to use goto statement in C language.

1. `#include <stdio.h>`
2. `int` main()
3. {
4. `int` num,i=1;

```
5. printf("Enter the number whose table you want to print?");
6. scanf("%d",&num);
7. table:
8. printf("%d x %d = %d\n",num,i,num*i);
9. i++;
10. if(i<=10)
11. goto table;
12.}
```

Output:

Enter the number whose table you want to print?10

```
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
10 x 4 = 40
10 x 5 = 50
10 x 6 = 60
10 x 7 = 70
10 x 8 = 80
10 x 9 = 90
10 x 10 = 100
```

When should we use goto?

The only condition in which using goto is preferable is when we need to break the multiple loops using a single statement at the same time. Consider the following example.

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i, j, k;
```

```
5.  for(i=0;i<10;i++)
6.  {
7.      for(j=0;j<5;j++)
8.      {
9.          for(k=0;k<3;k++)
10.         {
11.             printf("%d %d %d\n",i,j,k);
12.             if(j == 3)
13.             {
14.                 goto out;
15.             }
16.         }
17.     }
18. }
19. out:
20. printf("came out of the loop");
21. }
```

0 0 0

0 0 1

0 0 2

0 1 0

0 1 1

0 1 2

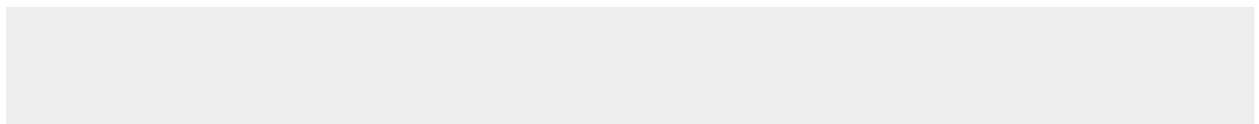
0 2 0

0 2 1

0 2 2

0 3 0

came out of the loop



Type Casting in C

Typecasting allows us to convert one data type into other. In C language, we use cast operator for typecasting which is denoted by (type).

Syntax:

1. (type)value;

Note: It is always recommended to convert the lower value to higher for avoiding data loss.

Without Type Casting:

1. `int f= 9/4;`
2. `printf("f : %d\n", f );//Output: 2`

With Type Casting:

1. `float f=(float) 9/4;`
2. `printf("f : %f\n", f );//Output: 2.250000`

Type Casting example

Let's see a simple example to cast int value into the float.

1. `#include<stdio.h>`
2. `int main(){`
3. `float f= (float)9/4;`
4. `printf("f : %f\n", f );`
5. `return 0;`
6. `}`

Output:

f : 2.250000

C Strings

The string can be defined as the one-dimensional array of characters terminated by a null ('\0'). The character array or the string is used to manipulate text such as word or sentences. Each character in the array occupies one byte of memory, and the last character must always be 0. The termination character ('\0') is important in a string since it is the only way to identify where the string ends. When we define a string as `char s[10]`, the character `s[10]` is implicitly initialized with the null in the memory.

There are two ways to declare a string in c language.

1. By char array
2. By string literal

Let's see the example of declaring **string by char array** in C language.

1. `char ch[10]={'j','a','v','a','t','p','o','i','n','t','\0'};`

As we know, array index starts from 0, so it will be represented as in the figure given below.

0	1	2	3	4	5	6	7	8	9	10
j	a	v	a	t	p	o	i	n	t	\0

While declaring string, size is not mandatory. So we can write the above code as given below:

1. `char ch[]={'j','a','v','a','t','p','o','i','n','t','\0'};`

We can also define the **string by the string literal** in C language. For example:

1. `char ch[]="javatpoint";`

In such case, '\0' will be appended at the end of the string by the compiler.

Difference between char array and string literal

There are two main differences between char array and literal.

- We need to add the null character '\0' at the end of the array by ourself whereas, it is appended internally by the compiler in the case of the character array.
- The string literal cannot be reassigned to another set of characters whereas, we can reassign the characters of the array.

String Example in C

Let's see a simple example where a string is declared and being printed. The '%s' is used as a format specifier for the string in c language.

```
1. #include<stdio.h>
2. #include <string.h>
3. int main(){
4.     char ch[11]={'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0'};
5.     char ch2[11]="javatpoint";
6.
7.     printf("Char Array Value is: %s\n", ch);
8.     printf("String Literal Value is: %s\n", ch2);
9.     return 0;
10.}
```

Output

Char Array Value is: javatpoint

String Literal Value is: javatpoint

Traversing String

Traversing the string is one of the most important aspects in any of the programming languages. We may need to manipulate a very large text which can be done by traversing the text. Traversing string is somewhat different from the traversing an integer array. We need to know the length of the array to traverse an integer array, whereas we may use the null character in the case of string to identify the end of the string and terminate the loop.

Hence, there are two ways to traverse a string.

- By using the length of string
- By using the null character.

Let's discuss each one of them.

Using the length of string

Let's see an example of counting the number of vowels in a string.

```
1. #include<stdio.h>
2. void main ()
3. {
4.     char s[11] = "javatpoint";
5.     int i = 0;
6.     int count = 0;
7.     while(i<11)
8.     {
9.         if(s[i]=='a' || s[i] == 'e' || s[i] == 'i' || s[i] == 'u' || s[i] == 'o')
10.        {
11.            count ++;
12.        }
```

```
13.     i++;
14. }
15. printf("The number of vowels %d",count);
16.}
```

Output

The number of vowels 4

Using the null character

Let's see the same example of counting the number of vowels by using the null character.

```
1. #include<stdio.h>
2. void main ()
3. {
4.     char s[11] = "javatpoint";
5.     int i = 0;
6.     int count = 0;
7.     while(s[i] != NULL)
8.     {
9.         if(s[i]=='a' || s[i] == 'e' || s[i] == 'i' || s[i] == 'u' || s[i] == 'o')
10.        {
11.            count ++;
12.        }
13.        i++;
14.    }
15.    printf("The number of vowels %d",count);
16.}
```

Output

The number of vowels 4

Accepting string as the input

Till now, we have used `scanf` to accept the input from the user. However, it can also be used in the case of strings but with a different scenario. Consider the below code which stores the string while space is encountered.

```
1. #include<stdio.h>
2. void main ()
3. {
4.     char s[20];
5.     printf("Enter the string?");
6.     scanf("%s",s);
7.     printf("You entered %s",s);
8. }
```

Output

Enter the string?javatpoint is the best

You entered javatpoint

It is clear from the output that, the above code will not work for space separated strings. To make this code working for the space separated strings, the minor change required in the `scanf` function, i.e., instead of writing `scanf("%s",s)`, we must write: `scanf("%[^\n]s",s)` which instructs the compiler to store the string `s` while the new line (`\n`) is encountered. Let's consider the following example to store the space-separated strings.

```
1. #include<stdio.h>
```

```
2. void main ()
3. {
4.     char s[20];
5.     printf("Enter the string?");
6.     scanf("%[^\n]s",s);
7.     printf("You entered %s",s);
8. }
```

Output

Enter the string?javatpoint is the best

You entered javatpoint is the best

Here we must also notice that we do not need to use address of (&) operator in scanf to store a string since string s is an array of characters and the name of the array, i.e., s indicates the base address of the string (character array) therefore we need not use & with it.

Some important points

However, there are the following points which must be noticed while entering the strings by using scanf.

- The compiler doesn't perform bounds checking on the character array. Hence, there can be a case where the length of the string can exceed the dimension of the character array which may always overwrite some important data.
- Instead of using scanf, we may use gets() which is an inbuilt function defined in a header file string.h. The gets() is capable of receiving only one string at a time.

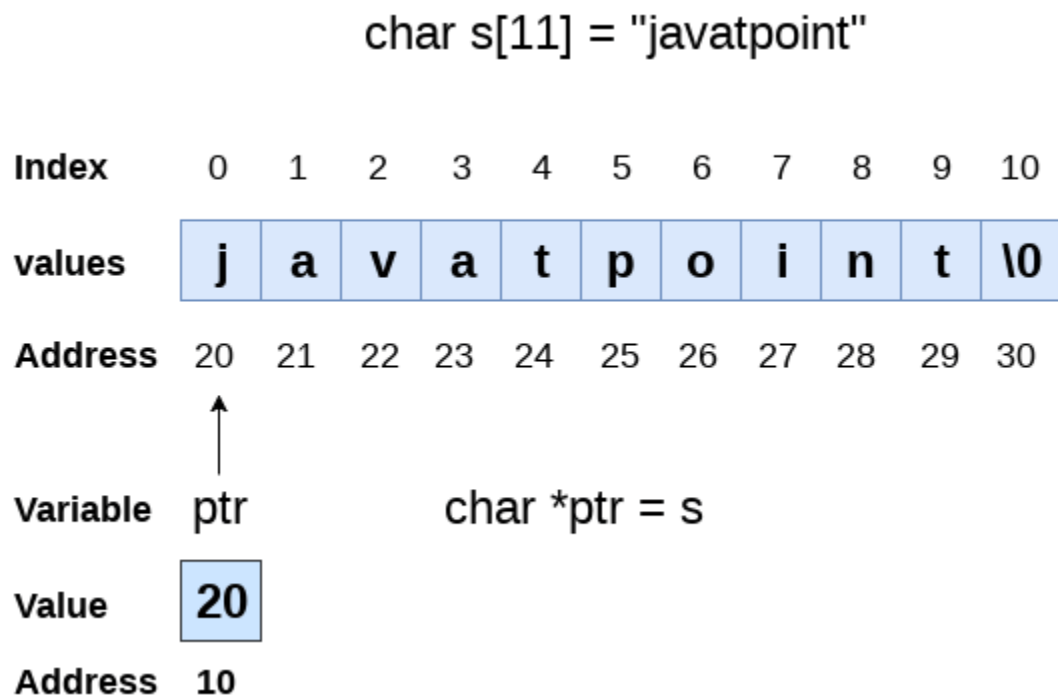
Pointers with strings

We have used pointers with the array, functions, and primitive data types so far. However, pointers can be used to point to the strings. There are various advantages of using pointers to point strings. Let us consider the following example to access the string via the pointer.

1. `#include<stdio.h>`
2. `void main ()`
3. `{`
4. `char s[11] = "javatpoint";`
5. `char *p = s; // pointer p is pointing to string s.`
6. `printf("%s",p); // the string javatpoint is printed if we print p.`
7. `}`

Output

javatpoint



As we know that string is an array of characters, the pointers can be used in the same way they were used with arrays. In the above example, p is declared as a pointer to the array of characters s. P affects similar to s since s is the base address of the string and treated as a pointer internally. However, we can not change the content of s or copy the content of s into another string directly. For this purpose, we need to use the pointers to store the strings. In the following example, we have shown the use of pointers to copy the content of a string into another.

```
1. #include<stdio.h>
2. void main ()
3. {
4.     char *p = "hello javatpoint";
5.     printf("String p: %s\n",p);
6.     char *q;
7.     printf("copying the content of p into q...\n");
8.     q = p;
9.     printf("String q: %s\n",q);
10. }
```

Output

String p: hello javatpoint

copying the content of p into q...

String q: hello javatpoint

Once a string is defined, it cannot be reassigned to another set of characters. However, using pointers, we can assign the set of characters to the string. Consider the following example.

```
1. #include<stdio.h>
2. void main ()
3. {
```

```
4.  char *p = "hello javatpoint";
5.  printf("Before assigning: %s\n",p);
6.  p = "hello";
7.  printf("After assigning: %s\n",p);
8. }
```

Output

Before assigning: hello javatpoint

After assigning: hello

C gets() and puts() functions

The gets() and puts() are declared in the header file stdio.h. Both the functions are involved in the input/output operations of the strings.

C gets() function

The gets() function enables the user to enter some characters followed by the enter key. All the characters entered by the user get stored in a character array. The null character is added to the array to make it a string. The gets() allows the user to enter the space-separated strings. It returns the string entered by the user.

Declaration

```
1. char[] gets(char[]);
```

Reading string using gets()

```
1. #include<stdio.h>
2. void main ()
```

```
3. {
4.   char s[30];
5.   printf("Enter the string? ");
6.   gets(s);
7.   printf("You entered %s",s);
8. }
```

Output

Enter the string?

javatpoint is the best

You entered javatpoint is the best

The gets() function is risky to use since it doesn't perform any array bound checking and keep reading the characters until the new line (enter) is encountered. It suffers from buffer overflow, which can be avoided by using fgets(). The fgets() makes sure that not more than the maximum limit of characters are read. Consider the following example.

```
1. #include<stdio.h>
2. void main()
3. {
4.   char str[20];
5.   printf("Enter the string? ");
6.   fgets(str, 20, stdin);
7.   printf("%s", str);
8. }
```

Output

Enter the string? javatpoint is the best website

javatpoint is the b

C puts() function

The puts() function is very much similar to printf() function. The puts() function is used to print the string on the console which is previously read by using gets() or scanf() function. The puts() function returns an integer value representing the number of characters being printed on the console. Since, it prints an additional newline character with the string, which moves the cursor to the new line on the console, the integer value returned by puts() will always be equal to the number of characters present in the string plus 1.

Declaration

1. `int puts(char[])`

Let's see an example to read a string using gets() and print it on the console using puts().

1. `#include<stdio.h>`
2. `#include <string.h>`
3. `int main(){`
4. `char name[50];`
5. `printf("Enter your name: ");`
6. `gets(name); //reads string from user`
7. `printf("Your name is: ");`
8. `puts(name); //displays string`
9. `return 0;`
10. `}`

Output:

Enter your name: Sonoo Jaiswal

Your name is: Sonoo Jaiswal

C String Functions

There are many important string functions defined in "string.h" library.

N o .	Function	Description
	1) <code>strlen(string_name)</code>	returns the length of string name.
	2) <code>strcpy(destination, source)</code>	copies the contents of source string to destination string.
	3) <code>strcat(first_string, second_string)</code>	concatenates or joins first string with second string. The result of the string is stored in first string.
	4) <code>strcmp(first_string, second_string)</code>	compares the first string with second string. If both strings are same, it returns 0.
	5) <code>strrev(string)</code>	returns reverse string.
	6) <code>strlwr(string)</code>	returns string characters in lowercase.
7)	<code>strupr(string)</code>	returns string characters in uppercase.

C String Length: `strlen()` function

The `strlen()` function returns the length of the given string. It doesn't count null character `'\0'`.

```
1. #include<stdio.h>
2. #include <string.h>
3. int main(){
4.  char ch[20]={ 'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0' };
5.  printf("Length of string is: %d",strlen(ch));
6.  return 0;
7. }
```

Output:

Length of string is: 10

1. **strlen()** – Length of String

Exercise 1

Write a program to find the length of the string "HELLO".

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "HELLO";
    int len = strlen(str);

    printf("Length = %d", len);
    return 0;
}
```

Output:

Length = 5

Exercise 2

Take a string from user and print its length.

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];

    printf("Enter string: ");
    gets(str);

    printf("Length = %d", strlen(str));
    return 0;
}
```

2. strcpy() – Copy String

Exercise 1

Copy "C Programming" into another string.

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "C Programming";
```

```
char dest[50];

strcpy(dest, src);

printf("Copied String = %s", dest);
return 0;
}
```

Exercise 2

Copy user input into another variable and print both.

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];

    printf("Enter string: ");
    gets(str1);

    strcpy(str2, str1);

    printf("Original = %s\n", str1);
    printf("Copied = %s", str2);

    return 0;
}
```

3. **strcat()** – Concatenate Strings

Exercise 1

Join "Hello" and "World".

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50] = "Hello ";
    char str2[] = "World";

    strcat(str1, str2);

    printf("Result = %s", str1);
    return 0;
}
```

Exercise 2

Join two user-input strings.

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];

    printf("Enter first string: ");
    gets(str1);

    printf("Enter second string: ");
    gets(str2);

    strcat(str1, str2);
}
```

```
printf("Concatenated = %s", str1);  
return 0;  
}
```

4. **strcmp()** – Compare Strings

Exercise 1

Compare "ABC" and "ABC".

✓ **Answer:**

```
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char str1[] = "ABC";  
    char str2[] = "ABC";  
  
    if (strcmp(str1, str2) == 0)  
        printf("Strings are equal");  
    else  
        printf("Strings are not equal");  
  
    return 0;  
}
```

Exercise 2

Compare two user strings.

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];

    printf("Enter first string: ");
    gets(str1);

    printf("Enter second string: ");
    gets(str2);

    if (strcmp(str1, str2) == 0)
        printf("Equal");
    else
        printf("Not Equal");

    return 0;
}
```

5. **strrev()** – Reverse String

Exercise

Reverse "HELLO".

✓ **Answer:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "HELLO";

    strrev(str);
```

```
printf("Reversed = %s", str);  
return 0;  
}
```

6. **strlwr()** – Lowercase

Exercise

Convert "HELLO" to lowercase.

✓ **Answer:**

```
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char str[] = "HELLO";  
  
    strlwr(str);  
  
    printf("Lowercase = %s", str);  
    return 0;  
}
```

7. **strupr()** – Uppercase

Exercise

Convert "hello" to uppercase.

Answer:

```
#include <stdio.h>
```

```
#include <string.h>

int main() {
    char str[] = "hello";

    strupr(str);

    printf("Uppercase = %s", str);
    return 0;
}
```

Important Notes (Very Important for Exams)

- `gets()` is unsafe → use `fgets()` in real programs
 - `strrev()`, `strlwr()`, `strupr()` are **not standard in all compilers** (mostly work in Turbo C / older compilers)
 - Always ensure destination array has enough size in `strcpy()` and `strcat()`
-

C Math

C Programming allows us to perform mathematical operations through the functions defined in `<math.h>` header file. The `<math.h>` header file contains various methods for performing mathematical operations such as `sqrt()`, `pow()`, `ceil()`, `floor()` etc.

C Math Functions

There are various methods in `math.h` header file. The commonly used functions of `math.h` header file are given below.

No.	Function	Description
1)	ceil(number)	rounds up the given number. It returns the integer value which is greater than or equal to given number.
2)	floor(number)	rounds down the given number. It returns the integer value which is less than or equal to given number.
3)	sqrt(number)	returns the square root of given number.
4)	pow(base, exponent)	returns the power of given number.
5)	abs(number)	returns the absolute value of given number.

C Math Example

Let's see a simple example of math functions found in math.h header file.

1. `#include<stdio.h>`
2. `#include <math.h>`
3. `int main(){`
4. `printf("\n%f",ceil(3.6));`
5. `printf("\n%f",ceil(3.3));`
6. `printf("\n%f",floor(3.6));`
7. `printf("\n%f",floor(3.2));`

```
8. printf("\n%f",sqrt(16));
9. printf("\n%f",sqrt(7));
10.printf("\n%f",pow(2,4));
11.printf("\n%f",pow(3,3));
12.printf("\n%d",abs(-12));
13. return 0;
14.}
```

Output:

```
4.000000
4.000000
3.000000
3.000000
4.000000
2.645751
16.000000
27.000000
12
```

C Functions

In c, we can divide a large program into the basic building blocks known as function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the C program. In other words, we can say that the collection of functions creates a program. The function is also known as *procedure* or *subroutine* in other programming languages.

Advantage of functions in C

There are the following advantages of C functions.

- By using functions, we can avoid rewriting same logic/code again and again in a program.
- We can call C functions any number of times in a program and from any place in a program.
- We can track a large C program easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.
- However, Function calling is always a overhead in a C program.

Function Aspects

There are three aspects of a C function.

- **Function declaration** A function must be declared globally in a c program to tell the compiler about the function name, function parameters, and return type.
- **Function call** Function can be called from anywhere in the program. The parameter list must not differ in function calling and function declaration. We must pass the same number of functions as it is declared in the function declaration.
- **Function definition** It contains the actual statements which are to be executed. It is the most important aspect to which the control comes when the function is called. Here, we must notice that only one value can be returned from the function.

S N	C function aspects	Syntax
1	Function declaration	return_type function_name (argument list);
2	Function call	function_name (argument_list)
3	Function definition	return_type function_name (argument list) {function body;}

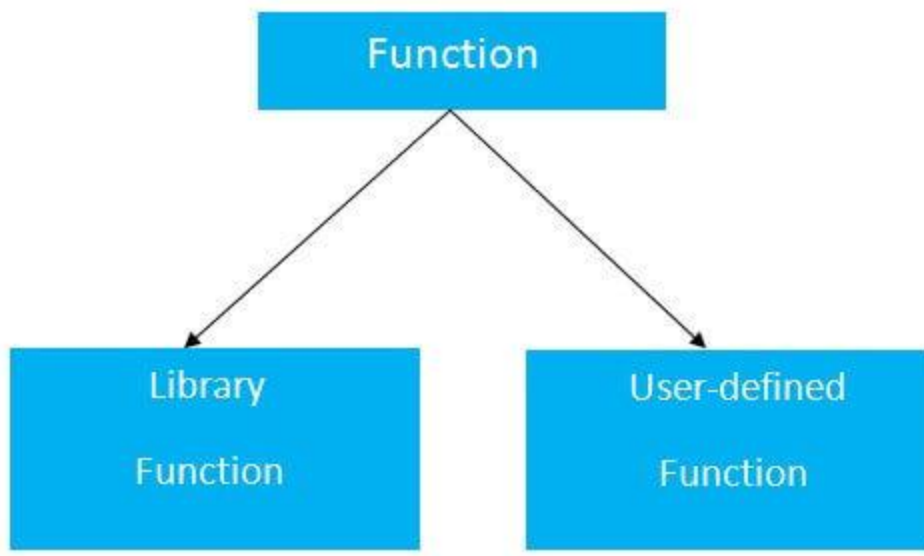
The syntax of creating function in c language is given below:

1. return_type function_name(data_type parameter...){
2. `//code to be executed`
3. }

Types of Functions

There are two types of functions in C programming:

1. **Library Functions:** are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc.
2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces the complexity of a big program and optimizes the code.



Return Value

A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

Let's see a simple example of C function that doesn't return any value from the function.

Example without return value:

1. `void hello(){`
2. `printf("hello c");`
3. `}`

If you want to return any value from the function, you need to use any data type such as int, long, char, etc. The return type depends on the value to be returned from the function.

Let's see a simple example of C function that returns int value from the function.

Example with return value:

1. `int get(){`
2. `return 10;`

3. }

In the above example, we have to return 10 as a value, so the return type is int. If you want to return floating-point value (e.g., 10.2, 3.1, 54.5, etc), you need to use float as the return type of the method.

```
1. float get(){  
2. return 10.2;  
3. }
```

Now, you need to call the function, to get the value of the function.

Different aspects of function calling

A function may or may not accept any argument. It may or may not return any value. Based on these facts, There are four different aspects of function calls.

- function without arguments and without return value
- function without arguments and with return value
- function with arguments and without return value
- function with arguments and with return value

Example for Function without argument and return value

Example 1

```
1. #include<stdio.h>  
2. void printName();  
3. void main ()  
4. {  
5.     printf("Hello ");  
6.     printName();  
7. }
```

```
8. void printName()
9. {
10. printf("Javatpoint");
11.}
```

Output

Hello Javatpoint

Example 2

```
1. #include<stdio.h>
2. void sum();
3. void main()
4. {
5. printf("\nGoing to calculate the sum of two numbers:");
6. sum();
7. }
8. void sum()
9. {
10. int a,b;
11. printf("\nEnter two numbers");
12. scanf("%d %d",&a,&b);
13. printf("The sum is %d",a+b);
14.}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

Example for Function without argument and with return value

Example 1

```
1. #include<stdio.h>
2. int sum();
3. void main()
4. {
5.     int result;
6.     printf("\nGoing to calculate the sum of two numbers:");
7.     result = sum();
8.     printf("%d",result);
9. }
10.int sum()
11.{
12.    int a,b;
13.    printf("\nEnter two numbers");
14.    scanf("%d %d",&a,&b);
15.    return a+b;
16.}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

Example 2: program to calculate the area of the square

```
1. #include<stdio.h>
2. int sum();
3. void main()
4. {
5.     printf("Going to calculate the area of the square\n");
6.     float area = square();
7.     printf("The area of the square: %f\n",area);
8. }
9. int square()
10.{
11.    float side;
12.    printf("Enter the length of the side in meters: ");
13.    scanf("%f",&side);
14.    return side * side;
15.}
```

Output

```
Going to calculate the area of the square
Enter the length of the side in meters: 10
The area of the square: 100.000000
```

Example for Function with argument and without return value

ADVERTISEMENT

ADVERTISEMENT

Example 1

```
1. #include<stdio.h>
2. void sum(int, int);
```

```
3. void main()
4. {
5.     int a,b,result;
6.     printf("\nGoing to calculate the sum of two numbers:");
7.     printf("\nEnter two numbers:");
8.     scanf("%d %d",&a,&b);
9.     sum(a,b);
10.}
11. void sum(int a, int b)
12.{
13.    printf("\nThe sum is %d",a+b);
14.}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

Example 2: program to calculate the average of five numbers.

```
1. #include<stdio.h>
2. void average(int, int, int, int, int);
3. void main()
4. {
5.     int a,b,c,d,e;
6.     printf("\nGoing to calculate the average of five numbers:");
7.     printf("\nEnter five numbers:");
8.     scanf("%d %d %d %d %d",&a,&b,&c,&d,&e);
```

```
9.   average(a,b,c,d,e);
10.}
11. void average(int a, int b, int c, int d, int e)
12.{
13.   float avg;
14.   avg = (a+b+c+d+e)/5;
15.   printf("The average of given five numbers : %f",avg);
16.}
```

Output

Going to calculate the average of five numbers:

Enter five numbers:10

20

30

40

50

The average of given five numbers : 30.000000

Example for Function with argument and with return value

Example 1

```
1. #include<stdio.h>
2. int sum(int, int);
3. void main()
4. {
5.   int a,b,result;
6.   printf("\nGoing to calculate the sum of two numbers:");
7.   printf("\nEnter two numbers:");
8.   scanf("%d %d",&a,&b);
9.   result = sum(a,b);
```

```
10. printf("\nThe sum is : %d",result);
11.}
12.int sum(int a, int b)
13.{
14.    return a+b;
15.}
```

Output

Going to calculate the sum of two numbers:
Enter two numbers:10
20
The sum is : 30

Example 2: Program to check whether a number is even or odd

```
1. #include<stdio.h>
2. int even_odd(int);
3. void main()
4. {
5.    int n,flag=0;
6.    printf("\nGoing to check whether a number is even or odd");
7.    printf("\nEnter the number: ");
8.    scanf("%d",&n);
9.    flag = even_odd(n);
10. if(flag == 0)
11. {
12.    printf("\nThe number is odd");
13. }
14. else
15. {
```

```
16. printf("\nThe number is even");
17. }
18.}
19.int even_odd(int n)
20.{
21.    if(n%2 == 0)
22.    {
23.        return 1;
24.    }
25.    else
26.    {
27.        return 0;
28.    }
29.}
```

Output

Going to check whether a number is even or odd

Enter the number: 100

The number is even

C Library Functions

Library functions are the inbuilt function in C that are grouped and placed at a common place called the library. Such functions are used to perform some specific operations. For example, printf is a library function used to print on the console. The library functions are created by the designers of compilers. All C standard library functions are defined inside the different header files saved with the extension **.h**. We need to include these header files in our program to make use of the library functions defined in such header files. For example, To use the library functions such as printf/scanf we need to include `stdio.h` in our program which is a header file that contains all the library functions regarding standard input/output.

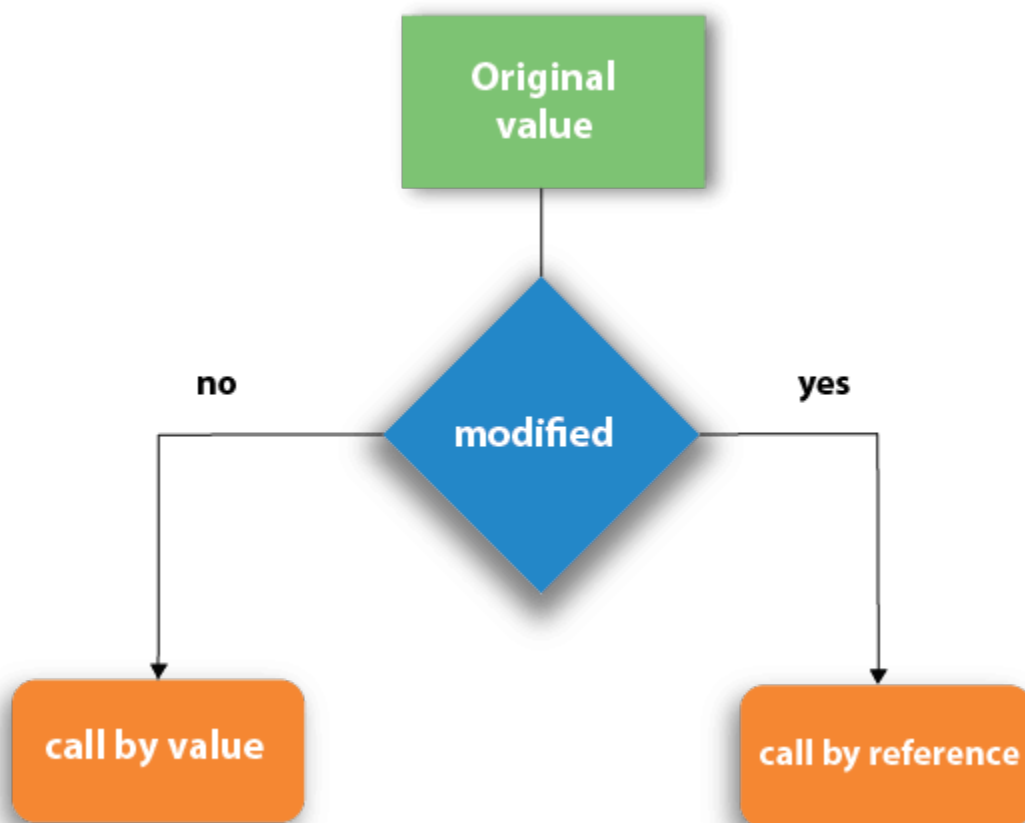
The list of mostly used header files is given in the following table.

S N	Header file	Description
1	stdio.h	This is a standard input/output header file. It contains all the library functions regarding standard input/output.
2	conio.h	This is a console input/output header file.
3	string.h	It contains all string related library functions like gets(), puts(),etc.
4	stdlib.h	This header file contains all the general library functions like malloc(), calloc(), exit(), etc.
5	math.h	This header file contains all the math operations related functions like sqrt(), pow(), etc.
6	time.h	This header file contains all the time-related functions.
7	ctype.h	This header file contains all character handling functions.
8	stdarg.h	Variable argument functions are defined in this header file.
9	signal.h	All the signal handling functions are defined in this header file.
10	setjmp.h	This file contains all the jump functions.
11	locale.h	This file contains locale functions.

12	errno.h	This file contains error handling functions.
13	assert.h	This file contains diagnostics functions.

Call by value and Call by reference in C

There are two methods to pass the data into the function in C language, i.e., *call by value* and *call by reference*.



Let's understand call by value and call by reference in c language one by one.

Call by value in C

ADVERTISEMENT

ADVERTISEMENT

- In call by value method, the value of the actual parameters is copied into the formal parameters. In other words, we can say that the value of the variable is used in the function call in the call by value method.
- In call by value method, we can not modify the value of the actual parameter by the formal parameter.
- In call by value, different memory is allocated for actual and formal parameters since the value of the actual parameter is copied into the formal parameter.
- The actual parameter is the argument which is used in the function call whereas formal parameter is the argument which is used in the function definition.

Let's try to understand the concept of call by value in c language by the example given below:

```
1. #include<stdio.h>
2. void change(int num) {
3.     printf("Before adding value inside function num=%d \n",num);
4.     num=num+100;
5.     printf("After adding value inside function num=%d \n", num);
6. }
7. int main() {
8.     int x=100;
9.     printf("Before function call x=%d \n", x);
10.    change(x); //passing value in function
11.    printf("After function call x=%d \n", x);
12. return 0;
```

13.}

Output

Before function call x=100

Before adding value inside function num=100

After adding value inside function num=200

After function call x=100

Call by Value Example: Swapping the values of the two variables

```
1. #include <stdio.h>
2. void swap(int , int); //prototype of the function
3. int main()
4. {
5.     int a = 10;
6.     int b = 20;
7.     printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the
        value of a and b in main
8.     swap(a,b);
9.     printf("After swapping values in main a = %d, b = %d\n",a,b); // The value of
        actual parameters do not change by changing the formal parameters in call by
        value, a = 10, b = 20
10.}
11. void swap (int a, int b)
12.{
13.     int temp;
14.     temp = a;
15.     a=b;
16.     b=temp;
```

```
17.  printf("After swapping values in function a = %d, b = %d\n",a,b); // Formal
    parameters, a = 20, b = 10
18.}
```

Output

Before swapping the values in main a = 10, b = 20

After swapping values in function a = 20, b = 10

After swapping values in main a = 10, b = 20

Call by reference in C

- In call by reference, the address of the variable is passed into the function call as the actual parameter.
- The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.
- In call by reference, the memory allocation is similar for both formal parameters and actual parameters. All the operations in the function are performed on the value stored at the address of the actual parameters, and the modified value gets stored at the same address.

Consider the following example for the call by reference.

ADVERTISEMENT

```
1. #include<stdio.h>
2. void change(int *num) {
3.     printf("Before adding value inside function num=%d \n",*num);
4.     (*num) += 100;
5.     printf("After adding value inside function num=%d \n", *num);
6. }
```

```

7. int main() {
8.     int x=100;
9.     printf("Before function call x=%d \n", x);
10.    change(&x); //passing reference in function
11.    printf("After function call x=%d \n", x);
12.    return 0;
13.}

```

Output

Before function call x=100

Before adding value inside function num=100

After adding value inside function num=200

After function call x=200

Call by reference Example: Swapping the values of the two variables

```

1. #include <stdio.h>
2. void swap(int *, int *); //prototype of the function
3. int main()
4. {
5.     int a = 10;
6.     int b = 20;
7.     printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the
        value of a and b in main
8.     swap(&a,&b);
9.     printf("After swapping values in main a = %d, b = %d\n",a,b); // The values of
        actual parameters do change in call by reference, a = 10, b = 20
10.}
11. void swap (int *a, int *b)
12.{
13.    int temp;

```

```

14. temp = *a;
15. *a=*b;
16. *b=temp;
17. printf("After swapping values in function a = %d, b = %d\n",*a,*b); // Formal
    parameters, a = 20, b = 10
18.}

```

Output

Before swapping the values in main a = 10, b = 20

After swapping values in function a = 20, b = 10

After swapping values in main a = 20, b = 10

Difference between call by value and call by reference in c

No.	Call by value	Call by reference
1	A copy of the value is passed into the function	An address of value is passed into the function
2	Changes made inside the function is limited to the function only. The values of the actual parameters do not change by changing the formal parameters.	Changes made inside the function validate outside of the function also. The values of the actual parameters do change by changing the formal parameters.

3	Actual and formal arguments are created at the different memory location	Actual and formal arguments are created at the same memory location
---	--	---

C Array

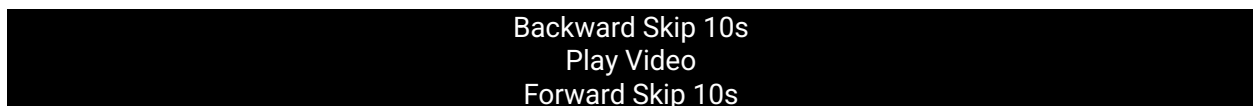
An array is defined as the collection of similar type of data items stored at contiguous memory locations. Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc. It also has the capability to store the collection of derived data types, such as pointers, structure, etc. The array is the simplest data structure where each data element can be randomly accessed by using its index number.

C array is beneficial if you have to store similar elements. For example, if we want to store the marks of a student in 6 subjects, then we don't need to define different variables for the marks in the different subject. Instead of that, we can define an array which can store the marks in each subject at the contiguous memory locations.

By using the array, we can access the elements easily. Only a few lines of code are required to access the elements of the array.

Properties of Array

The array contains the following properties.



ADVERTISEMENT

ADVERTISEMENT

- Each element of an array is of same data type and carries the same size, i.e., int = 4 bytes.

- Elements of the array are stored at contiguous memory locations where the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Advantage of C Array

- 1) **Code Optimization:** Less code to access the data.
- 2) **Ease of traversing:** By using the for loop, we can retrieve the elements of an array easily.
- 3) **Ease of sorting:** To sort the elements of the array, we need a few lines of code only.
- 4) **Random Access:** We can access any element randomly using the array.

Disadvantage of C Array

- 1) **Fixed Size:** Whatever size we define at the time of declaration of the array, we can't exceed the limit. So, it doesn't grow the size dynamically like LinkedList which we will learn later.

Declaration of C Array

We can declare an array in the C language in the following way.

```
1. data_type array_name[array_size];
```

Now, let us see the example to declare the array.

```
1. int marks[5];
```

Here, *int* is the *data_type*, *marks* are the *array_name*, and *5* is the *array_size*.

Initialization of C Array

The simplest way to initialize an array is by using the index of each element. We can initialize each element of the array by using the index. Consider the following example.

1. `marks[0]=80; //initialization of array`
2. `marks[1]=60;`
3. `marks[2]=70;`
4. `marks[3]=85;`
5. `marks[4]=75;`

80	60	70	85	75
<code>marks[0]</code>	<code>marks[1]</code>	<code>marks[2]</code>	<code>marks[3]</code>	<code>marks[4]</code>

Initialization of Array

C array example

1. `#include<stdio.h>`
2. `int main(){`
3. `int i=0;`
4. `int marks[5]; //declaration of array`
5. `marks[0]=80; //initialization of array`
6. `marks[1]=60;`
7. `marks[2]=70;`
8. `marks[3]=85;`
9. `marks[4]=75;`
10. `//traversal of array`
11. `for(i=0;i<5;i++){`
12. `printf("%d \n",marks[i]);`
13. `} //end of for loop`

```
14. return 0;
15.}
```

Output

```
80
60
70
85
75
```

C Array: Declaration with Initialization

We can initialize the c array at the time of declaration. Let's see the code.

```
1. int marks[5]={20,30,40,50,60};
```

In such case, there is **no requirement to define the size**. So it may also be written as the following code.

```
1. int marks[]={20,30,40,50,60};
```

Let's see the C program to declare and initialize the array in C.

```
1. #include<stdio.h>
2. int main(){
3.     int i=0;
4.     int marks[5]={20,30,40,50,60}; //declaration and initialization of array
5.     //traversal of array
6.     for(i=0;i<5;i++){
7.         printf("%d \n",marks[i]);
8.     }
9.     return 0;
10.}
```

Output

```
20
30
40
50
60
```

C Array Example: Sorting an array

In the following program, we are using bubble sort method to sort the array in ascending order.

```
1. #include<stdio.h>
2. void main ()
3. {
4.     int i, j,temp;
5.     int a[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
6.     for(i = 0; i<10; i++)
7.     {
8.         for(j = i+1; j<10; j++)
9.         {
10.            if(a[j] > a[i])
11.            {
12.                temp = a[i];
13.                a[i] = a[j];
14.                a[j] = temp;
15.            }
16.        }
17.    }
18.    printf("Printing Sorted Element List ...\n");
```

```
19.  for(i = 0; i<10; i++)
20.  {
21.      printf("%d\n",a[i]);
22.  }
23.}
```

Program to print the largest and second largest element of the array.

```
1.  #include<stdio.h>
2.  void main ()
3.  {
4.      int arr[100],i,n,largest,sec_largest;
5.      printf("Enter the size of the array?");
6.      scanf("%d",&n);
7.      printf("Enter the elements of the array?");
8.      for(i = 0; i<n; i++)
9.      {
10.         scanf("%d",&arr[i]);
11.     }
12.     largest = arr[0];
13.     sec_largest = arr[1];
14.     for(i=0;i<n;i++)
15.     {
16.         if(arr[i]>largest)
17.         {
18.             sec_largest = largest;
19.             largest = arr[i];
20.         }
```

```
21.     else if (arr[i]>sec_largest && arr[i]!=largest)
22.     {
23.         sec_largest=arr[i];
24.     }
25. }
26. printf("largest = %d, second largest = %d",largest,sec_largest);
27.
28.}
```

Two Dimensional Array in C

The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database lookalike data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

Declaration of two dimensional Array in C

The syntax to declare the 2D array is given below.

```
1. data_type array_name[rows][columns];
```

Consider the following example.

```
1. int twodimen[4][3];
```

Here, 4 is the number of rows, and 3 is the number of columns.

Initialization of 2D Array in C

In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously. However, this will not work with 2D arrays. We will have to define at least the second dimension of the array. The two-dimensional array can be declared and defined in the following way.

1. `int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};`

Two-dimensional array example in C

1. `#include<stdio.h>`
2. `int main(){`
3. `int i=0,j=0;`
4. `int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};`
5. `//traversing 2D array`
6. `for(i=0;i<4;i++){`
7. `for(j=0;j<3;j++){`
8. `printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);`
9. `}//end of j`
10. `}//end of i`
11. `return 0;`
12. `}`

Output

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6
```

C 2D array example: Storing elements in a matrix and printing it.

```
1. #include <stdio.h>
2. void main ()
3. {
4.     int arr[3][3],i,j;
5.     for (i=0;i<3;i++)
6.     {
7.         for (j=0;j<3;j++)
8.         {
9.             printf("Enter a[%d][%d]: ",i,j);
10.            scanf("%d",&arr[i][j]);
11.        }
12.    }
13.    printf("\n printing the elements ....\n");
14.    for(i=0;i<3;i++)
15.    {
16.        printf("\n");
17.        for (j=0;j<3;j++)
18.        {
19.            printf("%d\t",arr[i][j]);
20.        }
21.    }
22.}
```

Output

```
Enter a[0][0]: 56
Enter a[0][1]: 10
Enter a[0][2]: 30
Enter a[1][0]: 34
Enter a[1][1]: 21
Enter a[1][2]: 34
```

Enter a[2][0]: 45
Enter a[2][1]: 56
Enter a[2][2]: 78

printing the elements

56	10	30
34	21	34
45	56	78

C Pointers

The pointer in C language is a variable which stores the address of another variable. This variable can be of type int, char, array, function, or any other pointer. The size of the pointer depends on the architecture. However, in 32-bit architecture the size of a pointer is 2 byte.

Consider the following example to define a pointer which stores the address of an integer.

1. `int n = 10;`
2. `int* p = &n; // Variable p of type pointer is pointing to the address of the variable n of type integer.`

Declaring a pointer

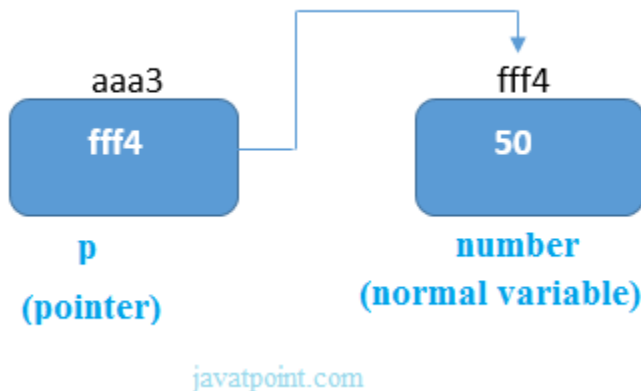
The pointer in c language can be declared using * (asterisk symbol). It is also known as indirection pointer used to dereference a pointer.

1. `int *a; //pointer to int`
2. `char *c; //pointer to char`

Pointer Example

An example of using pointers to print the address and value is given below.

ADVERTISEMENT



As you can see in the above figure, pointer variable stores the address of number variable, i.e., fff4. The value of number variable is 50. But the address of pointer variable p is aaa3.

By the help of * (**indirection operator**), we can print the value of pointer variable p.

Let's see the pointer example as explained for the above figure.

1. `#include<stdio.h>`
2. `int main(){`
3. `int number=50;`
4. `int *p;`
5. `p=&number;` //stores the address of number variable
6. `printf("Address of p variable is %x \n",p);` // p contains the address of the number therefore printing p gives the address of number.
7. `printf("Value of p variable is %d \n",*p);` // As we know that * is used to dereference a pointer therefore if we print *p, we will get the value stored at the address contained by p.
8. `return 0;`
9. `}`

Output

Address of number variable is fff4

Address of p variable is fff4

Value of p variable is 50

C Pointers

Pointers are powerful features of C and C++ programming. Before we learn pointers, let's learn about addresses in C programming.

Address in C

If you have a variable `var` in your program, `&var` will give you its address in the memory.

We have used address numerous times while using the `scanf()` function.

```
scanf("%d", &var);
```

Here, the value entered by the user is stored in the address of `var` variable.

Let's take a working example.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int var = 5;
```

```
    printf("var: %d\n", var);
```

```
// Notice the use of & before var

printf("address of var: %p", &var);

return 0;

}
```

Output

```
var: 5
address of var: 2686778
```

Note: You will probably get a different address when you run the above code.

C Pointers

Pointers (pointer variables) are special variables that are used to store addresses rather than values.

Pointer Syntax

Here is how we can declare pointers.

```
int* p;
```

Here, we have declared a pointer `p` of `int` type.

You can also declare pointers in these ways.

```
int *p1;
```

```
int * p2;
```

Let's take another example of declaring pointers.

```
int* p1, p2;
```

Here, we have declared a pointer `p1` and a normal variable `p2`.

Assigning addresses to Pointers

Let's take an example.

```
int* pc, c;
```

```
c = 5;
```

```
pc = &c;
```

Here, 5 is assigned to the `c` variable. And, the address of `c` is assigned to the `pc` pointer.

Get Value of Thing Pointed by Pointers

To get the value of the thing pointed by the pointers, we use the `*` operator.

For example:

```
int* pc, c;
```

```
c = 5;
```

```
pc = &c;
```

```
printf("%d", *pc); // Output: 5
```

Here, the address of `c` is assigned to the `pc` pointer. To get the value stored in that address, we used `*pc`.

Note: In the above example, `pc` is a pointer, not `*pc`. You cannot and should not do something like `*pc = &c;`

By the way, `*` is called the dereference operator (when working with pointers). It operates on a pointer and gives the value stored in that pointer.

Changing Value Pointed by Pointers

Let's take an example.

```
int* pc, c;
```

```
c = 5;
```

```
pc = &c;
```

```
c = 1;
```

```
printf("%d", c); // Output: 1
```

```
printf("%d", *pc); // Ouptut: 1
```

We have assigned the address of `c` to the `pc` pointer.

Then, we changed the value of `c` to 1. Since `pc` and the address of `c` is the same, `*pc` gives us 1.

Let's take another example.

```
int* pc, c;  
  
c = 5;  
  
pc = &c;  
  
*pc = 1;  
  
printf("%d", *pc); // Ouptut: 1  
  
printf("%d", c);   // Output: 1
```

We have assigned the address of `c` to the `pc` pointer.

Then, we changed `*pc` to 1 using `*pc = 1;`. Since `pc` and the address of `c` is the same, `c` will be equal to 1.

Let's take one more example.

```
int* pc, c, d;  
  
c = 5;  
  
d = -15;  
  
pc = &c; printf("%d", *pc); // Output: 5  
pc = &d; printf("%d", *pc); // Ouptut: -15
```

Initially, the address of `c` is assigned to the `pc` pointer using `pc = &c;`. Since `c` is 5, `*pc` gives us 5.

Then, the address of `d` is assigned to the `pc` pointer using `pc = &d;`. Since `d` is -15, `*pc` gives us -15.

Example: Working of Pointers

Let's take a working example.

```
#include <stdio.h>

int main()
{
    int* pc, c;

    c = 22;

    printf("Address of c: %p\n", &c);

    printf("Value of c: %d\n\n", c); // 22

    pc = &c;

    printf("Address of pointer pc: %p\n", pc);

    printf("Content of pointer pc: %d\n\n", *pc); // 22

    c = 11;

    printf("Address of pointer pc: %p\n", pc);

    printf("Content of pointer pc: %d\n\n", *pc); // 11

    *pc = 2;

    printf("Address of c: %p\n", &c);
```

```
printf("Value of c: %d\n\n", c); // 2
```

```
return 0;
```

```
}
```

Output

```
Address of c: 2686784
```

```
Value of c: 22
```

```
Address of pointer pc: 2686784
```

```
Content of pointer pc: 22
```

```
Address of pointer pc: 2686784
```

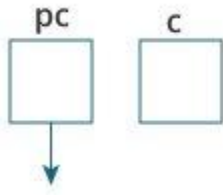
```
Content of pointer pc: 11
```

```
Address of c: 2686784
```

```
Value of c: 2
```

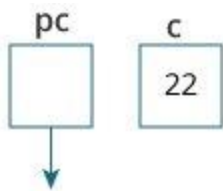
Explanation of the program

1. `int* pc, c;`



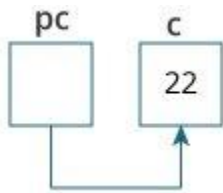
Here, a pointer `pc` and a normal variable `c`, both of type `int`, is created. Since `pc` and `c` are not initialized at initially, pointer `pc` points to either no address or a random address. And, variable `c` has an address but contains random garbage value.

2. `c = 22;`



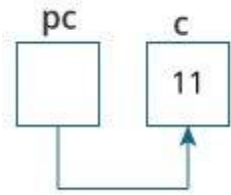
This assigns 22 to the variable `c`. That is, 22 is stored in the memory location of variable `c`.

3. `pc = &c;`



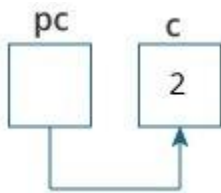
This assigns the address of variable `c` to the pointer `pc`.

4. `c = 11;`



This assigns 11 to variable `c`.

5. `*pc = 2;`



This change the value at the memory location pointed by the pointer `pc` to 2.

Common mistakes when working with pointers

Suppose, you want pointer `pc` to point to the address of `c`. Then,

```
int c, *pc;
```

```
// pc is address but c is not
```

```
pc = c; // Error
```

```
// &c is address but *pc is not
```

```
*pc = &c; // Error
```

```
// both &c and pc are addresses
```

```
pc = &c; // Not an error
```

```
// both c and *pc are values
```

```
*pc = c; // Not an error
```

Here's an example of pointer syntax beginners often find confusing.

```
#include <stdio.h>
```

```
int main() {
```

```
    int c = 5;
```

```
    int *p = &c;
```

```
    printf("%d", *p); // 5
```

```
    return 0;
```

```
}
```

Why didn't we get an error when using `int *p = &c;`?

It's because

```
int *p = &c;
```

is equivalent to

```
int *p;
```

```
p = &c;
```

In both cases, we are creating a pointer `p` (not `*p`) and assigning `&c` to it.

To avoid this confusion, we can use the statement like this:

```
int* p = &c;
```

Pointer to array

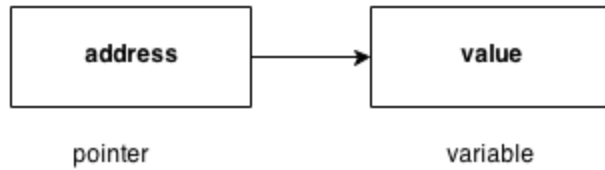
1. `int arr[10];`
2. `int *p[10]=&arr; // Variable p of type pointer is pointing to the address of an integer array arr.`

Pointer to a function

1. `void show (int);`
2. `void(*p)(int) = &display; // Pointer p is pointing to the address of a function`

Pointer to structure

1. `struct st {`
2. `int i;`
3. `float f;`
4. `}ref;`
5. `struct st *p = &ref;`



Advantage of pointer

- 1) Pointer **reduces the code** and **improves the performance**, it is used to retrieving strings, trees, etc. and used with arrays, structures, and functions.
- 2) We can **return multiple values from a function** using the pointer.
- 3) It makes you able to **access any memory location** in the computer's memory.

Usage of pointer

There are many applications of pointers in c language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using `malloc()` and `calloc()` functions where the pointer is used.

2) Arrays, Functions, and Structures

Pointers in c language are widely used in arrays, functions, and structures. It reduces the code and improves the performance.

Address Of (&) Operator

The address of operator '&' returns the address of a variable. But, we need to use %u to display the address of a variable.

1. `#include<stdio.h>`
2. `int main(){`
3. `int number=50;`
4. `printf("value of number is %d, address of number is %u",number,&number);`

5. `return 0;`
6. `}`

Output

value of number is 50, address of number is fff4

NULL Pointer

A pointer that is not assigned any value but NULL is known as the NULL pointer. If you don't have any address to be specified in the pointer at the time of declaration, you can assign NULL value. It will provide a better approach.

```
int *p=NULL;
```

In the most libraries, the value of the pointer is 0 (zero).

Pointer Program to swap two numbers without using the 3rd variable.

1. `#include<stdio.h>`
2. `int main(){`
3. `int a=10,b=20,*p1=&a,*p2=&b;`
- 4.
5. `printf("Before swap: *p1=%d *p2=%d",*p1,*p2);`
6. `*p1=*p1+*p2;`
7. `*p2=*p1-*p2;`
8. `*p1=*p1-*p2;`
9. `printf("\nAfter swap: *p1=%d *p2=%d",*p1,*p2);`
- 10.
11. `return 0;`

12.}

Output

Before swap: *p1=10 *p2=20

After swap: *p1=20 *p2=10

Reading complex pointers

There are several things which must be taken into the consideration while reading the complex pointers in C. Lets see the precedence and associativity of the operators which are used regarding pointers.

Operator	Precedence	Associativity
() , []	1	Left to right
*, identifier	2	Right to left
Data type	3	-

Here,we must notice that,

- `()`: This operator is a bracket operator used to declare and define the function.
- `[]`: This operator is an array subscript operator
- `*`: This operator is a pointer operator.
- Identifier: It is the name of the pointer. The priority will always be assigned to this.
- Data type: Data type is the type of the variable to which the pointer is intended to point. It also includes the modifier like signed int, long, etc).

How to read the pointer: `int (*p)[10]`.

To read the pointer, we must see that `()` and `[]` have the equal precedence. Therefore, their associativity must be considered here. The associativity is left to right, so the priority goes to `()`.

Inside the bracket `()`, pointer operator `*` and pointer name (identifier) `p` have the same precedence. Therefore, their associativity must be considered here which is right to left, so the priority goes to `p`, and the second priority goes to `*`.

Assign the 3rd priority to `[]` since the data type has the last precedence. Therefore the pointer will look like following.

- `char` -> 4
- `*` -> 2
- `p` -> 1
- `[10]` -> 3

The pointer will be read as `p` is a pointer to an array of integers of size 10.

Example

How to read the following pointer?

1. `int (*p)(int (*)[2], int (*)void)`

Explanation

This pointer will be read as `p` is a pointer to such function which accepts the first parameter as the pointer to a one-dimensional array of integers of size two and the second parameter as the pointer to a function which parameter is void and return type is the integer.

C File Handling

A file is a container in computer storage devices used for storing data.

Why files are needed?

- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- If you have to enter a large number of data, it will take a lot of time to enter them all.

However, if you have a file containing all the data, you can easily access the contents of the file using a few commands in C.

- You can easily move your data from one computer to another without any changes.

Types of Files

When dealing with files, there are two types of files you should know about:

1. Text files
2. Binary files

1. Text files

Text files are the normal .txt files. You can easily create text files using any simple text editors such as Notepad.

When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.

They take minimum effort to maintain, are easily readable, and provide the least security and takes bigger storage space.

2. Binary files

Binary files are mostly the .bin files in your computer.

Instead of storing data in plain text, they store it in the binary form (0's and 1's).

They can hold a higher amount of data, are not readable easily, and provides better security than text files.

File Operations

In C, you can perform four major operations on files, either text or binary:

1. Creating a new file
2. Opening an existing file
3. Closing a file
4. Reading from and writing information to a file

Working with files

When working with files, you need to declare a pointer of type file. This declaration is needed for communication between the file and the program.

```
FILE *fptr;
```

Opening a file - for creation and edit

Opening a file is performed using the `fopen()` function defined in the `stdio.h` header file.

The syntax for opening a file in standard I/O is:

```
ptr = fopen("filename", "mode");
```

For example,

```
fopen("E:\\cprogram\\newprogram.txt", "w");
```

```
fopen("E:\\cprogram\\oldprogram.bin", "rb");
```

- Let's suppose the file `newprogram.txt` doesn't exist in the location `E:\\cprogram`. The first function creates a new file named `newprogram.txt` and opens it for writing as per the mode 'w'.
The writing mode allows you to create and edit (overwrite) the contents of the file.
- Now let's suppose the second binary file `oldprogram.bin` exists in the location `E:\\cprogram`. The second function opens the existing file for reading in binary mode 'rb'.
The reading mode only allows you to read the file, you cannot write into the file.

Mode	Meaning of Mode	During Inexistence of file
r	Open for reading.	If the file does not exist, <code>fopen()</code> returns NULL.
rb	Open for reading in binary mode.	If the file does not exist, <code>fopen()</code> returns NULL.
w	Open for writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb	Open for writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a	Open for append.	If the file does not exist, it will be created.

	Data is added to the end of the file.	
ab	Open for append in binary mode.	If the file does not exist, it will be created.
	Data is added to the end of the file.	
r+	Open for both reading and writing.	If the file does not exist, <code>fopen()</code> returns NULL.
rb+	Open for both reading and writing in binary mode.	If the file does not exist, <code>fopen()</code> returns NULL.
w+	Open for both reading and writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb+	Open for both reading and writing in binary mode.	If the file exists, its contents are overwritten.

		If the file does not exist, it will be created.
a+	Open for both reading and appending.	If the file does not exist, it will be created.
ab+	Open for both reading and appending in binary mode.	If the file does not exist, it will be created.

Closing a File

The file (both text and binary) should be closed after reading/writing.

Closing a file is performed using the `fclose()` function.

```
fclose(fp_ptr);
```

Here, `fp_ptr` is a file pointer associated with the file to be closed.

Reading and writing to a text file

For reading and writing to a text file, we use the functions `fprintf()` and `fscanf()`.

They are just the file versions of `printf()` and `scanf()`. The only difference is that `fprintf()` and `fscanf()` expects a pointer to the structure `FILE`.

Example 1: Write to a text file

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num;
    FILE *fptr;

    // use appropriate location if you are using MacOS or Linux
    fptr = fopen("C:\\program.txt", "w");

    if (fptr == NULL)
    {
        printf("Error!");
        exit(1);
    }

    printf("Enter num: ");
    scanf("%d", &num);

    fprintf(fptr, "%d", num);
    fclose(fptr);

    return 0;
}
```

This program takes a number from the user and stores in the file `program.txt`.

After you compile and run this program, you can see a text file `program.txt` created in C drive of your computer. When you open the file, you can see the integer you entered.

Example 2: Read from a text file

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int num;
    FILE *fptr;

    if ((fptr = fopen("C:\\program.txt", "r")) == NULL) {
        printf("Error! opening file");

        // Program exits if the file pointer returns NULL.
        exit(1);
    }

    fscanf(fptr, "%d", &num);

    printf("Value of n=%d", num);
    fclose(fptr);

    return 0;
}
```

This program reads the integer present in the `program.txt` file and prints it onto the screen.

If you successfully created the file from Example 1, running this program will get you the integer you entered.

Other functions like `fgetchar()`, `fputc()` etc. can be used in a similar way.

Reading and writing to a binary file

Functions `fread()` and `fwrite()` are used for reading from and writing to a file on the disk respectively in case of binary files.

Writing to a binary file

To write into a binary file, you need to use the `fwrite()` function. The functions take four arguments:

1. address of data to be written in the disk
2. size of data to be written in the disk
3. number of such type of data
4. pointer to the file where you want to write.

```
fwrite(addressData, sizeData, numbersData, pointerToFile);
```

Example 3: Write to a binary file using fwrite()

```
#include <stdio.h>
#include <stdlib.h>

struct threeNum
{
    int n1, n2, n3;
};

int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;

    if ((fptr = fopen("C:\\program.bin", "wb")) == NULL) {
        printf("Error! opening file");
    }

    // Program exits if the file pointer returns NULL.
    exit(1);
}

for(n = 1; n < 5; ++n)
{
    num.n1 = n;
    num.n2 = 5*n;
    num.n3 = 5*n + 1;
    fwrite(&num, sizeof(struct threeNum), 1, fptr);
}
```

```
}  
fclose(fptr);  
  
return 0;  
}
```

In this program, we create a new file `program.bin` in the C drive.

We declare a structure `threeNum` with three numbers - `n1`, `n2` and `n3`, and define it in the main function as `num`.

Now, inside the for loop, we store the value into the file using `fwrite()`.

The first parameter takes the address of `num` and the second parameter takes the size of the structure `threeNum`.

Since we're only inserting one instance of `num`, the third parameter is `1`. And, the last parameter `*fptr` points to the file we're storing the data.

Finally, we close the file.

Reading from a binary file

Function `fread()` also take 4 arguments similar to the `fwrite()` function as above.

```
fread(addressData, sizeData, numbersData, pointerToFile);
```

Example 4: Read from a binary file using `fread()`

```
#include <stdio.h>  
#include <stdlib.h>
```

```

struct threeNum
{
    int n1, n2, n3;
};

int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;

    if ((fptr = fopen("C:\\program.bin", "rb")) == NULL) {
        printf("Error! opening file");

        // Program exits if the file pointer returns NULL.
        exit(1);
    }

    for(n = 1; n < 5; ++n)
    {
        fread(&num, sizeof(struct threeNum), 1, fptr);
        printf("n1: %d\tn2: %d\tn3: %d\n", num.n1, num.n2, num.n3);
    }
    fclose(fptr);

    return 0;
}

```

In this program, you read the same file `program.bin` and loop through the records one by one.

In simple terms, you read one `threeNum` record of `threeNum` size from the file pointed by `*fptr` into the structure `num`.

You'll get the same records you inserted in Example 3.

Getting data using `fseek()`

If you have many records inside a file and need to access a record at a specific position, you need to loop through all the records before it to get the record.

This will waste a lot of memory and operation time. An easier way to get to the required data can be achieved using `fseek()`.

As the name suggests, `fseek()` seeks the cursor to the given record in the file.

Syntax of `fseek()`

```
fseek(FILE * stream, long int offset, int whence);
```

The first parameter `stream` is the pointer to the file. The second parameter is the position of the record to be found, and the third parameter specifies the location where the offset starts.

Whence	Meaning
<code>SEEK_SET</code>	Starts the offset from the beginning of the file.
<code>SEEK_END</code>	Starts the offset from the end of the file.
<code>SEEK_CUR</code>	Starts the offset from the current location of the cursor in the file.

Example 5: `fseek()`

```

#include <stdio.h>
#include <stdlib.h>

struct threeNum
{
    int n1, n2, n3;
};

int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;

    if ((fptr = fopen("C:\\program.bin", "rb")) == NULL) {
        printf("Error! opening file");

        // Program exits if the file pointer returns NULL.
        exit(1);
    }

    // Moves the cursor to the end of the file
    fseek(fptr, -sizeof(struct threeNum), SEEK_END);

    for(n = 1; n < 5; ++n)
    {
        fread(&num, sizeof(struct threeNum), 1, fptr);
        printf("n1: %d\tn2: %d\tn3: %d\n", num.n1, num.n2, num.n3);
        fseek(fptr, -2*sizeof(struct threeNum), SEEK_CUR);
    }
    fclose(fptr);

    return 0;
}

```

This program will start reading the records from the file `program.bin` in the reverse order (last to first) and prints it.

FUNCTIONS IN C

FUNCTIONS IN C

1. What is a Function?

A **function** is a block of code that performs a specific task.

In simple words:

A function is a **small program inside your program**

2. Why Do We Need Functions?

Without functions:

```
#include <stdio.h>
```

```
int main() {  
    int a = 5, b = 3;  
    printf("Sum = %d\n", a + b);  
  
    int x = 10, y = 2;  
    printf("Sum = %d\n", x + y);  
  
    return 0;  
}
```

Same logic repeated again and again ❌

Problems:

- Code repetition
- Hard to manage
- More chances of mistakes

Solution: Function

We write logic once and reuse it.

3. Structure of Function (VERY IMPORTANT)

```
return_type function_name(parameters) {  
    // statements  
    return value;  
}
```

Understanding each part

Part	Meaning
return_type	What function returns (int, float, void)
function_name	Name of function
parameters	Inputs
return	Output

4. MOST IMPORTANT: HOW FUNCTION WORKS

Let's learn using a real example step-by-step.

Example: Add Two Numbers

Step 1: Function Definition

```
int add(int a, int b) {  
    int sum = a + b;  
    return sum;  
}
```

This defines what the function does

Step 2: Function Call (VERY IMPORTANT)

```
int result = add(5, 3);
```

This line:

- Sends values (5, 3)
 - Runs function
 - Gets result
-

Step 3: Complete Program

```
#include <stdio.h>
```

```
// function definition  
int add(int a, int b) {  
    int sum = a + b;  
    return sum;  
}
```

```
int main() {
```

```
    int result;
```

```
    // function call  
    result = add(5, 3);
```

```
    printf("Sum = %d\n", result);
```

```
    return 0;
```

```
}
```

CONTROL FLOW (IMPORTANT)

main() starts

↓

add(5,3) is called

↓

control goes to function

↓

function executes

↓

return value

↓

back to main()

↓

print result

5. Types of Functions (Basic Understanding)

Type 1: No input, no output

```
#include <stdio.h>
```

```
void greet() {  
    printf("Hello\n");  
}
```

```
int main() {  
    greet(); // function call  
}
```

Type 2: Input, no output

```
#include <stdio.h>

void showSum(int a, int b) {
    printf("Sum = %d\n", a + b);
}

int main() {
    showSum(5, 3); // function call
}
```

Type 3: No input, with output

```
#include <stdio.h>

int getNumber() {
    return 10;
}

int main() {
    int x = getNumber(); // function call
    printf("%d", x);
}
```

Type 4: Input and output (MOST IMPORTANT)

```
#include <stdio.h>

int multiply(int a, int b) {
    return a * b;
}

int main() {
    int result = multiply(4, 5); // function call
    printf("%d", result);
}
```

6. Passing Values to Function

Call by Value

```
#include <stdio.h>
```

```
void change(int x) {  
    x = 100;  
}
```

```
int main() {  
    int a = 10;  
  
    change(a); // function call  
  
    printf("%d", a); // 10 (no change)  
}
```

Only copy is sent

Call by Reference (Using Pointer)

```
#include <stdio.h>
```

```
void change(int *x) {  
    *x = 100;  
}
```

```
int main() {  
    int a = 10;  
  
    change(&a); // function call  
  
    printf("%d", a); // 100  
}
```

👉 Original value changes



7. Simple Recursion (Basic Idea)

```
#include <stdio.h>
```

```
int factorial(int n) {  
    if(n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

```
int main() {  
    int result = factorial(5); // function call  
    printf("%d", result);  
}
```



8. Common Mistakes

- Forgetting to call function
 - Wrong return type
 - Not using return
 - Infinite recursion
-



9. Practice Exercises (WITH FULL ANSWERS)

♦ Exercise 1: Even or Odd

```
#include <stdio.h>
```

```
void check(int n) {  
    if(n % 2 == 0)
```

```
        printf("Even\n");
    else
        printf("Odd\n");
}

int main() {
    check(7); // function call
}
```

◆ Exercise 2: Square

```
#include <stdio.h>

int square(int n) {
    return n * n;
}

int main() {
    int result = square(4); // function call
    printf("%d", result);
}
```

◆ Exercise 3: Maximum of 2 numbers

```
#include <stdio.h>

int max(int a, int b) {
    if(a > b)
        return a;
    else
        return b;
}

int main() {
    int result = max(10, 20); // function call
    printf("%d", result);
}
```

◆ Exercise 4: Swap (Basic idea)

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
int main() {  
    int x = 5, y = 10;  
  
    swap(&x, &y); // function call  
  
    printf("%d %d", x, y);  
}
```

STRUCTURE IN C

STRUCTURE IN C

1. What is a Structure?

A **structure** in C is a user-defined data type that allows you to store **different types of data** together.

Example:

A student has:

- Name (string)
- Roll number (int)
- Marks (float)

Instead of using separate variables, we can group them using a structure.

2. Why Do We Need Structure?

Without structure ❌

```
char name[50];  
int roll;  
float marks;
```

Problem:

- Difficult to manage many students
- No grouping of related data

With structure ✅

```
struct Student {  
    char name[50];  
    int roll;  
    float marks;  
};
```

Advantage:

- Data is organized
 - Easy to handle multiple records
-

3. Syntax of Structure

```
struct structure_name {  
    data_type member1;  
    data_type member2;  
    ...  
};
```

4. Example: Declaring Structure

```
#include <stdio.h>  
  
struct Student {  
    char name[50];  
    int roll;  
    float marks;  
};  
  
int main() {  
    struct Student s1;  
  
    return 0;  
}
```

5. Accessing Structure Members

We use **dot operator** (.)

Example:

```
#include <stdio.h>
```

```
struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s1;

    s1.roll = 1;
    s1.marks = 85.5;

    printf("Roll = %d\n", s1.roll);
    printf("Marks = %.2f", s1.marks);

    return 0;
}
```

6. Taking Input in Structure

```
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s1;

    printf("Enter name: ");
    scanf("%s", s1.name);

    printf("Enter roll: ");
    scanf("%d", &s1.roll);

    printf("Enter marks: ");
    scanf("%f", &s1.marks);

    printf("\nStudent Details:\n");
    printf("Name: %s\n", s1.name);
```

```
printf("Roll: %d\n", s1.roll);
printf("Marks: %.2f\n", s1.marks);

return 0;
}
```

7. Structure with Multiple Variables

```
#include <stdio.h>
```

```
struct Student {
    char name[50];
    int roll;
    float marks;
};
```

```
int main() {
    struct Student s1, s2;
```

```
    // First student
    s1.roll = 1;
    s1.marks = 80;
```

```
    // Second student
    s2.roll = 2;
    s2.marks = 90;
```

```
    printf("Student 1 Marks = %.2f\n", s1.marks);
    printf("Student 2 Marks = %.2f", s2.marks);
```

```
    return 0;
}
```

8. Array of Structures

Used to store multiple records

```
#include <stdio.h>
```

```
struct Student {
```

```

    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s[2];

    for(int i = 0; i < 2; i++) {
        printf("Enter name: ");
        scanf("%s", s[i].name);

        printf("Enter roll: ");
        scanf("%d", &s[i].roll);

        printf("Enter marks: ");
        scanf("%f", &s[i].marks);
    }

    printf("\nStudent Details:\n");

    for(int i = 0; i < 2; i++) {
        printf("\nName: %s", s[i].name);
        printf("\nRoll: %d", s[i].roll);
        printf("\nMarks: %.2f\n", s[i].marks);
    }

    return 0;
}

```

9. Structure Initialization

```

#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s1 = {"John", 1, 85.5};
}

```

```
printf("Name: %s\n", s1.name);
printf("Roll: %d\n", s1.roll);
printf("Marks: %.2f", s1.marks);

return 0;
}
```

10. Practice Exercises

Exercise 1

Create a structure `Employee` with name, id, salary and display details.

✓ Answer:

```
#include <stdio.h>

struct Employee {
    char name[50];
    int id;
    float salary;
};

int main() {
    struct Employee e1;

    printf("Enter name: ");
    scanf("%s", e1.name);

    printf("Enter ID: ");
    scanf("%d", &e1.id);

    printf("Enter salary: ");
    scanf("%f", &e1.salary);

    printf("\nEmployee Details:\n");
    printf("Name: %s\n", e1.name);
    printf("ID: %d\n", e1.id);
    printf("Salary: %.2f\n", e1.salary);
}
```

```
    return 0;
}
```

Exercise 2

Store details of 3 students using array of structures.

✓ Answer:

```
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s[3];

    for(int i = 0; i < 3; i++) {
        printf("Enter name: ");
        scanf("%s", s[i].name);

        printf("Enter roll: ");
        scanf("%d", &s[i].roll);

        printf("Enter marks: ");
        scanf("%f", &s[i].marks);
    }

    printf("\nStudent Details:\n");

    for(int i = 0; i < 3; i++) {
        printf("\nName: %s", s[i].name);
        printf("\nRoll: %d", s[i].roll);
        printf("\nMarks: %.2f\n", s[i].marks);
    }

    return 0;
}
```

Important Points

- Structure groups **different data types**
 - Access members using **dot (.) operator**
 - Use **array of structures** for multiple records
 - Always use **&** for non-string input in **scanf**
-

C exercises

C exercises that cover **loops, arrays, structures, conditions, pointers, and functions**.

1. Even or Odd (Conditions + Function)

Problem:

Check whether a number is even or odd using a function.

```
#include <stdio.h>

void checkEvenOdd(int num) {
    if(num % 2 == 0)
        printf("Even");
    else
        printf("Odd");
}

int main() {
    int n;
    printf("Enter number: ");
    scanf("%d", &n);
    checkEvenOdd(n);
    return 0;
}
```

2. Sum of First N Numbers (Loop)

Problem:

Find sum of first N natural numbers using loop.

```
#include <stdio.h>

int main() {
    int n, sum = 0;
    printf("Enter n: ");
    scanf("%d", &n);

    for(int i = 1; i <= n; i++) {
        sum += i;
    }
}
```

```
}

printf("Sum = %d", sum);
return 0;
}
```

3. Factorial Using Function (Loop + Function)

Problem:

Find factorial of a number using function.

```
#include <stdio.h>

int factorial(int n) {
    int fact = 1;
    for(int i = 1; i <= n; i++)
        fact *= i;
    return fact;
}

int main() {
    int n;
    printf("Enter number: ");
    scanf("%d", &n);

    printf("Factorial = %d", factorial(n));
    return 0;
}
```

4. Reverse an Array (Array + Loop)

Problem:

Reverse elements of an array.

```
#include <stdio.h>
```

```
int main() {
    int arr[5] = {1,2,3,4,5};

    for(int i = 4; i >= 0; i--) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

5. Largest Element in Array (Array + Condition)

Problem:

Find the largest number in an array.

```
#include <stdio.h>
```

```
int main() {
    int arr[5] = {10, 25, 5, 40, 15};
    int max = arr[0];

    for(int i = 1; i < 5; i++) {
        if(arr[i] > max)
            max = arr[i];
    }

    printf("Largest = %d", max);
    return 0;
}
```

6. Swap Two Numbers (Pointers)

Problem:

Swap two numbers using pointers.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;

    swap(&x, &y);

    printf("x = %d, y = %d", x, y);
    return 0;
}
```

7. Student Structure (Structure)

Problem:

Store and display student details.

```
#include <stdio.h>

struct Student {
    int id;
    char name[20];
    float marks;
};

int main() {
    struct Student s;

    printf("Enter ID Name Marks: ");
    scanf("%d %s %f", &s.id, s.name, &s.marks);

    printf("ID: %d\nName: %s\nMarks: %.2f", s.id, s.name, s.marks);

    return 0;
}
```

8. Sum of Array Using Pointer (Array + Pointer)

Problem:

Find sum of array elements using pointer.

```
#include <stdio.h>
```

```
int main() {  
    int arr[5] = {1,2,3,4,5};  
    int *ptr = arr;  
    int sum = 0;  
  
    for(int i = 0; i < 5; i++) {  
        sum += *(ptr + i);  
    }  
  
    printf("Sum = %d", sum);  
    return 0;  
}
```

9. Count Vowels in String (Array + Loop + Condition)

Problem:

Count vowels in a string.

```
#include <stdio.h>
```

```
int main() {  
    char str[50];  
    int count = 0;  
  
    printf("Enter string: ");  
    scanf("%s", str);
```

```

for(int i = 0; str[i] != '\0'; i++) {
    if(str[i]=='a' || str[i]=='e' || str[i]=='i' || str[i]=='o' || str[i]=='u' ||
       str[i]=='A' || str[i]=='E' || str[i]=='I' || str[i]=='O' || str[i]=='U') {
        count++;
    }
}

printf("Vowels = %d", count);
return 0;
}

```

10. Structure with Function (Structure + Function)

Problem:

Pass structure to function and display details.

```
#include <stdio.h>
```

```

struct Employee {
    int id;
    float salary;
};

```

```

void display(struct Employee e) {
    printf("ID: %d\nSalary: %.2f", e.id, e.salary);
}

```

```

int main() {
    struct Employee e1;

    printf("Enter ID and Salary: ");
    scanf("%d %f", &e1.id, &e1.salary);

    display(e1);

    return 0;
}

```

What These Cover

- ✓ Conditions (**if**)
 - ✓ Loops (**for**)
 - ✓ Arrays & Strings
 - ✓ Functions
 - ✓ Pointers
 - ✓ Structures
-